VEL
SOFTWARE SOLUTIONS

QUALITY ENGINEERING DEPARTMENT

Software Testing & QA Automation Department Handbook

From zero to professional: onboarding, training, client QA delivery, automation, performance, costing, metrics and release sign-off

Issued by **VEL Software Solutions**

velsoftwaresolutions.in@gmail.com

Document VEL-QA-001 · Version 1.0 (draft for management approval) · Issued 4 November 2026

Tool versions, licences, limits and prices change. Always verify against official documentation and vendor pricing before training, building or quoting. Confidential · internal use.

How to Use This Handbook

This document is the single operating manual for the Software Testing & QA Automation Department. It does two jobs: **(1)** it trains a beginner from zero to a professional who can join supervised client projects, and **(2)** it defines the mandatory 16-stage QA lifecycle every client project follows, with procedures, templates, evidence requirements and approval gates.

GOLDEN RULES

1. Do not skip stages. Do not start a stage until the previous stage's approval gate is signed.
2. **Evidence or it did not happen.** Every pass, fail, blocked or skipped result needs reproducible evidence (screenshot, log, HAR file, report or recording) stored in the project repository.
3. Test only systems you are **authorised in writing** to test. Never test production or third-party systems without the client's written approval.
4. Security testing is limited to authorised assessment activity and is coordinated with qualified security personnel. Testers report suspected vulnerabilities; they do not exploit them.
5. Never commit passwords, API keys, tokens, customer personal data or production database extracts.
6. Never manipulate, round up, or selectively report metrics to make a project look successful. Report unresolved risk honestly.
7. Never claim software is “bug-free”, “secure” or “guaranteed”. Testing reduces risk; it cannot prove the absence of defects.
8. Every release recommendation needs a second reviewer (QA Lead) before it reaches the client.

Reading path for a beginner

Part A: Learn (Levels 1–6) → Part B: Tools → Part C: Client lifecycle (16 stages) →
Part D: Repository, documents, metrics → Part E: Costing & roles → Part F: Projects, plan, assessment

The teaching method used for every topic

Each learning topic is delivered in the same ten parts so that nothing is left to chance. Trainers must not mark a topic complete until all ten parts are done.

#	Part	What the trainee must receive or produce
1	Definition	A one- or two-sentence plain-language definition, plus the formal term (ISTQB glossary where one exists).
2	Purpose	Why the topic matters to a client and what risk it reduces.
3	Setup	Tools, accounts, test data and environment needed before starting.
4	Procedure	Numbered steps the trainee follows exactly.

#	Part	What the trainee must receive or produce
5	Expected output	What the finished work must look like (file, report, screenshot, command output).
6	Verification	How the trainee proves the work is correct (self-check list or peer check).
7	Errors	The most common mistakes and how to recognise and fix them.
8	Reporting	How and where the result is recorded (template, repo path, defect tracker).
9	Assignment	A practical task, time-boxed, with deliverables.
10	Assessment	Pass criteria: a quiz, a demonstration, or a reviewed deliverable.

Contents

1. How to Use This Handbook
2. Company Service Portfolio
3. Beginner Onboarding
4. First Manual-Testing Assignment (A0)
5. Level 1 — Software Testing Fundamentals
6. Level 2 — Manual Testing
7. Level 3 — Web and Mobile Testing
8. Level 4 — API Testing
9. Level 5 — Automation Testing
10. Level 6 — Advanced QA
11. Tools We Teach and Use
12. Mandatory Client QA Lifecycle: Stages 01–16
13. Professional QA Repository Structure
14. Test Documentation Standards
15. QA Metrics
16. Costing and Pricing
17. Employee Roles
18. Practical Employee Projects
19. 90-Day Employee Training Plan
20. Assessment and Competency Framework
21. Templates and QA Sign-off Checklist
22. Appendices

Document control

Item	Value
Document	VEL-QA-001 · Software Testing & QA Automation Department Handbook
Version / status	1.0 · draft for management approval
Owner	Head of QA (to be assigned by management)
Approver	Managing Director / Delivery Director
Review cycle	Every 6 months, and whenever a tool, licence or standard referenced here changes materially
Companion files	90-Day Training Plan (VEL-QA-002) · Templates Pack (VEL-QA-003) · Practical Workbook (VEL-QA-004) · Assessment Framework (VEL-QA-005) · `qa-project-template/` repository skeleton and practice targets

VERIFY BEFORE YOU RELY

Tool versions, licences, free-tier limits and prices change. Version numbers and licences quoted in this handbook were checked against the npm and PyPI registries on 4 November 2026 where marked “registry-verified”; everything else must be re-verified on the vendor's official page before use or before quoting a client.

1. Company Service Portfolio

Seventeen commercially deliverable services. Each can be sold alone or combined into a project, a fixed-scope package, a time-based engagement or a monthly retainer (see Costing and Pricing).

Service	What we deliver	Typical client trigger	Primary roles
Manual software testing	Risk-based test design, scripted and exploratory execution, evidence capture, defect reporting.	"We release and hope." / "Customers find our bugs."	Manual QA, Junior QA
Web application testing	Functional, usability, session, form and navigation testing across supported browsers and viewports.	"The site breaks on some phones/browsers."	Manual QA, Automation QA
Mobile application testing	Android/iOS functional testing on real devices and emulators; OS-version, interruption and network-condition testing.	"The app crashes on certain devices."	Manual QA, Automation QA
Desktop application testing	Installer, upgrade, UI, file-handling and OS-compatibility testing for Windows/macOS/Linux applications.	"Our installer fails on some PCs."	Manual QA
API testing	Request/response, status-code, authentication, schema and negative testing; contract checks.	"Integrations fail in production."	API Tester
Database testing	Data integrity, migration, constraints, transactions and report-query validation on test data only.	"Numbers in reports do not match."	Manual QA, API Tester
Functional testing	Verification of each requirement and user story against acceptance criteria.	"Does it do what we specified?"	Manual QA
Regression testing	Repeatable suites (manual + automated) proving existing behaviour survives change.	"Every release breaks something old."	Manual QA, Automation QA
Integration testing	Data exchange, interface contracts, error handling and dependency behaviour between modules/systems.	"It works alone but not together."	API Tester, Automation QA
End-to-end testing	Business-critical journeys across UI, API, data and third-party services.	"Checkout/booking/payroll must never fail."	Automation QA
Cross-browser testing	Chromium, Firefox, WebKit and agreed real-browser/OS matrix.	"Looks fine on my machine."	Manual QA, Automation QA
Performance testing	Workload modelling, baseline, load, stress, soak and spike tests with agreed thresholds in controlled environments.	"Will it survive launch day?"	Performance Tester

Service	What we deliver	Typical client trigger	Primary roles
Load and stress testing	Finds capacity limits and failure modes; reports throughput, latency percentiles, error rate and saturation.	"How many users can we handle?"	Performance Tester
Accessibility testing	WCAG-based automated scans plus manual keyboard and screen-reader checks.	"We must meet accessibility requirements."	Manual QA, Automation QA
Test automation	Playwright/Selenium/Cypress UI suites, pytest/Postman API suites, CI integration, reporting.	"Regression takes a week by hand."	Automation QA
Release validation	Entry/exit criteria checks, QA sign-off, risk summary, production smoke test.	"Can we ship on Friday?"	QA Lead, Release Coordinator
QA consulting	Process assessment, test strategy, tool selection, team coaching, metrics design.	"We have no QA process."	QA Lead, Test Manager

SECURITY TESTING BOUNDARY

We perform **authorised assessment activity only** and coordinate it with qualified security personnel (client security team or a certified penetration-testing partner). Testers may run agreed, non-destructive checks (for example verifying that an API rejects a missing token, or that error pages do not leak stack traces) and must escalate anything that looks like a vulnerability. We do not run exploit tools, brute-force, denial-of-service or scanning against systems without a signed scope, written authorisation and a named security coordinator.

WORDING WE USE WITH CLIENTS

"Quality assurance support", "evidence-based release recommendation", "risk summary". We never certify a product as defect-free, compliant or secure, and we never guarantee that a release will have no production incidents. Certification and compliance attestation are performed by authorised auditors.

Service families and how they combine

Family	Services	Typical package
Core functional QA	Manual, web, mobile, desktop, functional, regression	Project-based or fixed-scope; 2–8 weeks
Technical QA	API, database, integration, end-to-end	Fixed-scope package alongside core QA
Automation engineering	Test automation, cross-browser, CI integration	Project + maintenance retainer
Non-functional QA	Performance, load/stress, accessibility	Time-boxed engagement with a defined workload model
Governance & advisory	Release validation, QA consulting	Retainer or day-rate

2. Beginner Onboarding

Day-one programme for a new QA trainee. By the end of week one the trainee has a working machine, a learning repository, has run the practice targets, and has met every working agreement.

Day 1 agenda

Time	Activity	Output / evidence
09:30	Welcome, department purpose, services, who is who (QA Lead, Test Manager, Release Coordinator)	Trainee knows their buddy and escalation path
10:30	Read and sign: confidentiality, acceptable use, test-data and evidence-integrity policy	Signed policy acknowledgements filed by HR
11:30	Accounts: company e-mail, GitHub organisation invite, issue tracker, test-management tool	Access checklist ticked (see below)
13:30	Workstation setup (table below)	Screenshot of every verification command
15:00	Clone the learning repository; run the VEL Practice Shop and Practice API locally	Two screenshots: app in browser, `GET /api/health` response
16:30	Read the Golden Rules and the lifecycle overview; first 1-to-1 with the buddy	Three questions written in the learning journal

Access checklist

- [ ] Company e-mail and calendar working; two-factor authentication enabled.
- [ ] GitHub account added to the company organisation with the **QA-Trainee** team role (read on client repos, write only to the learning repository).
- [ ] Issue tracker (Jira or equivalent) account; can open the **QA-TRAINING** project.
- [ ] Test-management tool (TestRail, Zephyr, Xray or Kiwi TCMS) account in the training project.
- [ ] Password manager installed. **No** shared passwords, ever.
- [ ] Access to client systems: **none** until the Test Manager issues written authorisation naming the person, the system and the dates.

Workstation setup and verification

Install each tool from the official site, then run the verification command. Paste the output into `journal/day-01-setup.md`. Exact version numbers will differ; the major version must satisfy the “Expected” column.

Tool	Verification command	Expected output	Used for
Git	`git --version`	`git version 2.x`	Version control for all QA assets
Python	`python3 --version`	`Python 3.11` or newer	pytest, API automation, utilities
Node.js (LTS)	`node --version`	`v20.x` or newer LTS	Playwright, Newman, Lighthouse
VS Code (or equivalent)	`code --version`	A version number	Editing Markdown, code, YAML
Chrome / Edge + Firefox	Open `about:version` / `about:support`	Browser version page	Manual and cross-browser testing
Browser developer tools	Press `F12`, open **Network** tab	Requests listed when you reload a page	Evidence, API inspection

```bash
# From the qa-project-template/ folder of your learning repository
python3 -m http.server 8000 --directory practice-app    # terminal 1
python3 practice-api/server.py --port 8080               # terminal 2

# Verify (terminal 3)
curl -s http://localhost:8080/api/health
# Expected: {"status":"ok","service":"vel-practice-api"}
# Open http://localhost:8000 in the browser: the VEL Practice Shop login view appears.
```

Working agreements

Topic	Rule	Why
Evidence integrity	Never edit a screenshot, log or recording. If you must redact personal data, keep the original in the restricted evidence store and note the redaction.	Evidence that cannot be trusted is worthless to the client.
Honest reporting	Report what happened, including your own mistakes and tests you could not finish.	A hidden gap is a production risk.
Confidentiality	No client data in personal e-mail, chat tools, public AI services or public repositories.	Contractual and legal exposure.
Test data	Use synthetic data only. Production extracts and real personal data are forbidden.	Privacy law and client contract.
Authorisation	Test only what is written in the scope. Out-of-scope observations are reported, not explored.	Unauthorised testing can cause outages and legal harm.
Asking for help	Spend a maximum of 30 minutes stuck, then ask your buddy. Write down what you tried.	Speed with learning.

Topic	Rule	Why
Daily rhythm	09:45 stand-up, 17:30 learning-journal entry (what I did, what I learned, what blocks me).	Visible progress and early warning.

Naming conventions used from day one

Item	Pattern	Example
Requirement	`REQ-<nnn>`	`REQ-003`
Test scenario	`TS-<area>-<nn>`	`TS-REG-01`
Test case	`TC-<area>-<nnn>`	`TC-REG-004`
Defect	`DEF-<nnn>`	`DEF-017`
Evidence file	`<yyyy-mm-dd>_<TC-id>_<PASS|FAIL|BLOCKED>_<n>.<ext>`	`2026-11-06_TC-REG-004_FAIL_1.png`
Git branch	`qa/<initials>/<topic>`	`qa/ab/registration-tests`
Commit message	`<type>(<scope>): <summary>`	`test(registration): add boundary cases for REQ-003`

End-of-week-one checklist

- [ ] All tools verified and screenshots committed to `journal/`.
- [ ] Practice Shop and Practice API run locally; you can explain what each URL does.
- [ ] You can state the Golden Rules from memory and explain why each exists.
- [ ] Assignment A0 (next chapter) started: requirement table read, questions raised with the buddy.
- [ ] Level 1 quiz scheduled for the end of week two.

3. First Manual-Testing Assignment (A0)

Test the **VEL Practice Shop** registration and login flow against a written specification, produce a traceable test suite, execute it, log defects and report results. Time-box: **4 working days**.

> **WHY THIS ASSIGNMENT FIRST**
>
> It exercises the whole manual-testing chain in miniature: read a requirement, design cases, run them, capture evidence, report a defect, and summarise status. Every later project repeats this chain with more scale or more tooling.

Specification under test

ID	Requirement
REQ-001	Registration: **Full name** is required, 2–40 characters after trimming spaces.
REQ-002	Registration: **Email** is required and must be a valid address (`name@domain.tld`).
REQ-003	Registration: **Age** is required, a whole number from **18 to 60 inclusive**.
REQ-004	Registration: **Password** 8–20 characters with at least one digit and at least one uppercase letter.
REQ-005	Registration: **Confirm password** must match the password exactly (case-sensitive).
REQ-006	Registration: the **Terms** checkbox must be ticked.
REQ-007	A successful registration shows “Registration successful” and opens the login view.
REQ-008	Login with registered credentials opens the shop. Any failure shows the generic message “Invalid email or password”.
REQ-009	On the **3rd consecutive failed login** the account locks for 30 seconds and the message “Account locked. Try again in 30 seconds.” is shown; attempts during the lock show the same message.
REQ-010	Shop: quantity per item is a whole number **1–10**. Cart total = sum(price × quantity) shown to 2 decimals; GST 18 % shown to 2 decimals (standard rounding).
REQ-011	Session expires after **90 seconds of inactivity**: “Session expired. Please log in again.” and the login view opens.
REQ-012	Logout ends the session. Using the browser Back button afterwards must **not** show shop content.
REQ-013	The application is usable at **360 px** viewport width with no horizontal scrolling.
REQ-014	Accessibility: every input has a programmatic label; every image has alt text; all functions work by keyboard with a visible focus indicator.

Scope

In scope	Out of scope
Registration view, login view, shop view (cart, quantity, totals), session timeout, logout, viewport 360 px and 1280 px, keyboard access.	Performance, security exploitation, other devices, real e-mail delivery, payment. Report any security concern you notice to your buddy instead of probing it.

Procedure

Day	Steps	Deliverable
1	1. Read REQ-001 to REQ-014 twice; list ambiguities and unanswered questions. 2. Run the app; explore for 30 minutes without a script and note what you observe (exploratory notes). 3. Create the requirements traceability matrix with one row per requirement.	`docs/requirements-traceability.md` with all 14 rows; `docs/questions.md` ; exploratory notes
2	1. For each requirement apply **equivalence partitioning** and **boundary-value analysis** where it has ranges (REQ-001, 003, 004, 010). 2. Write at least **30 test cases** using the template: ID, title, requirement, preconditions, steps, test data, expected result, priority. 3. Include positive and negative cases; at least 6 for each of REQ-003 and REQ-004.	`test-cases/functional/registration-login.csv` (or tool export), 30+ cases, each linked to a REQ
3	1. Execute every case on Chrome and Firefox at 1280 px; REQ-013 at 360 px. 2. Record Pass / Fail / Blocked with evidence file names per the convention. 3. Raise a defect (template) the moment you confirm a failure; attempt to reproduce it three times.	Filled execution log; `evidence/` files; `defects/DEF-nnn.md` for each defect
4	1. Re-check traceability: every requirement has at least one executed case. 2. Complete the test execution report with counts, defect list, coverage and your own risk statement. 3. Present for 10 minutes to the QA Lead.	`docs/test-execution-report.md` ; review notes from the QA Lead

Worked example: one test case and one defect

Use these as the quality bar. Notice that the expected result is **observable and unambiguous**, and the defect states exactly what happened versus what was required.

Test case ID	`TC-REG-004` · Requirement: REQ-003 · Technique: boundary-value analysis · Priority: High
Title	Registration accepts age 18 (lower boundary)
Preconditions	Practice Shop open at the Register view; no account exists for `bva18@example.test` .
Steps	1. Enter name `Asha Rao` . 2. Enter email `bva18@example.test` .

	3. Enter age `18`. 4. Enter password `Passw0rdOK` and the same value in Confirm password. 5. Tick Terms. 6. Click **Register**.
Expected result	The message “Registration successful” is displayed and the Login view opens. No error is shown next to Age.
Actual result / status	Fill in during execution: **PASS** / **FAIL** / **BLOCKED** + evidence file name.

Defect ID / title	`DEF-001` · Age 18 rejected although REQ-003 allows 18 to 60 inclusive
Severity / priority	Major (valid users cannot register) / High
Environment	Practice Shop 1.0, Chrome 130 on Windows 11, viewport 1280×800, 2026-11-06
Steps to reproduce	1. Open Register view. 2. Fill all fields validly, using age `18`. 3. Tick Terms and click **Register**.
Expected	Registration succeeds (18 is the inclusive lower limit).
Actual	Age field shows an error; registration is blocked. Reproduced 3 of 3 attempts.
Evidence	`evidence/2026-11-06_TC-REG-004_FAIL_1.png`

Acceptance criteria and marking (100 points)

Criterion	Points	Full marks when
Traceability	15	All 14 requirements appear in the matrix and each links to at least one executed case.
Test design quality	25	Equivalence classes and boundaries are identified and named; cases are atomic, repeatable and have measurable expected results.
Execution discipline	20	Every case has a status and evidence; results are on two browsers; REQ-013 checked at 360 px.
Defect reports	25	Each defect is reproducible by someone else from the report alone; severity and priority are justified; no duplicates.
Report and honesty	15	Counts reconcile (pass + fail + blocked + not run = total); risks and untested areas are stated plainly.

PASS MARK: 70 / 100

Below 70 the trainee repeats the weakest part with feedback. The QA Lead keeps a hidden answer key of seeded defects (never shown to trainees). Finding more defects does not earn extra marks; **report quality** does.

4. Level 1 — Software Testing Fundamentals

The vocabulary and mental models every tester uses daily. Nothing in later levels makes sense without them.

Duration	About 2 weeks (10 working days)
Prerequisites	None. Basic computer use and English reading ability.
Outcome	Explains how software is built and tested, classifies a problem correctly as error, defect or failure, writes a test case and a bug report that others can follow, and applies severity and priority sensibly.
Exit gate	20-question quiz (pass ≥ 80 %) + one reviewed test case + one reviewed bug report, demonstrated to the QA Lead.

Software development lifecycle (SDLC) models

Model	How it works	Where testing happens	Tester's challenge
Waterfall	Phases run once in order: requirements → design → build → test → release.	Late, as one phase.	Defects found late are expensive; requirements may already be outdated.
V-model	Each development phase has a matching test level (requirements ↔ acceptance, design ↔ system, code ↔ unit).	Test design starts early, execution follows each build level.	Needs stable, well-written requirements.
Agile / Scrum	Short iterations (sprints) deliver small increments with continuous feedback.	Continuously, inside every sprint.	Little documentation; regression grows every sprint; automation becomes essential.
DevOps / CI-CD	Every commit is built, tested and deployed through an automated pipeline.	Automated gates on each commit plus exploratory testing.	Fast, reliable, non-flaky automation and production monitoring.

Software testing lifecycle (STLC)

Requirement analysis → Test planning → Test case design → Environment setup → Execution → Defect reporting → Closure

Each STLC phase has an **entry criterion** (what must be true to start) and an **exit criterion** (what must be true to finish). The client lifecycle later in this handbook expands STLC into 16 gated stages.

Key distinctions every trainee must be able to explain

Pair	Meaning	Example
QA vs QC	**Quality assurance** is process-oriented and prevents defects (standards, reviews, training). **Quality control** is product-oriented and detects defects (testing, inspection).	QA: introduce a code-review checklist. QC: run the regression suite before release.
Verification vs validation	**Verification**: are we building the product right (against specification)? **Validation**: are we building the right product (against user need)?	Verification: review the spec for contradictions. Validation: let a real user try the prototype.
Error → defect → failure	A human **error** introduces a **defect** (bug) in the work product; executing it may cause a **failure** (observable wrong behaviour).	Developer mistypes `>=` as `>` (error) → age check wrong (defect) → age 18 rejected at run time (failure).
Test scenario vs test case	A **scenario** is a high-level thing to test; a **test case** is a precise, repeatable set of steps, data and expected results.	Scenario: "Registration validates age". Cases: ages 17, 18, 60, 61, `abc`, empty.

The seven testing principles (ISTQB)

1. Testing shows the **presence** of defects, not their absence.
2. **Exhaustive testing is impossible**; use risk and priorities to focus effort.
3. **Early testing** saves time and money (review requirements before code exists).
4. **Defects cluster**: a few modules usually contain most defects.
5. **Pesticide paradox**: repeating the same tests stops finding new defects; refresh them.
6. Testing is **context dependent**: a medical device and a game need different testing.
7. **Absence-of-defects fallacy**: a defect-free system that does not meet user needs is still a failure.

Bug lifecycle

New → Assigned → In progress → Fixed → Ready for retest → Retest → Closed

State	Owner	Meaning and rule
New	Tester	Reported with full template. Tester has reproduced it at least twice.
Assigned / In progress	Developer lead → developer	Triage accepted; severity/priority agreed; fix started.
Fixed / Ready for retest	Developer	Fix merged to a named build; developer notes the build number and the area at risk.
Retest	Tester	Retest on the **fixed build** using the original steps, then run related regression.
Closed	Tester	Original failure no longer occurs and no new defect introduced. Only the tester closes.

State	Owner	Meaning and rule
Reopened	Tester	Fix did not work or regressed: attach new evidence and the build number.
Rejected / Not a bug	Developer or product owner	Behaviour is as specified or intended. The tester may challenge once with the requirement reference.
Duplicate	Triage	Link to the master defect; keep the clearest report.
Deferred / Won't fix	Product owner	Accepted risk; must be listed in the release risk summary. Never silently dropped.

Severity and priority

Severity is the technical impact of the defect. **Priority** is the business urgency of fixing it. The tester proposes severity; the product owner decides priority.

Severity	Definition	Example
Critical (S1)	System unusable, data loss/corruption, security exposure, no workaround.	Payment charged twice; application crashes on start.
Major (S2)	Key function fails or gives wrong results; workaround is hard.	Valid users cannot register (age 18 rejected).
Minor (S3)	Function works with deviation; an easy workaround exists.	Sort order wrong on a secondary list.
Trivial (S4)	Cosmetic or wording only.	Typo in a success message.

Combination	Example	Typical decision
High severity, high priority	Checkout fails for every customer.	Fix immediately; block release.
High severity, low priority	Crash in a feature used by one client once a year.	Schedule; document the workaround.
Low severity, high priority	Company name misspelled on the home page before a launch.	Fix before launch.
Low severity, low priority	Misaligned icon on an admin screen.	Backlog.

Test data

Type	Purpose	Rule
Valid data	Positive paths.	Representative of real users but fully synthetic.
Invalid data	Negative paths.	Wrong type, wrong format, too long/short, special characters.
Boundary data	Edges of ranges.	Derived from boundary-value analysis.
Bulk / volume data	Lists, search, performance.	Generated by script with a fixed seed so it is reproducible.
Masked data	Realistic shape without real identities.	Only when the client provides an approved, masked copy. Never a raw production extract.

Topic packs (ten-part teaching method)

1.1 Software development lifecycle

Definition	The ordered set of activities used to plan, build, test, release and maintain software.
Purpose	Shows where defects are cheapest to find and where QA adds value.
Setup	Whiteboard or diagram tool; the SDLC table above.
Procedure	1. Name the model your current project uses. 2. Mark where requirements, build, test, release happen. 3. Mark where QA is involved today and where it should be.
Expected output	A one-page lifecycle diagram with QA touchpoints marked.
Verification	Buddy confirms every phase has an input, an output and an owner.
Errors	Treating testing as one phase at the end; ignoring maintenance.
Reporting	`journal/l1-sdlc.md`.
Assignment	Draw the lifecycle for the Practice Shop assuming Scrum and assuming Waterfall; list three differences for the tester.
Assessment	Oral: explain why a defect found in design is cheaper than one found in production.

1.2 Software testing lifecycle

Definition	Seven phases of test work from requirement analysis to closure, each with entry and exit criteria.
Purpose	Makes test work repeatable and auditable.
Setup	STLC pills above; blank entry/exit table.
Procedure	1. For each phase write entry and exit criteria. 2. Identify the deliverable. 3. Name the approver.
Expected output	Table of seven phases × entry, exit, deliverable, approver.
Verification	Check that no exit criterion is “when done”; each must be measurable.
Errors	Vague criteria (“enough testing”); skipping closure.
Reporting	`journal/l1-stlc.md`.
Assignment	Fill the table for Assignment A0.
Assessment	Quiz question on entry versus exit criteria.

1.3 Quality assurance vs quality control

Definition	QA prevents defects through process; QC finds defects in the product.
Purpose	Clients buy both; confusing them leads to wrong promises.
Setup	The comparison table above.
Procedure	1. List five activities from a software project. 2. Classify each as QA or QC. 3. Justify each.
Expected output	A five-row classification with reasons.
Verification	QA Lead checks each classification against the definitions.
Errors	Calling test execution “QA” and nothing else.
Reporting	`journal/l1-qa-qc.md`.
Assignment	Classify: code review checklist, regression run, requirements workshop, bug triage, coding standard.
Assessment	Quiz: 3 classification questions.

1.4 Verification vs validation

Definition	Verification checks conformance to specification; validation checks fitness for user need.
Purpose	A product can pass verification and still fail validation.
Setup	Practice Shop spec (REQ list).
Procedure	1. Review the spec for contradictions (verification). 2. Imagine a first-time customer using the app (validation). 3. Record one finding for each.
Expected output	Two findings, one per activity.
Verification	Each finding cites a requirement ID or a user need.
Errors	Mixing the two; validating against your own opinion.
Reporting	`journal/l1-v-and-v.md`.
Assignment	Produce one spec-review finding and one usability finding.
Assessment	Oral Q&A with the buddy.

1.5 Bugs, defects and failures

Definition	Error (human) → defect (in the work product) → failure (observable wrong behaviour).
Purpose	Correct words give correct root-cause conversations.
Setup	Three small examples supplied by the trainer.
Procedure	1. For each example identify the error, defect and failure. 2. State how testing would detect it.
Expected output	Three completed chains.
Verification	Trainer compares with the model answer.
Errors	Reporting the failure but guessing the defect location.
Reporting	`journal/l1-defects.md`.
Assignment	Write a chain for the age-boundary problem in A0.
Assessment	Quiz: 4 questions.

1.6 Test cases

Definition	A documented set of preconditions, inputs, steps, expected results and postconditions to verify one condition.
Purpose	Makes testing repeatable by anyone and traceable to requirements.
Setup	Template from the Templates Pack; spreadsheet or test-management tool.
Procedure	1. Pick one requirement. 2. Write ID, title, preconditions, steps, data, expected result. 3. Peer-review it.
Expected output	One atomic, measurable test case.
Verification	A colleague who has never seen the app can run it and reach the same verdict.
Errors	Several verifications in one case; expected result like “works correctly”.
Reporting	Repository `test-cases/functional/`.
Assignment	Write five cases for REQ-002 (email).
Assessment	Reviewed by the buddy against a 6-point checklist.

1.7 Test scenarios

Definition	A high-level statement of what to test, from which several cases are derived.
Purpose	Gives quick coverage overviews to managers and clients.
Setup	Requirement list.
Procedure	1. Read a requirement. 2. Write scenario titles starting with “Verify that…”. 3. Derive cases later.
Expected output	A list of scenarios with requirement IDs.
Verification	Each requirement has at least one scenario.
Errors	Scenarios that are really single test cases; missing negative scenarios.
Reporting	`docs/requirements-traceability.md`.
Assignment	Write 10 scenarios for REQ-001 to REQ-009.
Assessment	Review by the QA Lead.

1.8 Test plans

Definition	A document describing scope, approach, resources, schedule, risks and criteria for testing.
Purpose	Aligns the client and the team on what will and will not be tested.
Setup	Test plan template.
Procedure	1. Define scope in/out. 2. Choose test types. 3. List environments and data. 4. Add schedule, roles, risks. 5. Set entry/exit criteria.
Expected output	A two-page test plan for A0.
Verification	Every section is filled; assumptions are listed.
Errors	Copying a plan without adapting scope and risks.
Reporting	`docs/test-plan.md`.
Assignment	Write the test plan for the Practice Shop registration module.
Assessment	Reviewed by the Test Manager.

1.9 Test data

Definition	Inputs and preconditions' data used by test cases.
Purpose	Wrong data gives wrong results and can break privacy rules.
Setup	Synthetic data generators or hand-written CSV.
Procedure	1. Classify data needed per case. 2. Create synthetic records. 3. Record where each record is used.
Expected output	`test-data/` file with documented purpose per row.
Verification	No real personal data present; boundary values included.
Errors	Re-using one account for all cases; using real e-mail addresses.
Reporting	`test-data/README.md`.
Assignment	Create 15 registration data rows including boundaries.
Assessment	Reviewed for privacy and coverage.

1.10 Bug lifecycle

Definition	The states a defect passes through from discovery to closure.
Purpose	Shared understanding prevents lost or silently closed defects.
Setup	Issue tracker workflow.
Procedure	1. Create a defect. 2. Move it through assigned → fixed → retest → closed in a sandbox project. 3. Reopen one with new evidence.
Expected output	A defect that has visited every state.
Verification	History log shows who changed what and when.
Errors	Closing your own fix without retest; skipping the build number.
Reporting	Issue tracker history.
Assignment	Run a lifecycle simulation with your buddy playing the developer.
Assessment	Quiz on states and ownership.

1.11 Severity and priority

Definition	Severity = technical impact; priority = urgency to fix.
Purpose	Drives fix order and release decisions.
Setup	Severity table above.
Procedure	1. Take five sample defects. 2. Assign severity. 3. Propose priority with business reasoning.
Expected output	Five rows with justification.
Verification	QA Lead agrees or explains differences.
Errors	Using “critical” for every bug; mixing severity and priority.
Reporting	Each defect report.
Assignment	Rate the seven defects found in your A0 run.
Assessment	Quiz: 4 scenarios, choose and defend.

LEVEL 1 EXIT GATE

Quiz ≥ 80 %, one test case and one bug report reviewed without major comments, and the trainee can explain the Golden Rules. Sign-off: QA Lead.

5. Level 2 — Manual Testing

How to analyse requirements, design effective test cases with proven techniques, execute them with evidence, and run the standard test types: smoke, sanity, regression, exploratory and UAT.

Duration	About 3 weeks
Prerequisites	Level 1 passed.
Outcome	Designs boundary and equivalence tests, performs exploratory sessions with a charter, runs smoke and regression cycles, and supports client acceptance testing.
Exit gate	Assignment A0 passed (≥ 70) + one exploratory-session report + technique quiz (≥ 80 %).

Requirement analysis

A requirement is **testable** when two people can independently decide whether a product meets it. Review every requirement with the checklist below and raise questions before writing test cases.

Check	Question to ask	Red-flag words / signs
Clear	Can it be read only one way?	“fast”, “user-friendly”, “etc.”, “as appropriate”, “flexible”
Measurable	Is there a number, a limit or an observable outcome?	No limits for fields, times, quantities
Complete	Are error and boundary behaviours stated?	Only the happy path described
Consistent	Does it contradict another requirement?	Two different limits for the same field
Feasible & traceable	Can it be tested in our environment and given an ID?	Needs production access; no requirement ID

Equivalence partitioning and boundary-value analysis

Equivalence partitioning (EP) divides inputs into classes that the system should treat the same way; test one value from each class. **Boundary-value analysis (BVA)** adds tests at the edges of each valid class, where off-by-one defects live.

Worked example: Age field, valid 18 to 60 inclusive (REQ-003)

Partition	Class	Sample value	Expected
P1	Below range (invalid)	10	Rejected with age message
P2	Valid range	35	Accepted

Partition	Class	Sample value	Expected
P3	Above range (invalid)	75	Rejected with age message
P4	Non-numeric	`abc`	Rejected
P5	Empty	(blank)	Rejected: required
P6	Decimal	`25.5`	Rejected: whole numbers only
P7	Negative	`-5`	Rejected

Boundary	Value	Valid?	Reason
Lower − 1	17	No	Just below the minimum
Lower	18	Yes	Minimum, inclusive
Lower + 1	19	Yes	Just inside
Upper − 1	59	Yes	Just inside
Upper	60	Yes	Maximum, inclusive
Upper + 1	61	No	Just above the maximum

> **RULE OF THUMB**
>
> Two-value BVA tests each edge and its nearest invalid neighbour (17, 18, 60, 61). Three-value BVA adds the nearest inside neighbours (19, 59). Use three-value BVA for money, health and safety fields.

Second example: Password length 8–20 (REQ-004)

Length	Valid?	Note
7	No	Lower − 1
8	Yes (if one digit + one uppercase)	Lower boundary
20	Yes	Upper boundary
21	No	Upper + 1
8, no digit / no uppercase	No	Rule combinations are separate cases

Other design techniques you must recognise

Technique	Use it when	Mini example
Decision table	Several conditions combine to give an outcome.	Discount: member? × order ≥ ₹1000? → 4 combinations, 4 expected discounts.
State transition	Behaviour depends on history.	Login lock-out: 0 failures → 1 → 2 → 3 (locked, 30 s) → unlock.
Use-case / user journey	End-to-end flows matter.	Register → login → add to cart → logout.
Error guessing	Experience tells you where bugs hide.	Spaces only in name; emoji; very long text; double-click on submit.
Pairwise (combinatorial)	Many parameters with several values.	Browser × OS × language reduced from 27 to ~9 cases.

Exploratory testing

Simultaneous learning, test design and execution, guided by a **charter** and time-boxed (typically 60–90 minutes). Record notes as you go; exploratory testing is not “random clicking”.

```
exploratory-session-sheet.txt

CHARTER:   Explore the Register view with unusual inputs to discover validation gaps
AREAS:     Registration form (name, email, age, passwords, terms)
TIME-BOX:  60 minutes          TESTER: <name>          BUILD: <version>
HEURISTICS: boundaries, special characters, copy/paste, double submit, keyboard only, resize window
NOTES:     (timestamped observations)
BUGS:      DEF-0xx, DEF-0xx          ISSUES/QUESTIONS: ...
DEBRIEF:   % on setup / test design / bug investigation; what to test next
```

Smoke, sanity, regression and UAT compared

Type	Question it answers	Scope	When	Who
Smoke	Is the build stable enough to test at all?	Broad and shallow: critical paths only (10–30 min).	Every new build / after deployment	QA or automated pipeline
Sanity	Does the specific fix/change work and nothing obvious broke around it?	Narrow and deep, unscripted or short script.	After a fix or small change	QA
Regression	Did the change break existing behaviour?	Selected or full suite of previously passing tests.	Before every release	QA + automation
UAT	Does it meet business needs in real conditions?	Business scenarios in a production-like environment.	After QA exit, before go-live	Client users; QA supports

REGRESSION SELECTION RULE

Run the full suite when risk is high or the change is wide. Otherwise run: (1) tests for changed areas, (2) tests for areas that integrate with them, (3) all critical-path tests, (4) a rotating sample of the rest. Record why anything was skipped.

Topic packs (ten-part teaching method)

2.1 Requirement analysis	
Definition	Reviewing requirements for clarity, completeness, consistency and testability before designing tests.
Purpose	Cheapest defect prevention: ambiguity found now costs nothing.
Setup	Spec or user stories; the checklist table above.
Procedure	1. Read twice. 2. Mark ambiguous or untestable statements. 3. Write questions. 4. Create the traceability rows.
Expected output	`questions.md` + traceability matrix skeleton.
Verification	Every requirement has an ID, a testability verdict and an owner for open questions.
Errors	Assuming unstated behaviour; writing tests for requirements you do not understand.
Reporting	`docs/requirements-traceability.md`.
Assignment	Analyse REQ-001 to REQ-014 and list at least 6 questions.
Assessment	QA Lead reviews questions for quality, not quantity.

2.2 Test case writing

Definition	Producing atomic, repeatable cases with measurable expected results.
Purpose	Quality of cases decides quality of testing.
Setup	Test case template, test data file.
Procedure	1. One case = one verification. 2. State preconditions. 3. Number the steps. 4. Write an observable expected result. 5. Link requirement and priority.
Expected output	A suite of 30+ cases.
Verification	A peer executes five random cases without asking questions.
Errors	Steps that assume knowledge (“log in normally”); duplicate cases.
Reporting	`test-cases/functional/`.
Assignment	Deliver A0 test suite.
Assessment	6-point checklist: atomic, ID, precondition, data, measurable, traceable.

2.3 Positive and negative testing

Definition	Positive tests use valid input to confirm correct behaviour; negative tests use invalid input to confirm safe rejection.
Purpose	Production users make mistakes; the system must fail safely.
Setup	Equivalence classes from each requirement.
Procedure	1. List valid classes. 2. List invalid classes. 3. Write at least one case per class. 4. Check error messages are clear and do not leak internals.
Expected output	Balanced suite (roughly 40 % positive, 60 % negative is typical).
Verification	Each field has both types; messages checked.
Errors	Only testing the happy path.
Reporting	Case suite, tagged `positive` / `negative`.
Assignment	For REQ-004 write 8 positive/negative cases.
Assessment	Review ratio and message clarity.

2.4 Boundary-value analysis

Definition	Testing at and next to the edges of valid ranges.
Purpose	Off-by-one defects are among the most common.
Setup	Range limits from the spec.
Procedure	1. Identify min/max. 2. Compute min−1, min, min+1, max−1, max, max+1. 3. Add them as cases.
Expected output	A boundary table per ranged field.
Verification	Values computed correctly (including inclusive/exclusive wording).
Errors	Misreading “up to 60” as exclusive; skipping upper bounds.
Reporting	Boundary table in the test-case file.
Assignment	Produce tables for REQ-001, 003, 004, 010.
Assessment	Quiz: compute boundaries for 5 rules.

2.5 Equivalence partitioning

Definition	Grouping inputs into classes treated identically; one representative per class.
Purpose	Reduces test count without losing coverage.
Setup	Input domain per field.
Procedure	1. Split the domain into valid and invalid classes. 2. Choose one value per class. 3. Record class names.
Expected output	A partition table.
Verification	Classes are disjoint and cover the whole domain.
Errors	Overlapping classes; missing “empty” or “wrong type” classes.
Reporting	Partition table in the suite.
Assignment	Partition the email field.
Assessment	Quiz: 3 partitioning questions.

2.6 Exploratory testing

Definition	Time-boxed, charter-driven learning and testing without pre-written steps.
Purpose	Finds defects scripts miss; covers risk quickly.
Setup	Charter sheet; screen recorder; notebook.
Procedure	1. Write the charter. 2. Test for the time-box while taking notes. 3. Log bugs immediately. 4. Debrief.
Expected output	Session sheet with notes, bugs and next steps.
Verification	Notes are detailed enough to reproduce each finding.
Errors	No notes; wandering off charter; no time-box.
Reporting	`evidence/exploratory/` + defects.
Assignment	Run one 60-minute session on the Shop view.
Assessment	Debrief with the QA Lead.

2.7 Regression testing

Definition	Re-running tests to confirm that changes did not break existing behaviour.
Purpose	Most production incidents after releases are regressions.
Setup	Stable, prioritised suite; build under test.
Procedure	1. Select tests by risk and change impact. 2. Execute. 3. Compare with the baseline. 4. Log new failures as regressions.
Expected output	Regression result summary with reasons for skips.
Verification	Counts reconcile; every failure has a defect or an explanation.
Errors	Running everything blindly; never retiring obsolete tests.
Reporting	Execution report.
Assignment	Build a 20-case regression pack from A0 and run it on a modified build.
Assessment	Reviewed by the Test Manager.

2.8 Smoke testing

Definition	A short, broad check that the build's critical functions work.
Purpose	Stops wasting time on unusable builds.
Setup	List of 10–15 critical-path checks.
Procedure	1. Deploy build. 2. Execute checks in order. 3. Any fail → reject build.
Expected output	Pass/fail smoke verdict in under 30 minutes.
Verification	Same checks every time, ordered by dependency.
Errors	Smoke suite growing into a full regression suite.
Reporting	Smoke result in the build ticket.
Assignment	Write the Practice Shop smoke list.
Assessment	Timed demonstration.

2.9 Sanity testing

Definition	A narrow check that a specific fix or small change works and nothing obvious nearby broke.
Purpose	Quick confidence after a fix before deeper regression.
Setup	Defect or change description.
Procedure	1. Read the fix note. 2. Retest the original steps. 3. Test two neighbouring behaviours.
Expected output	Short verdict recorded on the defect.
Verification	Original steps now pass; neighbours unaffected.
Errors	Treating sanity as a replacement for regression.
Reporting	Defect tracker comment.
Assignment	Retest three fixed defects.
Assessment	Buddy checks retest evidence.

2.10 User acceptance testing (UAT)

Definition	Business users confirm the system meets their needs in a production-like environment.
Purpose	Final validation and contractual acceptance.
Setup	UAT plan, scripts, accounts, test data, UAT environment.
Procedure	1. Agree scope and entry criteria. 2. Brief users. 3. Observe, log issues. 4. Triage with the client. 5. Obtain written acceptance.
Expected output	UAT report and signed acceptance.
Verification	All critical issues resolved or formally accepted.
Errors	Letting QA decide business acceptance; vague scripts.
Reporting	UAT report template.
Assignment	Shadow a mock UAT and write the report.
Assessment	QA Lead reviews report.

LEVEL 2 EXIT GATE

A0 passed (≥ 70 / 100); technique quiz ≥ 80 %; one exploratory session reported; 20-case regression pack run successfully. Sign-off: QA Lead.

6. Level 3 — Web and Mobile Testing

Browsers, devices, forms, navigation, sessions, operating-system compatibility and accessibility: the environment-specific skills that turn generic test cases into real-world coverage.

Duration	About 2 weeks
Prerequisites	Level 2 passed.
Outcome	Builds a risk-based browser/device matrix, uses developer tools to investigate failures, tests forms, navigation and sessions systematically, and performs first-line accessibility checks.
Exit gate	Responsive + cross-browser test report on the Practice Shop, session-test checklist, accessibility checklist run, practical demo.

Choosing the browser, device and OS matrix

Never promise “all browsers”. Agree a **supported matrix** with the client, based on their analytics, contract and risk. A typical starting point is shown below; replace it with the client's own data.

Dimension	Typical minimum	How to decide
Desktop browsers	Latest Chrome, Edge, Firefox, Safari (macOS) + one previous major version where analytics show > 2 % share	Client analytics (top 90 % of sessions)
Mobile browsers	Chrome on Android, Safari on iOS	Analytics; app store minimum OS
Viewports	360×800 (small phone), 768×1024 (tablet), 1280×800 (laptop), 1920×1080 (desktop)	Responsive breakpoints in the CSS
Operating systems	Windows 11, macOS current, Android (current and current−2), iOS (current and current−1)	Support policy in the contract
Real devices	At least 1 low-end Android, 1 mid-range, 1 iPhone, 1 tablet	Budget and risk; use a device cloud for the long tail

Browser developer tools: the tester's microscope

Panel	What to use it for	Evidence to save
Elements	Inspect DOM and CSS; check labels, alt text, `lang`, viewport meta; test CSS states.	Screenshot of the element and computed styles
Console	JavaScript errors and warnings (red entries).	Copy of the message with the URL and time
Network	Requests, status codes, timing, payloads, caching; throttle to Slow 3G; block a request to test error handling.	HAR export (remove tokens and personal data first)
Application	Cookies, local/session storage, service workers; check cookie flags (`Secure`, `HttpOnly`, `SameSite`) and session clearing on logout.	Screenshot of storage before and after logout
Device toolbar	Responsive viewport and device emulation (an approximation, not a real device).	Screenshot at each tested viewport
Lighthouse / Accessibility	Quick audits; see Level 6.	Report file attached to the test run

Form testing checklist

Area	Check
Required fields	Submit with each required field empty, one at a time and all together; error is next to the field, announced, and the value already typed is preserved.
Type and format	Wrong type (letters in numeric), invalid e-mail forms (`a@b`, `a@b.`, spaces), leading/trailing spaces, Unicode names, emoji, very long text.
Limits	Min/max length and range boundaries (BVA); paste a value longer than the limit.
Behaviour	Enter key submits; double-click on Submit sends one request; Tab order follows visual order; autofill and password managers work.
Messages	Plain language, no stack traces, no leakage of internal data; same wording for credential failures (do not reveal which part was wrong).
State	Browser Back/Refresh after submit; form keeps or clears data as specified; success confirmation shown once.

Navigation and session testing

Topic	What to verify
Links and routes	No broken links (404); external links open as specified; deep links work when logged in and redirect to login (then back) when logged out; breadcrumbs and active menu state.
Back / forward / refresh	No duplicate submissions; protected pages are **not** shown after logout when pressing Back (REQ-012).
Session timeout	Idle expiry after the specified time; clear message; unsaved work warning where required; session cannot be revived by simply reloading.
Concurrent sessions	Same account in two browsers: behaviour matches the specification (allowed, or older session ended).
Cookies / storage	Session identifier changes after login; removed on logout; flags set; sensitive values are not stored in local storage in clear text (report, do not exploit).
Tabs and windows	Actions in one tab are reflected (or safely rejected) in another.

Mobile testing

Approach	Strengths	Limits	Use for
Real devices	True performance, sensors, gestures, OEM quirks.	Cost, availability, setup time.	Final verification, performance feel, camera/biometrics
Emulators / simulators	Cheap, fast, many OS versions.	Not exact on performance, hardware, vendor skins.	Early functional testing, OS-version coverage
Device cloud	Large device catalogue, remote access.	Licence cost; network latency; data-privacy review needed.	Long-tail coverage

Mobile-specific cases: install, upgrade (data preserved), uninstall; permissions (allow, deny, revoke later); interruptions (call, notification, low battery, app switch, screen rotation); network (offline, Wi-Fi ↔ mobile data switch, slow 3G); storage full; different languages and font sizes; push notifications; deep links; app store rules.

Accessibility testing (first-line checks)

Accessibility follows the four **POUR** principles of WCAG: Perceivable, Operable, Understandable, Robust. Automated tools find only a portion of issues; manual checks are mandatory.

Check	How	Pass criterion
Keyboard only	Unplug the mouse; use Tab, Shift+Tab, Enter, Space, Esc, arrow keys.	Every function reachable and operable; focus is always visible; no keyboard trap.
Labels and names	Inspect inputs/buttons; check `<label for>` or `aria-label`.	Every control has an accessible name.
Alt text	Inspect images.	Informative images have meaningful `alt`; decorative `alt=""`.
Colour contrast	DevTools contrast ratio or an accessibility checker.	Text ≥ 4.5 : 1 (large text ≥ 3 : 1).
Zoom and reflow	Browser zoom 200 %; 320 px width.	No loss of content or function; no horizontal scrolling for text.
Screen reader	NVDA (Windows), VoiceOver (macOS/iOS), TalkBack (Android); read a form and a table.	Order, names, states and errors are announced.

STANDARD TO QUOTE

Contracts normally require **WCAG 2.1 or 2.2 Level AA** (published by the W3C). Confirm the exact version and level in the scope. We report conformance **findings**; we do not issue legal compliance certificates.

Topic packs (ten-part teaching method)

3.1 Browser testing

Definition	Checking that the application works in each supported browser and version.
Purpose	Rendering and JavaScript engines differ; users are split across browsers.
Setup	Supported-browser matrix; BrowserStack-class cloud or local browsers.
Procedure	1. Fix the matrix. 2. Run smoke + top journeys per browser. 3. Compare screenshots. 4. Log browser-specific defects with version.
Expected output	Matrix with pass/fail per journey.
Verification	Each fail reproduced twice and checked in a second browser to confirm it is browser-specific.
Errors	Testing only in the developer's browser; not recording the exact version.
Reporting	Cross-browser table in the execution report.
Assignment	Run the A0 suite in Chrome, Firefox and WebKit.
Assessment	Report reviewed by QA Lead.

3.2 Responsive testing

Definition	Verifying layout and function at different viewport sizes and orientations.
Purpose	A large share of traffic is mobile.
Setup	DevTools device toolbar; real phone/tablet.
Procedure	1. Test the breakpoints and 10 px either side. 2. Rotate. 3. Check touch targets (≥ 24 px) and text zoom.
Expected output	Screenshots per viewport; list of layout defects.
Verification	No horizontal scroll, no overlap, no clipped controls.
Errors	Relying on emulation only; ignoring landscape.
Reporting	Evidence folder per viewport.
Assignment	Test REQ-013 at 360 px.
Assessment	Practical review.

3.3 Form testing

Definition	Validation, submission, error handling and usability of input forms.
Purpose	Forms are where users and data enter the system.
Setup	Form test checklist; synthetic data.
Procedure	1. Map every field and rule. 2. Apply EP/BVA. 3. Test submission behaviours. 4. Verify messages and persistence.
Expected output	Form test suite and results.
Verification	Each rule has at least one passing and one failing input.
Errors	Not testing paste, autofill, double submit or whitespace.
Reporting	Test cases linked to requirements.
Assignment	Full registration-form suite (A0).
Assessment	A0 marking rubric.

3.4 Navigation testing

Definition	Verifying links, menus, routing, back/forward and deep links.
Purpose	Broken navigation means lost users.
Setup	Sitemap; link checker (optional).
Procedure	1. Build a route map. 2. Click every route. 3. Test logged-in/out access. 4. Test Back/forward/refresh.
Expected output	Route map with verdicts.
Verification	No 404s; protected routes redirect correctly.
Errors	Skipping footer and error pages.
Reporting	Route table.
Assignment	Map and test all routes of the Practice Shop.
Assessment	Demonstration.

3.5 Session testing

Definition	Verifying login state, timeout, logout and concurrent-session behaviour.
Purpose	Session faults cause data exposure and lost work.
Setup	Two browsers; DevTools Application panel.
Procedure	1. Log in. 2. Stay idle until timeout. 3. Log out and use Back. 4. Open second session. 5. Compare with REQ-011/012.
Expected output	Session checklist with evidence.
Verification	Observed behaviour matches the spec in each step.
Errors	Using a stopwatch only once; forgetting to clear storage between cases.
Reporting	Checklist + screenshots.
Assignment	Run the session checklist.
Assessment	QA Lead review.

3.6 Mobile device testing

Definition	Testing apps and mobile websites on real or virtual devices.
Purpose	Hardware, OS and network conditions change behaviour.
Setup	Android emulator, one real phone, cloud account if licensed.
Procedure	1. Install the build. 2. Run core journeys. 3. Run interruptions and network cases. 4. Record device/OS.
Expected output	Device-by-journey results.
Verification	Every failure notes device model, OS version, build.
Errors	Not recording the OS version; testing only on a flagship phone.
Reporting	Execution report.
Assignment	Test the Practice Shop on a phone browser.
Assessment	Practical review.

3.7 Operating-system compatibility

Definition	Confirming the product works across supported OS versions and configurations.
Purpose	OS updates frequently break software.
Setup	VMs or test machines; compatibility matrix.
Procedure	1. List supported OS versions. 2. Install/run smoke. 3. Test upgrade from previous version.
Expected output	Compatibility matrix.
Verification	Install, run, upgrade and uninstall pass on each OS.
Errors	Skipping upgrade and uninstall paths.
Reporting	Matrix document.
Assignment	Build a matrix for a desktop app of your choice.
Assessment	Reviewed by the QA Lead.

3.8 Accessibility testing

Definition	Checking that people with disabilities can use the product.
Purpose	Legal duty, ethical duty and a bigger market.
Setup	Keyboard, NVDA/VoiceOver, axe DevTools, Lighthouse.
Procedure	1. Keyboard walk-through. 2. Automated scan. 3. Screen-reader read. 4. Zoom/contrast checks. 5. Log issues with WCAG criterion.
Expected output	Accessibility findings list with WCAG references.
Verification	Every automated finding triaged; manual checks documented.
Errors	Trusting automated scores alone.
Reporting	Findings in the defect tracker with the `a11y` label.
Assignment	Audit the registration page.
Assessment	Assessment by the QA Lead.

LEVEL 3 EXIT GATE

Cross-browser/responsive report, session checklist and accessibility findings for the Practice Shop reviewed and approved. Sign-off: QA Lead.

7. Level 4 — API Testing

Testing the interface behind the screen: HTTP, authentication, JSON, schemas, negative cases, and the foundations of API automation.

Duration	About 2 weeks
Prerequisites	Level 3 passed. Basic JSON understanding.
Outcome	Sends and validates requests with curl and Postman, designs positive and negative API cases, validates JSON against a schema, and reports API defects with request/response evidence.
Exit gate	Practice API test collection with ≥ 40 requests, ≥ 8 defects correctly reported, schema validation demonstrated.

HTTP methods

Method	Purpose	Safe?	Idempotent?	Typical success
GET	Read a resource.	Yes	Yes	200
POST	Create a resource or trigger an action.	No	No	201 (+ `Location`), 200, 202
PUT	Replace a resource completely.	No	Yes	200 or 204
PATCH	Partially update a resource.	No	Not guaranteed	200 or 204
DELETE	Remove a resource.	No	Yes	204 (or 200)
HEAD / OPTIONS	Headers only / allowed methods and CORS.	Yes	Yes	200 / 204

IDEMPOTENT MEANS REPEAT-SAFE

Repeating an idempotent request leaves the server in the same state. Test it: send the same `PUT` twice and check state is unchanged; send `DELETE` twice and check the second response is `404` (resource already gone), not another `204`.

Status codes to know by heart

Code	Meaning	When a test expects it
200 OK	Success with a body.	Valid GET or update
201 Created	Resource created; `Location` header gives its URL.	Valid POST

Code	Meaning	When a test expects it
204 No Content	Success, no body.	Valid DELETE
400 Bad Request	Malformed or invalid input.	Missing field, wrong type, bad range
401 Unauthorized	Not authenticated (missing/invalid/expired token).	No token or bad token
403 Forbidden	Authenticated but not allowed.	Normal user calls admin endpoint
404 Not Found	Resource does not exist.	Unknown id; deleted resource
405 Method Not Allowed	Method unsupported on this URL.	`DELETE /api/health`
409 Conflict	State conflict.	Duplicate unique value
415 / 422	Unsupported media type / semantically invalid.	Wrong `Content-Type`; business-rule failure
429 Too Many Requests	Rate limit.	Burst above the limit (only if rate limiting is specified)
500 / 502 / 503 / 504	Server error / bad gateway / unavailable / gateway timeout.	**Never** expected from client mistakes; report as defects

Authentication types

Type	How it travels	What to test
API key	Header such as `X-API-Key`.	Missing, wrong, revoked key; key never in URLs or logs.
Basic auth	`Authorization: Basic base64(user:pass)`.	Only over HTTPS; wrong credentials → 401.
Bearer / JWT	`Authorization: Bearer <token>`.	Missing, malformed, expired, tampered token → 401; role checks → 403.
OAuth 2.0	Access token obtained from an authorisation server.	Scopes, token expiry, refresh flow, invalid grant.

Practice API contract (the target for Projects 3 and 5)

Method and path	Auth	Success	Specified errors
`GET /api/health`	No	200 `{"status":"ok","service":"vel-practice-api"}`	—
`POST /api/auth/login`	No	200 `{token, expiresIn}`	400 missing field; 401 invalid credentials

Method and path	Auth	Success	Specified errors
`GET /api/products?page=&limit=`	No	200 `{items,page,limit,total}`; `limit` 1–50, default 5	400 invalid page/limit
`GET /api/products/{id}`	No	200 product `{id,name,price,stock}`	400 id not an integer; 404 unknown id
`POST /api/products`	Bearer	201 + `Location`	400 invalid body; 401; 409 duplicate name
`PUT /api/products/{id}`	Bearer	200 updated product	400; 401; 404
`DELETE /api/products/{id}`	Bearer	204	401; 404 unknown or already deleted
`GET /api/orders`	Bearer	200 list	401 without valid token
`GET /api/slow?delay=ms`	No	200 after delay (0–5000 ms)	400 invalid delay

Validation rules for products: `name` required string 1–60 characters; `price` number **> 0**; `stock` integer **≥ 0**. Practice credentials (synthetic, for the local server only): `qa@example.test` / `Passw0rd!QA`.

Worked example with curl

```bash
# 1. Health
curl -i http://localhost:8080/api/health

# 2. Login and capture the token (jq optional)
TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"qa@example.test","password":"Passw0rd!QA"}' | python3 -c 'import
sys,json;print(json.load(sys.stdin)["token"])')

# 3. Create a product (positive)
curl -i -X POST http://localhost:8080/api/products \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"name":"Marker","price":3.5,"stock":10}'
# Expect: HTTP/1.1 201 Created  +  Location: /api/products/<id>

# 4. Negative: price must be > 0
curl -i -X POST http://localhost:8080/api/products \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"name":"Free item","price":0,"stock":1}'
# Expect: 400 with an error message  (if you get 201, you have found a defect)
```

JSON schema validation

A **JSON Schema** states the shape of a valid response: required fields, types, ranges. Validating every response against a schema catches type drift (for example `price` returned as a string) that status-code checks miss.

```
automation/api/schemas/product.schema.json

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Product",
  "type": "object",
  "required": ["id", "name", "price", "stock"],
  "properties": {
    "id":    { "type": "integer", "minimum": 1 },
    "name":  { "type": "string", "minLength": 1, "maxLength": 60 },
    "price": { "type": "number", "exclusiveMinimum": 0 },
    "stock": { "type": "integer", "minimum": 0 }
  },
  "additionalProperties": false
}
```

API negative-testing checklist

Category	Cases
Authentication	No token; malformed token; expired token; token of another user; token in the wrong header.
Input validation	Missing required field; wrong type (string for number); null; empty string; too long; negative; zero; huge number; special characters; wrong `Content-Type`; invalid JSON.
Boundaries	`limit` 0, 1, 50, 51; `page` 0, −1; price 0.01 and 0.
Resource state	Unknown id; already-deleted id; duplicate create; update of a deleted resource.
Methods	Unsupported method on each URL → 405 with `Allow` header.
Errors and leakage	No stack traces, SQL, file paths or internal hostnames in error bodies; errors use the documented JSON shape and `Content-Type`.
Performance sanity	Response time for a single request within the agreed budget; large `limit` does not exhaust the server.

Postman essentials

1. Create a **Collection** (Practice API) with a **Collection Variable** `baseUrl` = `http://localhost:8080`.
2. Create an **Environment** `local`; store the token in a variable (secrets use *secret*-type variables and are never committed).
3. Add requests per endpoint; add **Tests** scripts (JavaScript) to assert status, time, headers and body.
4. Run the collection with the **Collection Runner**, or from CI with `newman run collection.json -e local.json` (Newman is Apache-2.0, registry-verified).
5. Export collection and a sanitised environment into `automation/api/postman/`.

```
postman-tests.js

// Postman "Tests" tab for POST /api/products
pm.test("status is 201", () => pm.response.to.have.status(201));
pm.test("Location header present", () => pm.response.to.have.header("Location"));
```

```
pm.test("response time < 500 ms", () => pm.expect(pm.response.responseTime).to.be.below(500));
const body = pm.response.json();
pm.test("price is a number", () => pm.expect(body.price).to.be.a("number"));
pm.collectionVariables.set("createdId", body.id);
```

Topic packs (ten-part teaching method)

4.1 HTTP methods	
Definition	Verbs that tell the server what operation to perform on a resource.
Purpose	Wrong method use is a common API defect and a security smell.
Setup	curl or Postman; Practice API running.
Procedure	1. Call each endpoint with each method. 2. Record status. 3. Compare with the contract.
Expected output	A method × endpoint matrix of observed statuses.
Verification	Unsupported methods return 405 with `Allow`.
Errors	Testing only GET and POST.
Reporting	API test sheet.
Assignment	Build the matrix for all Practice API endpoints.
Assessment	Reviewed against the contract.

4.2 Request and response validation	
Definition	Checking headers, status, body and timing of every call.
Purpose	Tests are only as good as their assertions.
Setup	Postman Tests tab or pytest.
Procedure	1. Define the expected status, headers and body. 2. Assert each. 3. Add time budget.
Expected output	A request with 4+ assertions.
Verification	Break the request on purpose; the assertion must fail.
Errors	Only asserting 200.
Reporting	Collection export.
Assignment	Add assertions to five requests.
Assessment	Demo.

4.3 Status codes

Definition	Three-digit codes that summarise the result of a request.
Purpose	Clients branch on them; wrong codes break integrations.
Setup	Status-code table.
Procedure	1. For each endpoint list success and error codes from the contract. 2. Provoke each. 3. Compare.
Expected output	Expected vs actual table.
Verification	All specified codes provoked at least once.
Errors	Accepting 500 as an “error response”.
Reporting	Defects for each mismatch.
Assignment	Provoke 400, 401, 404, 409 on the Practice API.
Assessment	Quiz: 10 codes.

4.4 Authentication

Definition	Proving identity to the API (key, basic, bearer/JWT, OAuth).
Purpose	Broken authentication exposes data.
Setup	Practice credentials; Postman.
Procedure	1. Call without token. 2. Wrong token. 3. Valid token. 4. Expired token (if available).
Expected output	401 for each invalid case; 200/201 for valid.
Verification	No protected endpoint works without a valid token.
Errors	Sharing tokens in screenshots; committing tokens.
Reporting	Defects; security coordinator notified for access-control faults.
Assignment	Test all protected endpoints without a token.
Assessment	QA Lead review; escalation rule explained.

4.5 JSON validation

Definition	Checking the structure and values of JSON bodies.
Purpose	Bad JSON breaks clients.
Setup	Browser/JSON viewer, Postman.
Procedure	1. Check types. 2. Check required fields. 3. Check nested objects and arrays. 4. Check ranges.
Expected output	Validation notes per endpoint.
Verification	Matches the documented model.
Errors	Eyeballing instead of asserting.
Reporting	Assertions in tests.
Assignment	Validate `/products` response items.
Assessment	Practical test.

4.6 Schema validation

Definition	Automated check of a response against a JSON Schema.
Purpose	Detects silent type changes and missing fields.
Setup	Schema file; Postman `tv4` /Ajv or Python `jsonschema` (MIT, registry-verified).
Procedure	1. Write the schema. 2. Validate every product in the list. 3. Add negative schema (extra field).
Expected output	Schema test that fails for the broken item.
Verification	Schema catches the seeded string-price defect.
Errors	Schema too loose (`type: object` only).
Reporting	Defect with schema error message.
Assignment	Write schemas for Product and Product list.
Assessment	Peer review.

4.7 API negative testing

Definition	Sending invalid, missing or hostile input to prove safe failure.
Purpose	Real clients and attackers send unexpected data.
Setup	Negative-testing checklist.
Procedure	1. Pick endpoint. 2. Apply each category. 3. Record actual response. 4. Raise defects for 5xx and silent acceptance.
Expected output	20+ negative cases with results.
Verification	No 5xx; clear 4xx; no data change on rejected requests.
Errors	Stopping at obvious cases.
Reporting	Defect reports with request/response.
Assignment	Write 20 negative cases for `/api/products`.
Assessment	Review.

4.8 API automation

Definition	Running API checks from code (pytest + requests) or Newman, repeatably.
Purpose	Fast, stable regression for integrations.
Setup	Python 3.11+, `pytest`, `requests`, `jsonschema`.
Procedure	1. Create fixtures for base URL and token. 2. Write parametrised tests. 3. Run in CI. 4. Publish results.
Expected output	Passing and failing tests with HTML/JUnit report.
Verification	Known defects show as expected failures (`xfail` with defect ID).
Errors	Shared state between tests; hard-coded ids and secrets.
Reporting	`automation/api/` and reports.
Assignment	Automate 15 cases (Project 5).
Assessment	Code review against checklist.

LEVEL 4 EXIT GATE

Collection ≥ 40 requests with assertions, ≥ 8 correctly reported API defects, schema validation demonstrated. Sign-off: QA Lead.

8. Level 5 — Automation Testing

Turn stable, repeatable, valuable manual tests into reliable code: Python or JavaScript/TypeScript fundamentals, Playwright, Selenium, Cypress, pytest, Postman, Page Object Model, fixtures, test data and CI.

Duration	About 4 weeks
Prerequisites	Levels 1–4 passed. Comfortable using a terminal and Git.
Outcome	Writes maintainable UI and API automation with a Page Object Model and fixtures, runs it locally and in CI, and reports automation coverage honestly.
Exit gate	Automated suite (≥ 15 UI + ≥ 15 API tests) running green in GitHub Actions with a published report; code review passed.

What to automate (and what not to)

Automate	Do not automate (yet)
Stable features with clear expected results · smoke and regression paths · data-driven checks (many inputs, same steps) · API contract and schema checks · tests run on every build · tests too tedious or error-prone by hand.	Features still changing daily · one-off checks · exploratory testing and usability judgement · visual design opinions · tests that need physical hardware you cannot control · anything whose expected result you cannot state precisely.

AUTOMATION RETURN-ON-INVESTMENT RULE

Automate when **(manual minutes per run × number of runs)** is greater than **(build time + maintenance time)**. Example: a 30-minute manual regression run 40 times a year = 20 hours. If automating costs 16 hours and maintenance 4 hours/year, break-even is in the first year; with 6 runs a year it never pays back.

The test pyramid

Layer	Share	Speed	Examples
Unit (developers)	Many (≈ 70 %)	Milliseconds	Function tests, mocked dependencies
API / integration (QA + dev)	Some (≈ 20 %)	Seconds	Contract, schema, auth, error handling
UI end-to-end (QA)	Few (≈ 10 %)	Tens of seconds	Critical journeys: register → login → buy

QA owns the top two layers. The more logic you can verify at API level, the faster and less flaky the suite.

Choosing a UI automation tool

Tool	Languages	Browsers	Strengths	Limitations	Licence (registry-verified 2026-11-04)
Playwright	TypeScript/JS, Python, Java, .NET	Chromium, Firefox, WebKit	Auto-waiting, tracing, parallel, API testing, codegen	WebKit ≠ real Safari; no real mobile devices (emulation)	Apache-2.0 (v1.64.1)
Selenium WebDriver	Java, Python, C#, JS, Ruby	All major via drivers	Standard, huge ecosystem, Grid	More boilerplate, manual waits, slower	Apache-2.0 (v4.52.0)
Cypress	JavaScript/TypeScript	Chromium, Firefox, WebKit (experimental)	Great developer experience, time-travel debugging	Single-origin history, no multi-tab; paid cloud features	MIT (v16.2.1); Cypress Cloud is a paid service

Selection rule: default to Playwright for new projects; use Selenium when the client already owns a Selenium Grid or needs a language or browser Playwright lacks; use Cypress when the client team is JavaScript-only and wants developers to write tests too. Alternatives to know: WebdriverIO (MIT), Appium for native mobile (Apache-2.0), Robot Framework.

Python fundamentals required (one-page primer)

```
python-primer.py

# Variables, collections, f-strings
user = {"email": "qa@example.test", "age": 18}
ages = [17, 18, 60, 61]
print(f"{user['email']} has age {user['age']}")

# Functions and loops
def is_valid_age(age: int) -> bool:
    return 18 <= age <= 60          # the spec: inclusive range

for a in ages:
    print(a, is_valid_age(a))       # False True True False

# Exceptions and files
import json, pathlib
try:
    data = json.loads(pathlib.Path("test-data/users.json").read_text(encoding="utf-8"))
except FileNotFoundError:
    data = []
```

JavaScript/TypeScript equivalent topics: `const/let`, arrow functions, arrays/objects, `async/await`, modules (`import`), `try/catch`, JSON. Teach one language first; the second takes days, not weeks.

pytest essentials

Concept	Use	Example
Test discovery	Files `test_*.py`, functions `test_*`.	`pytest -q`
Fixtures	Reusable setup/teardown with scopes (`function`, `module`, `session`).	`@pytest.fixture(scope="session")`
Parametrisation	One test, many data rows.	`@pytest.mark.parametrize("age,ok", [(17,False),(18,True)])`
Markers	Select groups; mark known defects.	`-m smoke`; `@pytest.mark.xfail(reason="DEF-001", strict=True)`
conftest.py	Shared fixtures per directory.	base URL, auth token, browser page
Reports	JUnit XML for CI; HTML via `pytest-html` (MPL-2.0).	`--junitxml=reports/junit.xml`

Playwright (Python) with Page Object Model and fixtures

The code below is the **real starter code** from `qa-project-template/` (it is embedded from the repository when this handbook is built, so it cannot drift). Install and run (Playwright downloads its browsers once):

```bash
cd qa-project-template
python3 -m venv .venv && source .venv/bin/activate
pip install -r automation/requirements.txt
playwright install chromium firefox
export QA_EMAIL=qa@example.test QA_PASSWORD='<see practice-api/README.md>'
pytest -m smoke                 # critical path only
pytest automation/ui --browser chromium --browser firefox
```

automation/page-objects/register_page.py

```
"""Page object: Register view of the VEL Practice Shop. Locators and actions only; assertions belong in the tests."""
from playwright.sync_api import Page


class RegisterPage:
    def __init__(self, page: Page, url: str):
        self.page, self.url = page, url
        self.name = page.get_by_label("Full name")
        self.email = page.get_by_label("Email")
        self.age = page.get_by_label("Age")
        self.password = page.get_by_label("Password", exact=True)
        self.confirm = page.get_by_label("Confirm password")
        self.terms = page.get_by_label("I accept the terms")
        self.submit_btn = page.get_by_role("button", name="Register")
        self.alerts = page.get_by_role("alert")
```

```
    def open(self) -> "RegisterPage":
        self.page.goto(f"{self.url}#/register")
        return self

    def register(self, name="Asha Rao", email="qa@example.test", age=35, password="Passw0rdOK",
confirm=None, terms=True):
        self.name.fill(name)
        self.email.fill(email)
        self.age.fill(str(age))
        self.password.fill(password)
        self.confirm.fill(password if confirm is None else confirm)
        if terms:
            self.terms.check()
        self.submit_btn.click()
```

automation/ui/test_registration.py

```
"""Starter UI tests (Project 4 starts here). Extend them: boundaries, negative cases, login, shop,
session, responsive, accessibility."""
import pytest
from markers import known
from playwright.sync_api import expect
from test_data import unique_email


@pytest.mark.smoke
def test_valid_registration_succeeds(register_page, login_page):
    register_page.open().register("Asha Rao", unique_email(), 35, "Passw0rdOK")
    expect(login_page.status).to_contain_text("Registration successful")            # REQ-007


@pytest.mark.parametrize("age", [17, "abc", "", "25.5", pytest.param(61, marks=known("DEF-002: age 61
accepted although REQ-003 maximum is 60"))])
def test_invalid_age_is_rejected(register_page, age):                               # REQ-003: invalid
classes + neighbours of the boundaries
    register_page.open().register("Asha Rao", unique_email(), age, "Passw0rdOK")
    expect(register_page.alerts.first).to_be_visible()


@known("DEF-001: age 18 rejected although REQ-003 allows 18-60 inclusive")
def test_age_lower_boundary_18_accepted(register_page, login_page):
    register_page.open().register("Asha Rao", unique_email(), 18, "Passw0rdOK")
    expect(login_page.status).to_contain_text("Registration successful")
```

automation/ui/conftest.py

```
"""UI fixtures. Environment: BASE_URL (default http://localhost:8000), BUILD (1.0 or 1.1)."""
import os

import pytest
from login_page import LoginPage
from register_page import RegisterPage
from shop_page import ShopPage
from test_data import valid_user

BASE = os.getenv("BASE_URL", "http://localhost:8000").rstrip("/")
BUILD = os.getenv("BUILD", "1.0")
```

```
@pytest.fixture(scope="session")
def browser_type_launch_args(browser_type_launch_args):
    """Optional: point Playwright at an existing Chromium/Chrome (CHROMIUM_EXECUTABLE) when `playwright
install` is not possible."""
    exe = os.getenv("CHROMIUM_EXECUTABLE")
    return {**browser_type_launch_args, **({"executable_path": exe} if exe else {})}


@pytest.fixture(scope="session")
def build() -> str:
    return BUILD


@pytest.fixture
def app_url() -> str:
    return f"{BASE}/?build={BUILD}"


@pytest.fixture
def register_page(page, app_url):
    return RegisterPage(page, app_url)


@pytest.fixture
def login_page(page, app_url):
    return LoginPage(page, app_url)


@pytest.fixture
def registered_user(page, app_url) -> dict:
    """Register a fresh synthetic user through the UI (each test has its own browser context, so its
own storage)."""
    user = valid_user()
    reg = RegisterPage(page, app_url).open()
    reg.register(**user)
    LoginPage(page, app_url).status.wait_for()
    return user


@pytest.fixture
def shop(page, app_url, registered_user) -> ShopPage:
    login = LoginPage(page, app_url)
    login.login(registered_user["email"], registered_user["password"])
    page.get_by_role("heading", name="Welcome", exact=False).wait_for()
    return ShopPage(page, app_url)
```

pytest.ini

```
[pytest]
testpaths = automation
pythonpath = automation/page-objects automation/fixtures
addopts = -ra --strict-markers
markers =
    smoke: critical-path checks (run on every build)
    regression: full regression pack
    api: API tests
```

```
    ui: browser tests
    a11y: accessibility checks
```

WHY `XFAIL(STRICT=TRUE)`

A known defect appears as an **expected failure** linked to its defect ID. When developers fix it, the test suddenly passes and `strict=True` turns that into a failure, forcing you to remove the marker and close the defect. The suite stays green yet honest. Never hide an unexplained failure behind `xfail`.

Selenium and Cypress at a glance

selenium-example.py

```
# Selenium (Python) - explicit wait, never time.sleep
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
try:
    driver.get("http://localhost:8000/#/login")
    WebDriverWait(driver, 10).until(EC.visibility_of_element_located((By.ID, "login-email")))
    driver.find_element(By.ID, "login-email").send_keys("nobody@example.test")
finally:
    driver.quit()
```

cypress-example.cy.js

```
// Cypress (JavaScript) - retries assertions automatically
describe('Login view', () => {
  it('shows a generic message for a wrong password', () => {
    cy.visit('http://localhost:8000/#/login');
    cy.get('#login-email').type('qa@example.test');
    cy.get('#login-password').type('wrong-password');
    cy.contains('button', 'Log in').click();
    cy.get('[role=alert]').should('contain.text', 'Invalid email or password');
  });
});
```

Page Object Model, fixtures and test data: the rules

Topic	Rules
Page Object Model	One class per page/component. Locators and actions live in the page object; **assertions live in the test**. Prefer role/label locators (`get_by_role` , `get_by_label`) over CSS/XPath; never use positional XPath such as `//div[3]/span[2]` .
Waiting	Never `sleep` . Use auto-waiting actions and web-first assertions (`expect(locator).to_be_visible()`).
Independence	Each test creates its own data and can run alone, in any order, in parallel.
Fixtures	Use fixtures for browser context, base URL, authenticated session, API client; put clean-up after `yield` .
Test data	Synthetic only; unique values (`uuid`) for anything that must be unique; data files in `test-data/` with a README; no secrets in code — read from environment variables or the CI secret store.
Reporting	JUnit XML + HTML report + traces/screenshots on failure, uploaded as CI artifacts.

Continuous integration with GitHub Actions

```
.github/workflows/qa.yml

# QA pipeline: starts the local practice targets, runs API + UI tests, uploads reports.
# Required repository secret: QA_PASSWORD (the practice API password). Never write credentials in this
file.
name: QA
on:
  pull_request:
  push:
    branches: [main]
  schedule:
    - cron: "0 2 * * 1-5"        # nightly regression on weekdays (UTC)
permissions:
  contents: read
jobs:
  api-and-ui:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    env:
      QA_EMAIL: qa@example.test            # synthetic practice user (not a secret)
      QA_PASSWORD: ${{ secrets.QA_PASSWORD }}
      PRACTICE_API_PASSWORD: ${{ secrets.QA_PASSWORD }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: automation/requirements.txt
      - run: pip install -r automation/requirements.txt
      - run: playwright install --with-deps chromium
      - name: Start practice targets
```

```
        run: |
          python3 practice-api/server.py --port 8080 &
          python3 -m http.server 8000 --directory practice-app &
          sleep 2
          curl --fail --silent http://127.0.0.1:8080/api/health
      - name: Run tests
        run: pytest automation -q --junitxml=automation/reports/junit.xml
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: qa-reports
          path: automation/reports
```

Pin actions to a major version tag at minimum (a full commit SHA is stricter). Keep secrets in **GitHub Actions secrets**, never in the YAML. Free-tier minutes and runner limits change: check the current GitHub Actions documentation before promising a client a CI budget.

Topic packs (ten-part teaching method)

5.1 Python or JavaScript/TypeScript fundamentals	
Definition	The programming basics needed to read and write test code.
Purpose	Automation is software development; weak code makes flaky tests.
Setup	Python 3.11+ or Node LTS; VS Code.
Procedure	1. Work through variables, functions, loops, collections, files, exceptions. 2. Solve 10 exercises. 3. Write a script that reads CSV test data.
Expected output	Script that prints a pass/fail table from a CSV.
Verification	Runs on a clean machine with one command.
Errors	Copy-pasting code you cannot explain.
Reporting	`journal/l5-programming.md`.
Assignment	Implement `is_valid_age` and test it with all BVA values.
Assessment	Coding quiz + code walk-through.

5.2 Playwright

Definition	A framework that drives Chromium, Firefox and WebKit with auto-waiting and tracing.
Purpose	Fast, reliable cross-browser UI automation.
Setup	`pip install pytest-playwright` (or `npm init playwright`), `playwright install` .
Procedure	1. Record with codegen to learn locators. 2. Rewrite as page objects. 3. Add assertions. 4. Run headed, then headless.
Expected output	Green tests plus trace viewer on a failure.
Verification	Run 5 times; zero flakiness.
Errors	Fixed sleeps; brittle CSS selectors.
Reporting	HTML/JUnit report; trace zip.
Assignment	Automate the A0 smoke list.
Assessment	Code review.

5.3 Selenium

Definition	The long-standing WebDriver standard for browser automation.
Purpose	Many clients already run Selenium Grid.
Setup	`pip install selenium` ; Selenium Manager fetches drivers.
Procedure	1. Open a page. 2. Use explicit waits. 3. Interact. 4. Always `quit()` .
Expected output	A passing login-failure test.
Verification	No `time.sleep` in the code.
Errors	Stale-element errors; implicit + explicit wait mixing.
Reporting	Same report format.
Assignment	Port one Playwright test to Selenium and compare effort.
Assessment	Short comparison write-up.

5.4 Cypress

Definition	A JavaScript test runner that executes inside the browser.
Purpose	Developer-friendly tests for JS teams.
Setup	Node LTS; `npm i -D cypress`.
Procedure	1. `npx cypress open`. 2. Write a spec. 3. Use `cy.intercept` to stub a network call.
Expected output	Passing spec with video/screenshots.
Verification	Break the app; test fails with a clear message.
Errors	Trying multi-tab/multi-origin flows; async/await misuse.
Reporting	Cypress run output + artifacts.
Assignment	Write 3 specs for the login view.
Assessment	Review.

5.5 pytest

Definition	The standard Python test framework with fixtures and parametrisation.
Purpose	Clean, scalable test suites with rich plugins.
Setup	`pip install pytest`; folder `automation/`.
Procedure	1. Write tests. 2. Add fixtures. 3. Parametrise BVA data. 4. Add markers (`smoke`, `regression`).
Expected output	Suite selectable by marker.
Verification	`pytest -m smoke` runs only smoke tests.
Errors	Over-complicated fixtures; shared mutable state.
Reporting	JUnit XML.
Assignment	Convert 10 boundary cases to a parametrised test.
Assessment	Review.

5.6 Postman

Definition	A GUI and runner for building, testing and documenting API requests.
Purpose	Fastest way to explore and regress-test an API.
Setup	Postman app (proprietary, free tier); Newman CLI (Apache-2.0).
Procedure	1. Build a collection. 2. Add tests. 3. Use environments. 4. Run with Newman.
Expected output	Collection + environment files; Newman run output.
Verification	Newman exits non-zero when a test fails.
Errors	Secrets stored in exported files.
Reporting	Newman JUnit report.
Assignment	Automate Level-4 collection through Newman.
Assessment	Demonstration.

5.7 API automation

Definition	Code-driven API tests with pytest and `requests`.
Purpose	Stable, fast regression and contract checks.
Setup	`pip install requests jsonschema pytest`.
Procedure	1. Fixtures for base URL, token. 2. Test positive and negative cases. 3. Validate schema. 4. Mark known defects `xfail`.
Expected output	15+ API tests including schema checks.
Verification	Seeded API defects appear as strict `xfail`.
Errors	Hard-coded ids; test order dependence.
Reporting	JUnit report.
Assignment	Project 5.
Assessment	Code review.

5.8 Page Object Model

Definition	A pattern that wraps each page's locators and actions in a class.
Purpose	One change in the UI means one change in the code.
Setup	Existing UI tests.
Procedure	1. Identify repeated locators. 2. Move them into a class. 3. Keep assertions in tests.
Expected output	Refactored suite with page objects.
Verification	Tests still pass; duplicated locators removed.
Errors	Page objects that contain assertions and business rules.
Reporting	Commit message `refactor(ui): introduce RegisterPage`.
Assignment	Refactor your Playwright tests.
Assessment	Review.

5.9 Fixtures

Definition	Reusable setup and teardown code injected into tests.
Purpose	Removes duplication and guarantees clean state.
Setup	pytest fixtures.
Procedure	1. Create session, module and function fixtures. 2. Use `yield` for cleanup. 3. Compose fixtures.
Expected output	`conftest.py` with documented fixtures.
Verification	Tests pass in any order and in parallel (`pytest -p xdist -n 2`).
Errors	Global state; hidden dependencies.
Reporting	Docstrings on fixtures.
Assignment	Write an authenticated-API fixture.
Assessment	Review.

5.10 Test data management

Definition	Creating, storing and cleaning the data that tests use.
Purpose	Most flaky tests are data problems.
Setup	`test-data/` , factories, API seeding.
Procedure	1. Generate unique data. 2. Seed via API where possible. 3. Clean up. 4. Document datasets.
Expected output	Data strategy page and factory helpers.
Verification	Two parallel runs do not collide.
Errors	Shared accounts; real data.
Reporting	`test-data/README.md` .
Assignment	Write a user factory with unique e-mails.
Assessment	Review.

5.11 CI integration

Definition	Running tests automatically on every change in a pipeline.
Purpose	Fast feedback; protects the main branch.
Setup	GitHub repository, Actions workflow, secrets.
Procedure	1. Add workflow. 2. Cache dependencies. 3. Upload reports. 4. Make the check required on pull requests.
Expected output	Green workflow run with artifacts.
Verification	Break a test on a branch; the check fails.
Errors	Secrets in YAML; tests depending on a developer's machine.
Reporting	Link to workflow run in the QA report.
Assignment	Commit the workflow and show a failing and a passing run.
Assessment	Review.

LEVEL 5 EXIT GATE

≥ 15 UI + ≥ 15 API tests, green in CI with published reports, reviewed code, and an automation coverage statement that separates *automated*, *manual-only* and *not yet tested*. Sign-off: QA Lead + Automation QA.

9. Level 6 — Advanced QA

Performance, load, stress and reliability testing; accessibility at scale; security-test coordination; database validation; coverage; and managing flaky tests.

Duration	About 4 weeks
Prerequisites	Levels 1–5 passed.
Outcome	Plans and runs a controlled performance test with a workload model and thresholds, automates accessibility checks, validates data directly in a test database, coordinates security testing with qualified people, and keeps the automation suite trustworthy.
Exit gate	Controlled performance test report + accessibility automation + database validation script + flaky-test register, all reviewed.

Performance testing: vocabulary

Test type	Question	Load shape	Typical duration
Baseline / smoke	What is performance with one user?	1–5 virtual users (VUs)	Minutes
Load	Does it meet targets at expected peak?	Ramp to expected peak and hold	20–60 min steady state
Stress	Where does it break and how does it fail?	Increase until errors or saturation	Until breaking point
Spike	Can it absorb a sudden surge and recover?	Sudden jump up and down	10–30 min
Soak (endurance)	Does it leak memory or degrade over time?	Moderate load, long hold	4–24 h
Scalability	Does adding capacity add throughput?	Repeat load at different sizes	Per configuration
Reliability	Does it recover from faults?	Load + fault injection in non-production	Per scenario

Metric	Meaning	Note
Response time (p50, p90, p95, p99)	Time to complete a request, by percentile.	Never report only the average; p95 and p99 show the user pain.
Throughput	Requests (or transactions) per second.	Compare with the workload model.
Error rate	Failed requests ÷ total requests.	Define “failed” (status ≥ 500, timeout, failed assertion).
Concurrency (VUs)	Simulated simultaneous users.	Not the same as registered users.
Saturation	CPU, memory, disk, network, connection pools.	Monitor server-side during the test.

Workload model: worked calculation

Assumptions agreed with the client and written into the test plan: **10,000 users per day**, **20 % of them in the peak hour**, each session makes **6 requests**, safety factor **2×**.

Step	Calculation	Result
Users in peak hour	10,000 × 0.20	2,000 users
User arrival rate	2,000 ÷ 3,600 s	0.556 users/s
Request rate	0.556 × 6 requests	3.33 requests/s
Target with safety factor	3.33 × 2	**6.67 requests/s**
Concurrent users (Little's law)	N = X × R, with X = 0.556 users/s and a 3-minute session (R = 180 s)	≈ 100 concurrent sessions

CONTROLLED CONDITIONS ARE MANDATORY

Run performance tests **only** on an environment the client has authorised in writing, outside business hours or on an isolated environment, with monitoring in place and a named contact who can stop the test. Never load-test production, shared third-party services or systems you do not own. A stress test against an unauthorised system is a denial-of-service attack.

Example performance script (k6)

performance/k6-products.js

```
// performance/k6-products.js   Run: k6 run performance/k6-products.js
// LOCAL practice API only - never point this at a system you are not authorised to load-test.
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 10 },   // ramp-up
```

```
    { duration: '2m',  target: 10 },   // steady state (measure here)
    { duration: '20s', target: 0 },    // ramp-down
  ],
  thresholds: {                         // agreed pass/fail criteria
    http_req_failed:   ['rate<0.01'],   // < 1 % errors
    http_req_duration: ['p(95)<500'],   // 95 % of requests under 500 ms
  },
};

const BASE = __ENV.BASE_URL || 'http://127.0.0.1:8080';

export default function () {
  const res = http.get(`${BASE}/api/products?limit=5`);
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);                             // think time
}
```

The k6 script was syntax-checked in the build environment but **not executed** (k6 is not installed there). The equivalent Locust script (`performance/locustfile.py`) *was* executed against the local practice API.

Tool	Licence	Strengths	Limitations
k6 (Grafana)	AGPL-3.0 — verify on the official repository and note the copyleft implications before bundling or modifying	Scripts in JavaScript, thresholds, CI-friendly, low resource use	No browser rendering by default; protocol-level tests
Apache JMeter	Apache-2.0 — verify on the official site	GUI plan builder, many protocols, mature plugins, CLI mode	XML plans are hard to review in Git; heavier on resources
Locust	MIT (registry-verified, v2.47.0)	Python scripts; web UI; distributed	Python GIL limits one process per core

```bash
# JMeter: build the plan in the GUI, but RUN it headless (non-GUI) for real tests
jmeter -n -t performance/products.jmx -l performance/results.jtl -e -o performance/report/

# Performance report must state: environment, build, data volume, workload model, thresholds,
# results per percentile, errors, server metrics, bottleneck analysis, and limitations.
```

Accessibility automation

automation/ui/test_accessibility.py

```
# automation/ui/test_accessibility.py  (axe-core via the Playwright page; axe-core is MPL-2.0,
registry-verified)
from pathlib import Path
import json, pytest

AXE = Path("node_modules/axe-core/axe.min.js")      # npm i -D axe-core

def test_register_view_has_no_serious_a11y_violations(register_page):
    register_page.open()
```

```
    register_page.page.add_script_tag(content=AXE.read_text(encoding="utf-8"))
    result = register_page.page.evaluate("axe.run()")
    serious = [v for v in result["violations"] if v["impact"] in ("serious", "critical")]
    assert not serious, json.dumps([(v["id"], v["help"]) for v in serious], indent=2)
```

Lighthouse (Apache-2.0, registry-verified v13.6.0): `npx lighthouse http://localhost:8000 --only-categories=accessibility --output=html --output-path=accessibility/lighthouse.html`. Treat scores as a screening aid; **manual keyboard and screen-reader checks remain mandatory** because automated tools detect only part of the possible issues.

Security test coordination

Step	QA responsibility	Security specialist responsibility
1. Scope	Record in the test plan which security-relevant behaviours QA will check and which are handed over.	Define penetration-test scope, rules of engagement, authorisation.
2. Safe checks by QA	Authentication required? Role checks? Error messages leak internals? Cookie flags? Password rules? Session timeout?	—
3. Escalate	Report suspicion privately to the named security coordinator; do **not** exploit, do not post in public channels, do not attach real secrets.	Triage, reproduce in a safe way, rate (e.g. CVSS), advise fix.
4. Retest	Retest the fix functionally.	Verify the vulnerability is closed.
5. Report	List security observations in the release risk summary.	Provide formal assessment report.

Reference for awareness (not for self-directed attacks): OWASP Top 10 and OWASP ASVS.

Database validation

> **RULES**
>
> Use a **test database** and a **read-only** account unless a test case explicitly needs writes. Never connect to production. Never copy production data into test environments without an approved masking process.

validation.sql

```
-- Row-count reconciliation after migration (source vs target)
SELECT (SELECT COUNT(*) FROM legacy.customers) AS src, (SELECT COUNT(*) FROM app.customers) AS tgt;

-- Orphan check: orders without a customer
SELECT o.id FROM app.orders o LEFT JOIN app.customers c ON c.id = o.customer_id WHERE c.id IS NULL;

-- Duplicate check on a business key that must be unique
SELECT email, COUNT(*) FROM app.customers GROUP BY email HAVING COUNT(*) > 1;
```

```
-- Derived value check: order total equals sum of lines (to the paisa)
SELECT o.id FROM app.orders o
JOIN (SELECT order_id, ROUND(SUM(qty * unit_price), 2) AS s FROM app.order_lines GROUP BY order_id) l
ON l.order_id = o.id
WHERE ROUND(o.total, 2) <> l.s;
```

Each query returns **zero rows when the data is correct** (except the first, where the two numbers must match). Save the query, the result and the date as evidence.

Test coverage

Coverage type	Definition	Caution
Requirement coverage	Requirements with ≥ 1 executed test ÷ total requirements.	Having a test is not the same as having enough tests.
Risk coverage	High-risk items tested ÷ high-risk items identified.	Needs an agreed risk register.
Automation coverage	Automated test cases ÷ total test cases (or automated requirement coverage).	Say what the denominator is.
Code coverage (statement/branch)	Percentage of code executed by tests (e.g. `coverage.py`, Apache-2.0; SonarQube shows it).	100 % does not prove correctness; do not set it as a vanity target.

Flaky test management

A **flaky test** passes and fails on the same code without changes. Flaky tests destroy trust: people start ignoring red builds.

Common cause	Fix
Fixed sleeps and race conditions	Use auto-waits and web-first assertions.
Shared or order-dependent data	Unique data per test; independent tests.
Unstable environment or third-party service	Stub external calls; stabilise the environment; separate contract tests.
Time, date, locale, random	Freeze time; seed randomness; explicit locale.
Resource limits in CI	Right-size runners; reduce parallelism.

1. **Detect:** rerun the failed test 10 times on the same commit; flag as flaky if results differ.
2. **Record** in the flaky-test register: ID, first seen, owner, suspected cause, ticket.
3. **Quarantine** (tag `flaky`, run separately, not blocking) for at most **10 working days**.
4. **Fix or delete.** A quarantined test that is not fixed in time is removed and its coverage gap is listed as risk.
5. **Report** the flaky rate every release. Automatic retries are allowed only if every retry is counted and reported.

Topic packs (ten-part teaching method)

6.1 Performance testing	
Definition	Measuring speed, stability and resource use under defined conditions.
Purpose	Slow or unstable systems lose users and money.
Setup	Test environment, monitoring, k6/JMeter/Locust, written authorisation.
Procedure	1. Agree workload and thresholds. 2. Prepare data. 3. Run baseline. 4. Run measured test. 5. Analyse percentiles and server metrics.
Expected output	Performance report with p50/p95/p99, throughput, errors, bottleneck notes.
Verification	Repeat the run; results within an agreed variance.
Errors	Testing a different environment than production-like; average-only reporting.
Reporting	Performance test report template.
Assignment	Project 7.
Assessment	Report review.

6.2 Load testing	
Definition	Verifying behaviour at the expected peak load.
Purpose	Confirms launch readiness.
Setup	Workload model.
Procedure	1. Ramp to peak. 2. Hold steady state. 3. Ramp down. 4. Compare with thresholds.
Expected output	Pass/fail against thresholds.
Verification	Server metrics show no saturation at peak.
Errors	Measuring during ramp-up.
Reporting	Report.
Assignment	Run the k6 script on the Practice API.
Assessment	Review.

6.3 Stress testing

Definition	Pushing beyond peak to find the breaking point and failure mode.
Purpose	Shows how the system fails and recovers.
Setup	Isolated environment; written approval.
Procedure	1. Increase load stepwise. 2. Note the point errors rise. 3. Reduce load; check recovery.
Expected output	Breaking-point and recovery description.
Verification	System recovers without manual repair.
Errors	Running on shared/production systems.
Reporting	Report section.
Assignment	Stress the local Practice API; document the limit.
Assessment	Review.

6.4 Reliability testing

Definition	Checking that the system keeps working or recovers when parts fail.
Purpose	Real systems fail; recovery must be designed and proven.
Setup	Non-production environment; fault-injection plan.
Procedure	1. List failure scenarios (service down, slow dependency, restart). 2. Inject one at a time. 3. Observe behaviour and recovery time.
Expected output	Scenario table with outcomes.
Verification	No data loss; user-visible messages are sensible.
Errors	Injecting faults without a rollback plan.
Reporting	Reliability findings.
Assignment	Stop the API during a UI session; document UI behaviour.
Assessment	Review.

6.5 Accessibility testing (advanced)

Definition	Automated and manual audit against WCAG.
Purpose	Legal exposure and inclusive design.
Setup	axe-core, Lighthouse, screen reader.
Procedure	1. Automate scans for key pages. 2. Add manual checks. 3. Triage by impact and WCAG criterion.
Expected output	Accessibility report with WCAG mapping.
Verification	No unreviewed serious/critical issues.
Errors	Treating a Lighthouse score as conformance.
Reporting	Report + defects labelled `a11y`.
Assignment	Automate the scan on three pages.
Assessment	Review.

6.6 Security test coordination

Definition	Organising safe security checks with qualified specialists.
Purpose	Protects the client and the company legally.
Setup	Scope document; named security coordinator.
Procedure	1. Record scope. 2. Run safe checks. 3. Escalate privately. 4. Retest fixes. 5. Report risk.
Expected output	Security observation log.
Verification	Escalations acknowledged in writing.
Errors	Exploiting a finding to “prove it”.
Reporting	Private report to coordinator.
Assignment	Complete the safe-check list for the Practice API.
Assessment	Oral exam on boundaries.

6.7 Database validation

Definition	Directly verifying data correctness in the database.
Purpose	Many defects are invisible in the UI.
Setup	Test DB, read-only account, SQL client.
Procedure	1. Identify invariants. 2. Write queries returning violations. 3. Run after test and migration steps.
Expected output	Query set with zero-row expectations.
Verification	Deliberately introduce an error in a copy; query must detect it.
Errors	Testing against production; writes without approval.
Reporting	`evidence/db/`.
Assignment	Write 8 validation queries for a sample schema.
Assessment	Review.

6.8 Test coverage

Definition	Measuring how much of the requirements, risks or code the tests exercise.
Purpose	Shows gaps and supports release decisions.
Setup	Traceability matrix; coverage tools.
Procedure	1. Define denominators. 2. Compute. 3. Investigate gaps. 4. Report with context.
Expected output	Coverage summary.
Verification	Numbers recomputed by a reviewer match.
Errors	Chasing a percentage.
Reporting	Release report.
Assignment	Compute requirement and automation coverage for A0.
Assessment	Review.

6.9 Flaky test management

Definition	Detecting, quarantining and fixing nondeterministic tests.
Purpose	Protects the value of automation.
Setup	CI history; flaky register.
Procedure	1. Detect via reruns. 2. Log. 3. Quarantine. 4. Fix or delete. 5. Report rate.
Expected output	Flaky register and rate trend.
Verification	Rate decreases release over release.
Errors	Retrying until green.
Reporting	Register in `docs/`.
Assignment	Introduce and fix a deliberate race condition.
Assessment	Review.

LEVEL 6 EXIT GATE

Controlled performance report, accessibility automation, database validation set and flaky-test register approved by QA Lead and, for performance, by the Performance Tester role-holder. Sign-off: QA Lead.

10. Tools We Teach and Use

Every tool is introduced with installation, licensing, use cases, alternatives and limitations. Licences marked “registry-verified” were read from npm/PyPI metadata on 4 November 2026; all others come from published vendor information that must be re-checked before purchase, bundling or quoting.

Category	Tools	Cost model
Test management	Jira, TestRail or equivalent (Zephyr, Xray, Kiwi TCMS)	Commercial / free tiers / open source
Browser automation	Playwright, Selenium	Open source (Apache-2.0)
Web testing	Browser developer tools	Free (built in)
API testing	Postman (+ Newman CLI)	Freemium / open-source CLI
API automation	pytest, `requests` and other REST clients	Open source (MIT / Apache-2.0)
Performance	k6, Apache JMeter (+ Locust)	Open source (AGPL-3.0 / Apache-2.0 / MIT)
Accessibility	axe-core, Lighthouse	Open source (MPL-2.0 / Apache-2.0)
Code quality	SonarQube	Community edition free; paid editions
Version control	Git, GitHub	Open source / SaaS
CI/CD	GitHub Actions	SaaS with free allowance
Documentation	Markdown (+ markdownlint)	Free

TOOL AND LICENCE RULES

- Use only free, permissively licensed or properly purchased software. No cracked or pirated tools — ever.
- Record every tool, version and licence in the project's `THIRD_PARTY.md` before first use on a client project.
- Copyleft licences (GPL, LGPL, AGPL, MPL) are fine for **running** a tool; ask the QA Lead before **bundling, modifying or redistributing** one.
- Before sending client data or builds to a SaaS tool (device cloud, test-management, AI assistants), get the client's written approval and check data residency and retention.
- Pin versions in lock files; upgrade deliberately and re-run the suite.

Test management and issue tracking

Jira (Atlassian)	
Category	Issue / defect tracking, agile boards
Use cases	Defects, tasks, sprint boards, workflows (the bug lifecycle), dashboards, links to requirements and test runs.
Installation	Cloud service: create a site; no install. Server/Data Center editions are separate products — check Atlassian's current offering.
Licensing / cost	Proprietary, commercial per-user subscription; a free plan for small teams exists — **verify current limits and terms**.
Alternatives	GitHub Issues/Projects, YouTrack, Linear, Redmine (open source), Bugzilla (open source).
Limitations	Needs admin discipline (fields, workflows); test management needs an add-on; cost grows with users.

TestRail (or equivalent)	
Category	Test case management
Use cases	Test cases, suites, runs, results, requirement links, reports. Equivalents: Zephyr, Xray (Jira apps), Qase, Kiwi TCMS.
Installation	TestRail Cloud (no install) or self-hosted server (vendor instructions). Kiwi TCMS: open-source, run with Docker per its documentation.
Licensing / cost	TestRail: proprietary, commercial per-user. Kiwi TCMS: open source (verify licence on its repository). Others vary.
Alternatives	Spreadsheets/CSV in Git (small projects), Xray, Zephyr, Qase, TestLink.
Limitations	Another system to maintain; export formats differ; migration between tools is laborious.

Browser and UI automation

Playwright	
Category	Browser automation and end-to-end testing
Use cases	Cross-browser UI automation, API testing, tracing, codegen, parallel runs, mobile emulation.
Installation	`npm init playwright@latest` — or — `pip install pytest-playwright` then `playwright install chromium firefox webkit`.
Licensing / cost	Apache-2.0, free (registry-verified 2026-11-04: v1.64.1).
Alternatives	Selenium, Cypress, WebdriverIO, TestCafe.
Limitations	WebKit build is not real Safari; mobile is emulation; steep learning for complex frames and shadow DOM.

Selenium	
Category	Browser automation (W3C WebDriver)
Use cases	Long-lived suites, Selenium Grid, many languages, enterprise standards.
Installation	`pip install selenium` (Selenium Manager downloads drivers) or Maven/NuGet/npm equivalents.
Licensing / cost	Apache-2.0, free (registry-verified 2026-11-04: selenium-webdriver 4.52.0).
Alternatives	Playwright, Cypress, WebdriverIO.
Limitations	More verbose; you manage waits; slower; flakiness if waits are wrong.

Cypress	
Category	JavaScript end-to-end testing
Use cases	Component and E2E tests for JavaScript teams; time-travel debugging.
Installation	`npm install --save-dev cypress` ; `npx cypress open`.
Licensing / cost	Test runner MIT (registry-verified 2026-11-04: v16.2.1). Cypress Cloud (dashboards, parallelisation) is a paid service — verify pricing.
Alternatives	Playwright, Selenium, WebdriverIO.
Limitations	JavaScript/TypeScript only; one browser tab; multi-origin flows need special handling.

Browser developer tools	
Category	Web testing and diagnosis
Use cases	DOM/CSS inspection, console errors, network traces and HAR, storage and cookies, device emulation, throttling.
Installation	Built into Chrome, Edge, Firefox, Safari (Develop menu must be enabled). Press `F12`.
Licensing / cost	Free.
Alternatives	Fiddler, Charles Proxy, mitmproxy (use only on traffic you are authorised to inspect).
Limitations	Emulation is not a real device; HAR files may contain secrets — sanitise before sharing.

API testing and automation

Postman and Newman	
Category	API testing
Use cases	Exploring APIs, collections, environments, test scripts, mock servers, documentation; Newman runs collections in CI.
Installation	Postman desktop app from the official site; `npm install -g newman`.
Licensing / cost	Postman: proprietary freemium — verify current plan limits. Newman: Apache-2.0 (registry-verified 2026-11-04: v6.2.3).
Alternatives	Bruno, Insomnia, Hoppscotch, REST Client for VS Code, curl.
Limitations	Cloud sync raises data-handling questions; collections in JSON are noisy to review in Git.

pytest + requests (REST clients)	
Category	API automation framework
Use cases	Code-driven API tests with fixtures, parametrisation, schema validation, CI integration.
Installation	`pip install pytest requests jsonschema pytest-html`.
Licensing / cost	pytest MIT (v9.2.0), requests Apache-2.0 (v2.34.2), jsonschema MIT (v4.26.0), pytest-html MPL-2.0 (v4.2.0) — registry-verified 2026-11-04.
Alternatives	REST Assured (Java), Karate, httpx, Supertest (JS), Postman/Newman.
Limitations	Needs programming skill; you must design your own structure and reporting.

Performance

k6	
Category	Load and performance testing
Use cases	Scripted load, stress, spike and soak tests with thresholds; Grafana ecosystem integration.
Installation	Install from the official k6 documentation (package manager, binary or Docker image).
Licensing / cost	Open source, AGPL-3.0 (from training knowledge — **verify on the official repository** and read the copyleft terms).
Alternatives	JMeter, Locust, Gatling, Artillery.
Limitations	Protocol-level (no full browser by default); JavaScript runtime is k6-specific, not Node.

Apache JMeter	
Category	Load and performance testing
Use cases	Many protocols (HTTP, JDBC, JMS...), GUI plan design, distributed testing, HTML dashboards.
Installation	Install a current Java runtime, download the binary from jmeter.apache.org, unzip, run `bin/jmeter` ; run tests with `-n` (non-GUI).
Licensing / cost	Apache-2.0 (verify on the official site).
Alternatives	k6, Locust, Gatling, BlazeMeter (commercial).
Limitations	GUI is for building, not running load; XML test plans are hard to diff; memory-hungry.

Locust	
Category	Python load testing
Use cases	Load tests written as Python code, with a live web UI.
Installation	`pip install locust` .
Licensing / cost	MIT (registry-verified 2026-11-04: v2.47.0).
Alternatives	k6, JMeter, Gatling.
Limitations	Python GIL: scale with multiple processes/workers.

Accessibility

axe-core	
Category	Automated accessibility engine
Use cases	Scans a rendered page for WCAG failures; integrates with Playwright, Selenium, Cypress, Jest.
Installation	`npm install --save-dev axe-core` (and `@axe-core/playwright` for Playwright).
Licensing / cost	MPL-2.0 (registry-verified 2026-11-04: axe-core 4.14.0, @axe-core/playwright 4.14.0). Deque also sells commercial products.
Alternatives	Pa11y (LGPL-3.0, v10.0.0), WAVE, Accessibility Insights.
Limitations	Finds only a portion of issues; cannot judge meaning (e.g. whether alt text is accurate).

Lighthouse	
Category	Quality audits: accessibility, performance, SEO, best practices
Use cases	Quick audit in Chrome DevTools or CLI; Lighthouse CI for trends.
Installation	Chrome DevTools → Lighthouse tab; or `npm install -g lighthouse`.
Licensing / cost	Apache-2.0 (registry-verified 2026-11-04: v13.6.0; Lighthouse CI @lhci/cli v0.16.0).
Alternatives	WebPageTest, PageSpeed Insights, axe DevTools.
Limitations	Lab data from one machine; scores vary run to run; accessibility score is not conformance.

Code quality, version control and CI

SonarQube	
Category	Static code analysis and quality gates
Use cases	Finds bugs, code smells and some vulnerabilities; tracks coverage and duplication; quality gate on pull requests.
Installation	Self-host (for example the official Docker image, see SonarSource documentation) and run a scanner (e.g. `@sonar/scan`) in CI; or use SonarQube Cloud.
Licensing / cost	Community Build is free and open source (LGPL-3.0 per SonarSource — verify); paid Developer/Enterprise editions add branch/PR analysis and more. Scanner npm package `@sonar/scan` is LGPL-3.0 (registry-verified 2026-11-04).
Alternatives	SonarQube Cloud, CodeQL, ESLint/Pylint/Ruff, Semgrep.
Limitations	Static analysis is not testing; false positives need triage; some features are edition-gated.

Git and GitHub	
Category	Version control and collaboration
Use cases	All QA assets as code: test cases, scripts, docs, evidence indexes; pull requests; branch protection; issues.
Installation	Install Git from git-scm.com (or package manager); create a GitHub account; `git config --global user.name` / `user.email`.
Licensing / cost	Git: GPL-2.0 (verify). GitHub: SaaS; free and paid plans — verify limits (private repos, Actions minutes, storage).
Alternatives	GitLab, Bitbucket, Azure DevOps.
Limitations	Large binary evidence (videos) bloats repos — store in an artefact store and link it.

GitHub Actions	
Category	CI/CD automation
Use cases	Run tests on pull request, nightly regression, publish reports, gate merges.
Installation	Add YAML files under `.github/workflows/` in the repository.
Licensing / cost	Service with a free allowance that differs for public/private repositories — verify current minutes and limits.
Alternatives	GitLab CI, Jenkins, CircleCI, Azure Pipelines, Buildkite.
Limitations	Runner time limits; secrets handling must be disciplined; third-party actions are a supply-chain risk (pin versions).

Documentation

Markdown (CommonMark)	
Category	Lightweight documentation format
Use cases	READMEs, test plans, test cases, reports, defect files — all reviewable in Git.
Installation	Any text editor; VS Code preview; optional `markdownlint` for style checks.
Licensing / cost	Free. markdownlint-cli is MIT (verify).
Alternatives	AsciiDoc, reStructuredText, Word/Docs (for client-facing formats).
Limitations	Tables are awkward for big matrices (use CSV); no built-in diagrams (use Mermaid or images).

11. Mandatory Client QA Lifecycle: Stages 01–16

Every client project follows these 16 stages in order. Each stage lists its objective, responsibilities, inputs, procedure, deliverables, evidence, acceptance criteria and **approval gate**. A stage is closed only when its gate is signed by the named approver. Stages may overlap in time (for example, automation starts while manual execution continues), but a gate must never be skipped.

01 Qualification → 02 Requirements → 03 Strategy → 04 Planning → 05 Test cases → 06 Environment → 07 Execution → 08 Defects → 09 Automation → 10 API / integration → 11 Performance → 12 Regression → 13 UAT → 14 Release rec. → 15 Prod smoke → 16 Maintenance

Approval gates at a glance

Stage	Gate	Approver
01 Client Qualification	G1 · Proceed / decline	Test Manager
02 Requirement Analysis	G2 · Requirement baseline	Client (product owner)
03 Test Strategy	G3 · Strategy approved	Test Manager + client
04 Test Planning	G4 · Plan approved	Test Manager + client PM
05 Test Case Development	G5 · Test design approved	QA Lead
06 Test Environment Setup	G6 · Entry gate	QA Lead + client tech lead
07 Manual Test Execution	G7 · Execution cycle complete	QA Lead
08 Defect Management	G8 · Defect gate (applies until release)	Product owner + QA Lead
09 Automation Development	G9 · Automation accepted	QA Lead + Automation QA (independent reviewer)
10 API and Integration Testing	G10 · API/integration results accepted	QA Lead
11 Performance Testing	G11a · Pre-run approval / G11b · Report approval	Client (pre-run) · Test Manager (report)
12 Regression Testing	G12 · Regression exit	QA Lead
13 UAT Support	G13 · UAT acceptance	Client business owner
14 Release Recommendation	G14 · Release decision	Test Manager (report) + client release board (decision)
15 Production Smoke Testing	G15 · Post-release confirmation	Release Coordinator + client

Stage	Gate	Approver
16 Maintenance	G16 · Periodic health review	QA Lead

DO NOT START EXECUTION WITHOUT THE ENTRY GATE

Stage 07 must not begin until Gate G6 (environment and entry criteria) is signed. Testing on an unready environment produces false failures, wasted effort and unreliable metrics.

Role summary across stages (RACI)

R = responsible, A = accountable, C = consulted, I = informed. "—" means not involved. Each project's test plan may refine this table.

Stage	Junior QA	Manual QA	Automation QA	API Tester	Perf Tester	QA Lead	Test Manager	Release Coord.
01 Qualification	—	—	—	—	—	C	A/R	I
02 Requirements	I	R	C	C	C	A	I	—
03 Strategy	—	C	C	C	C	R	A	I
04 Planning	—	C	C	C	C	R	A	C
05 Test cases	R	R	C	C	—	A	I	—
06 Environment	I	R	C	C	C	A/R	I	C
07 Execution	R	R	I	I	—	A	I	I
08 Defects	R	R	R	R	R	A	I	I
09 Automation	—	C	A/R	R	—	A	I	—
10 API/integration	—	C	R	R	—	A	I	—
11 Performance	—	—	C	C	R	C	A	I
12 Regression	R	R	R	R	—	A	I	I
13 UAT support	I	R	—	—	—	A/R	I	C
14 Release rec.	—	C	C	C	C	R	A	C
15 Production smoke	—	R	C	—	—	C	I	A/R
16 Maintenance	C	R	R	R	C	A	I	—

Stage 01 — Client Qualification

Objective	Decide whether the engagement is a good fit and understand the product, scope, testing needs and delivery expectations before committing any effort.
Responsibilities	**Responsible:** Sales / Test Manager **Accountable:** Test Manager **Consulted:** QA Lead **Informed:** Management
Inputs	Enquiry, product brief, client contact, any existing documentation.
Procedure	1. Hold a discovery call using the qualification questionnaire: product and users, technology stack, release cadence, environments, existing tests, compliance needs, timeline, budget range, decision maker. 2. Check red flags: “guarantee zero bugs”, “test directly in production”, no test environment, no requirements, nobody owns access, unrealistic deadline. 3. Assess fit and capacity: skills, tools, devices, availability of the right roles. 4. Sign a mutual NDA **before** receiving builds, data or documents. 5. Record the go / no-go decision and reasons.
Deliverables and templates	Client Qualification Record; opportunity assessment (fit score, risks); NDA.
Evidence required	Signed NDA; call notes; completed questionnaire.
Acceptance criteria	Every field is filled or marked “unknown” with an owner and a date.

APPROVAL GATE G1 · PROCEED / DECLINE

Approver: Test Manager. **Checks:** NDA signed; red flags reviewed; capacity confirmed. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 02 — Requirement Analysis

Objective	Understand exactly what the product must do by reviewing specifications, user stories, workflows and acceptance criteria.
Responsibilities	**Responsible:** QA Lead, Manual QA **Accountable:** QA Lead **Consulted:** Client product owner **Informed:** Test Manager
Inputs	Specifications, user stories, wireframes, workflow diagrams, acceptance criteria, API documentation.
Procedure	1. Collect all documents and record their versions. 2. Walk through the product with the product owner; capture workflows and roles. 3. Review each requirement with the testability checklist (clear, measurable, complete, consistent, feasible, traceable). 4. Assign requirement IDs; build the requirement register; classify risk (impact × likelihood). 5. Log ambiguities, assumptions and questions with an owner and due date. 6. Agree the requirement baseline (version and date) with the client.
Deliverables and templates	Requirement register; questions and assumptions log; initial risk register.
Evidence required	Meeting notes; list of document versions reviewed; client e-mail confirming the baseline.
Acceptance criteria	Every requirement has an ID, a risk rating and a testability verdict; every open question has an owner.

APPROVAL GATE G2 · REQUIREMENT BASELINE

Approver: Client (product owner). **Checks:** Baseline confirmed in writing; open questions owned. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 03 — Test Strategy

Objective	Choose the testing types, levels, environments, devices, browsers and tools that address the risks.
Responsibilities	**Responsible:** QA Lead **Accountable:** Test Manager **Consulted:** Client tech lead, Automation QA **Informed:** Delivery team
Inputs	Requirement register, risk register, client analytics, budget and timeline.
Procedure	1. Map each major risk to a test type and level (unit by developers; API, UI, performance, accessibility by QA). 2. Define the automation approach: what is automated, when, with which tools. 3. Define environments and test-data strategy (synthetic data only). 4. Define the browser/device/OS matrix from the client's analytics. 5. List tools, licences and costs; check licences against policy. 6. Define severity and priority scales, defect workflow, metrics and reporting cadence. 7. State the security-testing boundary and name the client's security coordinator.
Deliverables and templates	Test Strategy document (template provided).
Evidence required	Reviewed draft with comments resolved; approval record.
Acceptance criteria	Each risk has a test response; each in-scope service has a tool and an owner.

APPROVAL GATE G3 · STRATEGY APPROVED

Approver: Test Manager + client. **Checks:** Scope, tools, environments and boundaries agreed. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 04 — Test Planning

Objective	Turn the strategy into an executable plan: scope, schedule, resource allocation, risks, entry and exit criteria.
Responsibilities	**Responsible:** QA Lead **Accountable:** Test Manager **Consulted:** Client project manager **Informed:** Team
Inputs	Approved strategy, release schedule, costing sheet.
Procedure	1. Write in-scope and out-of-scope lists. 2. Estimate effort with the costing method (planning, design, execution, automation, retest, reporting, management). 3. Build the schedule with build dates and milestones; allocate roles. 4. List risks with mitigations and owners. 5. Write **entry criteria** (e.g. build deployed, smoke test passed, data loaded, environment stable) and **exit criteria** (e.g. all critical cases executed, no open Severity 1/2 without written acceptance). 6. Define suspension and resumption criteria and the change-control route.
Deliverables and templates	Test Plan (template provided).
Evidence required	Plan version in Git; client acknowledgement.
Acceptance criteria	Entry and exit criteria are measurable; plan matches the SOW and estimate.

APPROVAL GATE G4 · PLAN APPROVED

Approver: Test Manager + client PM. **Checks:** Scope, schedule and criteria agreed. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 05 — Test Case Development

Objective	Create reviewed, traceable test cases linked to requirements.
Responsibilities	**Responsible:** Manual QA, Junior QA **Accountable:** QA Lead **Consulted:** Client (critical cases) **Informed:** Developers
Inputs	Requirement register, test strategy, test plan.
Procedure	1. Create test scenarios per requirement. 2. Apply equivalence partitioning, boundary-value analysis, decision tables, state transitions, error guessing. 3. Write cases with ID, requirement link, preconditions, steps, data, expected result, priority, type tags (smoke, regression, UAT, automation candidate). 4. Peer-review every case (four-eyes); fix review comments. 5. Build the requirements traceability matrix (RTM). 6. Prepare test-data sets with documented purpose.
Deliverables and templates	Test cases (`test-cases/`); RTM; test-data specification; automation candidate list.
Evidence required	Review log; RTM showing every requirement mapped.
Acceptance criteria	100 % of requirements have at least one case; no case without a requirement; all cases reviewed.

APPROVAL GATE G5 · TEST DESIGN APPROVED

Approver: QA Lead. **Checks:** Coverage and review checks passed. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 06 — Test Environment Setup

Objective	Prepare a stable environment with the right build, data, accounts, browsers and devices.
Responsibilities	**Responsible:** QA Lead, client DevOps **Accountable:** QA Lead **Consulted:** Developers **Informed:** Test Manager
Inputs	Environment requirements, build, access request.
Procedure	1. Request or build the environment from the specification; record URL, build number, configuration. 2. Load **synthetic** test data; create individual named accounts per role (no shared admin accounts). 3. Obtain written authorisation for each access granted. 4. Install and record browser and device versions; verify monitoring and log access. 5. Agree environment refresh/reset and backup rules. 6. Run the environment-readiness checklist and a smoke test.
Deliverables and templates	`docs/test-environment.md` ; environment-readiness checklist.
Evidence required	Build number screenshot; configuration export; smoke result.
Acceptance criteria	Smoke test passes; no production data present; access list matches authorisation.

APPROVAL GATE G6 · ENTRY GATE

Approver: QA Lead + client tech lead. **Checks:** Environment readiness signed; entry criteria met. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 07 — Manual Test Execution

Objective	Execute the planned test cases and record trustworthy evidence.
Responsibilities	**Responsible:** Manual QA, Junior QA **Accountable:** QA Lead **Consulted:** Developers **Informed:** Client
Inputs	Approved cases, ready environment, test data.
Procedure	1. Verify entry criteria and the build number; run smoke tests first. 2. Execute by priority; record Pass / Fail / Blocked / Not run with date, tester, build. 3. Capture evidence per case (screenshot, video, HAR, log) using the naming convention. 4. Log defects immediately after reproducing them (Stage 8). 5. Report daily status: planned, executed, passed, failed, blocked; blockers and risks. 6. Never edit cases silently — change them through version control with a note.
Deliverables and templates	Execution log; evidence files; daily status; test execution report.
Evidence required	Execution results per case with evidence file names; build identifiers.
Acceptance criteria	Counts reconcile: passed + failed + blocked + not run = planned.

APPROVAL GATE G7 · EXECUTION CYCLE COMPLETE

Approver: QA Lead. **Checks:** Log reviewed for completeness and evidence quality. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 08 — Defect Management

Objective	Log, triage, track and verify defects with reproducible steps, expected and actual results, severity, priority and evidence.
Responsibilities	**Responsible:** All testers **Accountable:** QA Lead **Consulted:** Dev lead, product owner **Informed:** Client
Inputs	Failed test results and observations.
Procedure	1. Reproduce at least twice; search for duplicates. 2. Write the defect from the template: title, environment/build, steps, expected, actual, severity, evidence, requirement/test ID. 3. Propose severity; the product owner sets priority in triage. 4. Triage on a fixed schedule (daily in execution, otherwise twice weekly). 5. Retest fixes on the **named fixed build**, then run related regression; close only after verification. 6. Escalate stale Severity 1/2 defects; every deferral needs a written decision and appears in the risk summary.
Deliverables and templates	Defect log; weekly defect summary.
Evidence required	Defect history with state changes; retest evidence.
Acceptance criteria	All Severity 1/2 defects are fixed and verified, or formally accepted in writing by the product owner.

APPROVAL GATE G8 · DEFECT GATE (APPLIES UNTIL RELEASE)

Approver: Product owner + QA Lead. **Checks:** Open critical defects resolved or waived in writing. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 09 — Automation Development

Objective	Automate stable, repeatable and valuable test cases.
Responsibilities	**Responsible:** Automation QA **Accountable:** QA Lead **Consulted:** Developers **Informed:** Test Manager
Inputs	Automation candidate list, stable build, test data, framework standards.
Procedure	1. Select candidates by return on investment and stability. 2. Set up the framework: Page Object Model, fixtures, configuration through environment variables, reports. 3. Implement; code-review every pull request. 4. Run each new test five times locally to expose flakiness. 5. Integrate with CI; publish JUnit/HTML reports and failure artefacts. 6. Document how to run the suite; publish the automation coverage report.
Deliverables and templates	Automation code in Git; CI workflow; automation coverage report.
Evidence required	Green CI run link; code-review records.
Acceptance criteria	Flaky rate at or below the agreed target (e.g. 2 %); no secrets in code.

APPROVAL GATE G9 · AUTOMATION ACCEPTED

Approver: QA Lead + Automation QA (independent reviewer). **Checks:** Review passed; CI green; coverage stated honestly. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 10 — API and Integration Testing

Objective	Validate data exchange, authentication, error handling and dependencies between components and systems.
Responsibilities	**Responsible:** API Tester **Accountable:** QA Lead **Consulted:** Developers, client integrators **Informed:** Test Manager
Inputs	API specification (OpenAPI or document), credentials for test environment, integration diagrams.
Procedure	1. Build the API collection: positive, negative, authentication, boundary, schema checks. 2. Design integration scenarios: sequence, data mapping, retry, timeout, idempotency, failure of a dependency. 3. Stub or virtualise unavailable dependencies. 4. Verify data consistency in the test database with read-only queries. 5. Check error bodies and logs for sensitive-data leakage. 6. Hand security-relevant observations to the security coordinator.
Deliverables and templates	API test collection and automation; integration test report; schema files.
Evidence required	Request/response captures with tokens removed; Newman/pytest report.
Acceptance criteria	All specified status codes provoked; schema validation passes or defects raised.

APPROVAL GATE G10 · API/INTEGRATION RESULTS ACCEPTED

Approver: QA Lead. **Checks:** Critical defects resolved or waived; security observations handed over. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 11 — Performance Testing

Objective	Establish workload assumptions, baselines and thresholds, then test under controlled conditions.
Responsibilities	**Responsible:** Performance Tester **Accountable:** Test Manager **Consulted:** Client DevOps, developers **Informed:** Client
Inputs	Non-functional requirements, analytics, environment sizing, authorisation.
Procedure	1. Agree non-functional targets and the workload model in writing (see worked calculation). 2. Confirm the environment, data volume, monitoring and a named person who can stop the test. 3. Script and validate with one user; record the baseline. 4. Run load (and stress/soak/spike if in scope) with ramp-up, steady state and ramp-down. 5. Collect client-side and server-side metrics; analyse percentiles, errors, saturation. 6. Report results, bottlenecks and limitations; never extrapolate beyond what was measured.
Deliverables and templates	Performance test plan, scripts, raw results, performance test report.
Evidence required	Result files; monitoring screenshots; environment description.
Acceptance criteria	Thresholds met, or gaps documented with recommendations.

APPROVAL GATE G11A · PRE-RUN APPROVAL / G11B · REPORT APPROVAL

Approver: Client (pre-run) · Test Manager (report). **Checks:** Authorisation and workload model signed before running. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 12 — Regression Testing

Objective	Verify that changes have not broken existing functionality.
Responsibilities	**Responsible:** Manual QA, Automation QA **Accountable:** QA Lead **Consulted:** Developers **Informed:** Client
Inputs	Change list, regression suite, fixed build.
Procedure	1. Perform change-impact analysis; map changes to requirements and cases. 2. Select the suite: changed areas, integrating areas, critical paths, rotating sample. 3. Run automated and manual regression on the release candidate. 4. Classify each failure: product defect, test defect, flaky test, environment issue. 5. Retest fixes; update the RTM; record skipped tests with reasons.
Deliverables and templates	Regression execution report.
Evidence required	Run IDs; reports; defect links.
Acceptance criteria	All critical-path tests pass; every failure is explained.

APPROVAL GATE G12 · REGRESSION EXIT

Approver: QA Lead. **Checks:** Regression exit criteria from the plan are met. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 13 — UAT Support

Objective	Assist the client's business users to perform acceptance testing.
Responsibilities	**Responsible:** QA Lead, Manual QA **Accountable:** Client product owner **Consulted:** Developers **Informed:** Test Manager
Inputs	QA exit report, UAT scope, business users and schedule.
Procedure	1. Agree UAT scope and entry criteria (QA exit met). 2. Write UAT scripts in business language; prepare accounts, data and the environment. 3. Brief users; provide a support channel during UAT. 4. Log UAT issues in the same tracker with a `UAT` label; triage with the client daily. 5. Retest fixes; report UAT status; obtain the signed UAT acceptance.
Deliverables and templates	UAT plan; UAT scripts; UAT report.
Evidence required	Completed UAT scripts; issue list; signed acceptance.
Acceptance criteria	Client business owner signs UAT acceptance or lists conditions.

APPROVAL GATE G13 · UAT ACCEPTANCE

Approver: Client business owner. **Checks:** Signed UAT report. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 14 — Release Recommendation

Objective	Provide an evidence-based QA status and an honest unresolved-risk summary to support the release decision.
Responsibilities	**Responsible:** QA Lead **Accountable:** Test Manager **Consulted:** Release Coordinator, client release board **Informed:** Management
Inputs	All test results, defect log, metrics, UAT report, waiver records.
Procedure	1. Compile metrics: execution %, pass rate, open defects by severity, requirement coverage, automation results, performance and accessibility findings. 2. List unresolved risks and waivers with owners. 3. Compare results with the exit criteria in the plan. 4. Draft the recommendation: **Go**, **Conditional Go** (with conditions) or **No-Go**. The decision belongs to the client's release board; QA recommends. 5. Second-person review by the Test Manager; present and record the decision.
Deliverables and templates	Release QA report; QA sign-off checklist.
Evidence required	Metrics source data; signed checklist.
Acceptance criteria	Numbers reconcile with source data; no metric is selectively omitted.

APPROVAL GATE G14 · RELEASE DECISION

Approver: Test Manager (report) + client release board (decision). **Checks:** Independent review done; decision recorded. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 15 — Production Smoke Testing

Objective	After deployment, verify the agreed critical functions with minimal, safe actions.
Responsibilities	**Responsible:** Release Coordinator, Manual QA **Accountable:** Release Coordinator **Consulted:** Client DevOps **Informed:** Client
Inputs	Agreed production smoke list, test accounts, deployment window, rollback criteria.
Procedure	1. Agree before the release: the smoke list, accounts, window, who may decide a rollback. 2. Use dedicated test accounts and read-only or reversible actions only; no load; no data clean-up in production without approval. 3. Execute immediately after deployment; capture evidence. 4. Report within 30 minutes: pass, or fail with evidence and impact. 5. If rollback criteria are met, the client's authorised person decides; QA provides facts.
Deliverables and templates	Production smoke report.
Evidence required	Timestamped screenshots; build/version page.
Acceptance criteria	All listed functions pass or a documented rollback/hot-fix decision exists.

APPROVAL GATE G15 · POST-RELEASE CONFIRMATION

Approver: Release Coordinator + client. **Checks:** Smoke pass confirmed or rollback decided. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Stage 16 — Maintenance

Objective	Keep the test suites accurate, fast and trustworthy: update tests, remove obsolete ones and monitor flaky automation.
Responsibilities	**Responsible:** Automation QA, Manual QA **Accountable:** QA Lead **Consulted:** Client **Informed:** Test Manager
Inputs	Release notes, new requirements, CI history, flaky register.
Procedure	1. After each release update cases, RTM and automation for changed behaviour. 2. Retire obsolete tests (record the reason). 3. Review the flaky register; fix or delete within the policy time. 4. Hold a quarterly suite-health review: runtime, flaky rate, coverage, maintenance cost. 5. Upgrade tool and dependency versions deliberately; re-run the suite. 6. Send the monthly retainer report and hold a lessons-learned review.
Deliverables and templates	Updated suites; monthly maintenance report; suite-health review.
Evidence required	Commit history; CI trend; flaky register.
Acceptance criteria	Suite remains green on the main branch; flaky rate within target.

APPROVAL GATE G16 · PERIODIC HEALTH REVIEW

Approver: QA Lead. **Checks:** Review minutes filed; client acknowledges the report. Record the decision (approved / approved with conditions / rejected), date and approver name in `docs/qa-signoff.md`.

Change control during a project

1. Any change to scope, schedule, environments or exit criteria is raised in writing.
2. The QA Lead assesses impact on effort, risk and dates using the costing method.
3. The client approves the revised plan/quote.
4. The Test Plan version is incremented and filed before work on the change starts.

12. Professional QA Repository Structure

Every client project has one repository (or one folder in the client's repository) with this structure. A ready-made skeleton is supplied in `qa-project-template/`.

```
qa-project/

qa-project/
├── README.md                    # purpose, how to run, owners, links
├── THIRD_PARTY.md               # every tool + version + licence
├── docs/
│   ├── test-strategy.md
│   ├── test-plan.md
│   ├── requirements-traceability.md
│   ├── test-environment.md
│   ├── release-report.md
│   └── qa-signoff.md
├── test-cases/
│   ├── functional/
│   ├── regression/
│   ├── integration/
│   └── acceptance/
├── automation/
│   ├── ui/                      # Playwright / Selenium / Cypress tests
│   ├── api/                     # pytest + requests, Postman/Newman
│   ├── fixtures/
│   ├── page-objects/
│   └── reports/                 # generated; git-ignored
├── performance/                 # k6 / JMeter scripts, results summaries
├── accessibility/               # axe / Lighthouse configs and findings
├── test-data/                   # synthetic data only
├── evidence/                    # screenshots, logs (index in Git, bulk in artefact store)
├── defects/                     # defect reports (if not in a tracker)
└── .github/
    └── workflows/               # CI pipelines
```

SOFTWARE SOLUTIONS

Folder	Contents	Rules
`docs/`	Strategy, plan, traceability matrix, environment description, release report, QA sign-off.	Markdown, one file per document, version in the header, changes through pull requests.
`test-cases/`	Cases grouped by type: functional, regression, integration, acceptance.	One CSV or Markdown file per feature; ID pattern `TC-<area>-<nnn>`; each case links a `REQ-` ID.
`automation/ui`, `api`	Executable tests.	Independent tests; no secrets; code review before merge.
`automation/page-objects`, `fixtures`	Reusable page classes and fixtures.	Locators and actions only; fixtures documented.
`automation/reports/`	JUnit XML, HTML, traces.	Generated; listed in `.gitignore`; published as CI artefacts.

Folder	Contents	Rules
`performance/`	Scripts, workload model, result summaries.	Raw result files > 5 MB go to the artefact store.
`accessibility/`	Scan configuration, WCAG findings, manual-check records.	Findings reference WCAG success criteria.
`test-data/`	Synthetic data files and a README.	Never real personal data or production extracts.
`evidence/`	Index of evidence and small files.	Name pattern `<date>_<TC-id>_<result>_<n>.<ext>` ; large videos live in the artefact store, linked from the index.
`defects/`	Defect write-ups when no tracker is available.	One file per defect: `DEF-nnn.md`.
`.github/workflows/`	CI pipelines.	Secrets only via CI secret store.

NEVER COMMIT THESE

- Passwords, API keys, access tokens, private keys, certificates, `.env` files with real values.
- Sensitive customer data or personal data of any real person.
- Production database extracts or dumps, even “just a small sample”.
- Unsanitised HAR files, screenshots showing tokens, e-mails or personal data.
- Client confidential documents in public repositories.

If a secret is committed by accident

1. **Stop** and tell the QA Lead immediately; do not try to hide it.
2. **Revoke or rotate** the secret at its source first (the commit history cannot be trusted to be private).
3. Remove it from the repository history using the approved procedure from the QA Lead (rewriting history is a controlled operation).
4. Record an incident note: what, when, who was told, what was rotated.
5. Add a prevention step (pre-commit secret scan, push protection, `.gitignore` rule).

Starter `.gitignore` for QA repositories

```
.gitignore

# secrets and local config
.env
*.pem
*.key
secrets/
# generated output
automation/reports/
playwright-report/
```

```
test-results/
performance/results/
*.har
# environments and dependencies
.venv/
node_modules/
__pycache__/
.pytest_cache/
.DS_Store
```

Git workflow and conventions

Topic	Convention
Branches	`main` is always releasable. Work in `qa/<initials>/<topic>`; open a pull request; at least one reviewer (the QA Lead for strategy, plan, sign-off).
Commits	`<type>(<scope>): <summary>` with types `docs`, `test`, `fix`, `refactor`, `ci`, `chore`. One logical change per commit.
Pull requests	Describe what changed, why, how it was verified; link requirement or defect IDs.
Protected main	Require reviews and passing CI checks before merge; forbid force-push.
Releases	Tag the QA baseline of each release: `qa-<release>-<yyyymmdd>`.
Retention	Keep evidence for the period in the contract (default: until 12 months after final acceptance); delete securely afterwards.

`THIRD_PARTY.md` entry format

```
THIRD_PARTY.md

| Tool / package | Version | Licence | Used for | Source / checked on |
|---|---|---|---|---|
| Playwright (pytest-playwright) | 1.64.1 | Apache-2.0 | UI automation | npm/PyPI metadata, 2026-11-04
|
| pytest | 9.2.0 | MIT | Test runner | PyPI metadata, 2026-11-04 |
```

13. Test Documentation Standards

Eleven controlled documents are produced on every full project. Each has a template in the Templates chapter and a fixed place in the repository.

Document	Purpose	Produced at	Owner	Approver	Repo path
Test Strategy	How we will test and why.	Stage 03	QA Lead	Test Manager + client	`docs/test-strategy.md`
Test Plan	What, when, who, criteria, risks.	Stage 04	QA Lead	Test Manager + client	`docs/test-plan.md`
Test Scenario	High-level conditions to test.	Stage 05	Manual QA	QA Lead	`test-cases/`
Test Case	Precise repeatable test.	Stage 05	Manual QA	QA Lead (review)	`test-cases/<type>/`
Requirements Traceability Matrix	Maps requirements ↔ cases ↔ defects ↔ automation.	Stages 02–14	QA Lead	QA Lead	`docs/requirements-traceability.md`
Defect Report	Reproducible failure description.	Stage 08	Reporter	Triage	Tracker or `defects/`
Test Execution Report	Results of an execution cycle.	Stage 07, 12	QA Lead	Test Manager	`docs/`
Automation Coverage Report	What is automated, stable and running.	Stage 09, 16	Automation QA	QA Lead	`docs/`
Performance Test Report	Workload, thresholds, results, bottlenecks.	Stage 11	Performance Tester	Test Manager + client	`performance/`
UAT Report	Client acceptance results.	Stage 13	QA Lead	Client business owner	`docs/`
Release QA Sign-off	Recommendation, evidence, residual risk.	Stage 14	QA Lead	Test Manager + client	`docs/qa-signoff.md`, `docs/release-report.md`

Writing standards

- Every document starts with a header block: **ID, title, version, status (draft / in review / approved), author, reviewer, date**.
- Use plain, unambiguous language. Avoid “works correctly”, “should be fine”, “etc.”, “as expected”.
- State numbers with units and the date/build they refer to.
- Cite evidence by file name or link; do not paste unreferenced claims.

- Separate **facts** (observed), **interpretation** (what it means) and **recommendation** (what we suggest).
- Version every change; approved documents change only through a new version with a change note.
- Mark client-confidential documents and store them only in approved locations.

Status and approval vocabulary

Status	Meaning
Draft	Author is still writing; not for client use.
In review	Internal or client review in progress; comments tracked.
Approved	Signed by the named approver; baseline for further work.
Superseded	Replaced by a newer approved version; retained for history.

14. QA Metrics

Metrics support decisions; they are not a scorecard for people. Every metric has a written formula, a data source and an owner before the project starts.

Metric	Formula	Worked example	Reading it correctly
Test execution %	executed ÷ planned × 100, where executed = passed + failed	(342+30) ÷ 400 = **93.0 %**	Progress. Blocked and not-run cases are listed separately.
Pass rate	passed ÷ executed × 100	342 ÷ 372 = **91.9 %**	Quality signal for the executed scope only.
Fail rate	failed ÷ executed × 100	30 ÷ 372 = **8.1 %**	Pass rate + fail rate = 100 %.
Defect density	valid defects ÷ size (KLOC, function points or requirements)	46 ÷ 23 KLOC = **2.0 per KLOC**	Compare like with like (same size unit, same period).
Defect leakage	defects found after release ÷ (found in QA + found after release) × 100	4 ÷ (46+4) = **8.0 %**	Defect-removal efficiency = 100 − leakage = 92.0 %.
Defect reopening rate	reopened ÷ fixed × 100	6 ÷ 40 = **15.0 %**	High values point at fix quality or poor retest steps.
Requirement coverage	requirements with ≥ 1 executed test ÷ total requirements × 100	114 ÷ 120 = **95.0 %**	List the uncovered requirements by name.
Automation coverage	automated cases ÷ total cases × 100 (state the denominator)	160 ÷ 400 = **40.0 %**	Also report automation pass rate and runtime.
Flaky test rate	tests with inconsistent results ÷ automated tests × 100	8 ÷ 160 = **5.0 %**	Target example: ≤ 2 %; trend matters.
Mean time to detect (MTTD)	average of (detection time − time the defect was introduced / build delivered)	(1 + 2 + 2 + 3 + 4.5) ÷ 5 = **2.5 days**	Shorter means earlier testing.
Mean time to resolve (MTTR)	average of (closure time − report time)	(2 + 3 + 4 + 6 + 7.5) ÷ 5 = **4.5 days**	Report by severity; do not mix S1 with S4.

Example data used above: 400 planned cases = 342 passed + 30 failed + 8 blocked + 20 not run; 46 valid defects found in QA and 4 in production; 40 defects fixed, 6 reopened.

WHY METRICS MUST NEVER BE MANIPULATED

A metric that is gamed stops measuring quality and starts hiding risk. The cost surfaces later: a release goes live with unknown defects, the client loses trust, and the company's evidence becomes worthless in a dispute. We report the truth, including bad news, as early as possible.

Common manipulations and the controls that prevent them

Manipulation	Why it is wrong	Control
Counting "not run" or blocked cases as passed	Inflates pass rate and execution %.	Formula fixed in advance; counts must reconcile to the planned total.
Splitting one test into many (or merging many) to change counts	Case counts become meaningless.	Test-case design standard; review for atomicity; report requirement coverage next to case counts.
Lowering severity or relabelling defects as "enhancements"	Makes the open-defect picture look better.	Severity defined by the table, challenged in triage; changes need a recorded reason.
Closing defects without verified retest	Defect counts fall while risk remains.	Only the tester closes; closure needs build number and evidence.
Excluding failed or flaky tests from the denominator	Pass rate appears higher.	Report flaky tests separately and keep them in the totals.
Reporting average response time only	Hides the slow tail users experience.	Always give p90, p95, p99 and error rate.
Rewarding people for defect counts or pass percentages	Encourages gaming and conflict with developers.	Appraise report quality, reproducibility, coverage and teamwork instead.
Selecting the "good" builds or runs	Cherry-picking.	Publish all runs of the cycle; keep raw data in the repository.

RULES FOR HONEST REPORTING

- Publish the formula, the date, the build and the data source next to every number.
- A second person reconciles the report to raw data before it leaves the company.
- Show trends, not only snapshots; explain context (for example, test scope changed).
- If a target is missed, say so, explain why, and state the risk and the recommended action.
- Never present a metric as proof that the product is "safe" or "bug-free".

15. Costing and Pricing

How to cost a QA engagement from first principles, convert cost into a selling price, and check profitability. All rupee figures here are **illustrative assumptions** that show the method; replace them with the company's real salaries, utilisation, vendor quotes and tax rules before quoting any client.

Cost components

Component	How to calculate
Test planning hours	Strategy + plan + estimate + risk register effort (QA Lead and Manual QA).
Manual testing hours	Requirement analysis + test design + environment support + execution cycles + defect handling.
Automation development	Candidates × average build time per test + framework set-up + CI + review + stabilisation.
Test environment costs	Cloud environment (instances, storage, data transfer) for the period, or client-provided (then 0, with an assumption).
Device and browser costs	Device-cloud plan, purchased devices (depreciation over use period), OS licences.
Tool licences	Seats × price × months for commercial tools; open-source tools cost 0 licence but still cost set-up time.
Performance infrastructure	Load-generator machines, monitoring, data volume, test-run hours.
Reporting	Report preparation, review, client presentations.
Retesting	Defect retests and regression after fixes (typically 15–25 % of execution effort).
Maintenance	Suite updates after releases; flaky test fixes; tool upgrades (charged as retainer or per release).
Project management	Planning, status reporting, meetings, client communication (5–15 % of delivery hours).
Contingency	Allowance for uncertainty; 10–20 % depending on requirement stability and access risk.

Step 1 — Hourly cost rate per role

```
formula

cost per productive hour = monthly cost-to-company (CTC) ÷ productive hours per month
productive hours            = 160 working hours × 80 % utilisation = 128
Example (Manual QA):  ₹45,000 ÷ 128 = ₹351.56  →  rounded ₹352 per hour
```

Role	Monthly CTC (assumed)	Cost per hour	Day rate at 40 % margin (8 h)
Junior QA Tester	₹30,000	₹234	₹3,200

Role	Monthly CTC (assumed)	Cost per hour	Day rate at 40 % margin (8 h)
Manual QA Engineer	₹45,000	₹352	₹4,800
Automation QA Engineer	₹70,000	₹547	₹7,200
API Tester	₹60,000	₹469	₹6,400
Performance Tester	₹75,000	₹586	₹8,000
QA Lead	₹1,00,000	₹781	₹10,400
Test Manager	₹1,30,000	₹1,016	₹13,600

Step 2 — Estimate hours (worked example)

Scenario: release QA for a mid-size e-commerce web application, four weeks: manual and exploratory testing, 25 automated UI tests with CI, API/integration testing, one controlled performance campaign, two execution cycles, retest and sign-off.

Activity	Hours	Labour cost
Test strategy and planning	24	₹15,312
Requirement analysis and test design	118	₹40,248
Environment and data set-up	12	₹5,940
Manual execution (two cycles)	120	₹35,160
Defect management and retesting	40	₹12,192
Automation development (25 UI tests + CI)	86	₹48,446
API and integration testing	40	₹18,760
Performance testing (one controlled campaign)	32	₹18,752
Regression cycle	32	₹11,880
Reporting and QA sign-off	14	₹10,934
Project management (10 % of delivery hours, Test Manager)	52	₹52,832
Total labour	**570**	**₹2,70,456**

Non-labour direct costs (assumptions)	Cost
Test environment (cloud, 4 weeks)	₹12,000
Device/browser cloud (1 month, 1 parallel session)	₹9,000
Tool licences (test management, 3 seats × 1 month)	₹6,000
Performance load-generator infrastructure	₹4,000
Total non-labour	**₹31,000**

Step 3 — Cost, price, margins

Line	Calculation	Amount
Labour	570 h at an average ₹474/h	₹2,70,456
Non-labour direct costs	Table above	₹31,000
Contingency	10 % × (₹2,70,456 + ₹31,000)	₹30,146
Total delivery cost (COGS)	Labour + non-labour + contingency	**₹3,31,602**
Selling price (ex-tax)	COGS ÷ (1 − 0.40), rounded to ₹1,000	**₹5,53,000**
Gross profit	Price − COGS	₹2,21,398
Gross margin	Gross profit ÷ Price	**40.0 %**
Variable costs	Non-labour direct costs (salaried labour is fixed in the short term)	₹31,000
Contribution	Price − variable costs	₹5,22,000
Contribution margin	Contribution ÷ Price	**94.4 %**

GROSS MARGIN VERSUS CONTRIBUTION MARGIN

Gross margin shows profitability after *all* direct costs including salaried labour. **Contribution margin** shows what the project adds towards *fixed* costs after only the costs that vary with taking the project (environments, licences, infrastructure, subcontractors). A project can have a modest gross margin yet a high contribution margin; use both.

Step 4 — Break-even

```
formula

Break-even (projects/month) = monthly fixed costs ÷ contribution per project
Fixed costs (illustrative) = team CTC ₹5,10,000 + overheads ₹1,50,000 = ₹6,60,000 per month
Contribution per project   = ₹5,22,000
Break-even                 = ₹6,60,000 ÷ ₹5,22,000 = 1.26 projects of this size per month
```

```
Break-even in billable hours = 1.26 × 570 h = 721 hours per month
Team capacity                = 7 people × 128 productive hours = 896 hours = 1.57 projects
Break-even uses 80 % of capacity  →  margin of safety = 20 %
```

Interpretation: below the break-even volume the department makes a loss however good each project's gross margin looks. Here break-even already needs about four-fifths of the team's capacity, which leaves little room for bench time, sickness, training or a delayed client. Track break-even and utilisation every month; they set the pipeline target.

Sensitivity of price to target margin

Target gross margin	Price = COGS ÷ (1 – margin)	Price vs 40 % case
30 %	₹4,74,000	-14.3 %
35 %	₹5,10,000	-7.8 %
40 %	₹5,53,000	+0.0 %
50 %	₹6,63,000	+19.9 %

Four pricing models

Model	How the price is set	Best for	Main risk and control
Project-based	Estimate by activity (above) → price with milestones (e.g. 30 % at plan approval, 40 % at execution complete, 30 % at sign-off).	Defined releases with clear scope	Scope creep → change-request procedure; milestones tied to approval gates.
Fixed-scope	Fixed price for an explicit scope list (features, platforms, cycles). Includes contingency and a risk premium. Extra cycles or scope are priced at the rate card.	Clients who need budget certainty and have stable requirements	Under-estimation → strict assumptions, exclusions, maximum cycles (e.g. 2 execution + 1 retest cycle).
Time-based (T&M)	Role day/hour rates × actual timesheets, with an optional cap. Rates from the table above.	Uncertain scope, discovery work, staff augmentation	Client budget anxiety → weekly timesheets, burn-down report, agreed cap and approval above cap.
Monthly QA retainer	Committed hours per month at a blended rate + tool pass-through; unused hours expire (or roll over by agreement); overage at an agreed rate.	Continuous releases, maintenance, regression upkeep	Under-use or over-use → monthly utilisation report; quarterly re-sizing.

Retainer worked example

Role	Hours/month	Cost/hour	Cost
QA Lead	8	₹781	₹6,248
Manual QA Engineer	40	₹352	₹14,080
Automation QA Engineer	24	₹547	₹13,128
API Tester	8	₹469	₹3,752
Tools and reporting pass-through	—	—	₹5,000
Total cost to deliver	**80**		**₹42,208**

```
formula

Retainer price (35 % target margin, steady work) = ₹42,208 ÷ (1 − 0.35) = ₹64,935  →  quote ₹65,000 per month for 80 committed hours
Margin check = (₹65,000 − ₹42,208) ÷ ₹65,000 = 35.1 %
Overage rate (blended cost per hour ₹465 ÷ (1 − 0.40)) ≈ ₹800 per extra hour
```

QUOTING RULES

- Quote only from a written estimate; list assumptions and exclusions in the proposal.
- Verify every tool licence and cloud cost on the vendor's current price page on the day of quoting.
- Add taxes (e.g. GST) as separate lines according to current rules; never bury them.
- Never promise outcomes (“zero defects”, “100 % automation”); sell evidence, coverage and risk reduction.
- A second person (Test Manager) reviews every quote before it is sent.

16. Employee Roles

Eight roles define who does what. A person may hold more than one role on a small project, but the reviewer of any work product must never be its author.

Junior QA Tester → Manual QA Engineer → Automation QA / API Tester / Performance Tester → QA Lead → Test Manager

Role	One-line purpose	Primary level gate
Junior QA Tester	Execute and report under supervision.	Levels 1–2
Manual QA Engineer	Design and run risk-based manual tests.	Levels 1–3
Automation QA Engineer	Build maintainable automation and CI.	Levels 1–5
API Tester	Validate interfaces and integrations.	Levels 1–4
Performance Tester	Run controlled performance campaigns.	Levels 1–6
QA Lead	Own quality approach and technical sign-off.	All levels + leadership assessment
Test Manager	Own governance, budget and client trust.	Management assessment
Release Coordinator	Coordinate release readiness and sign-off.	Levels 1–2 + release process assessment

SEPARATION OF DUTIES

Author ≠ reviewer. The tester who ran a test does not approve its sign-off; the QA Lead who wrote the recommendation does not approve it alone; the person who prepared a quote does not approve it. This is the department's main protection against errors and against pressure to “make the numbers look good”.

Role cards

Junior QA Tester	
Purpose	Execute test cases, record evidence and report defects accurately under supervision.
Core responsibilities	• Execute assigned test cases and exploratory sessions; record Pass / Fail / Blocked with evidence. • Log defects with reproducible steps; retest fixes. • Prepare and maintain synthetic test data; update traceability rows. • Write the daily learning journal; follow the 90-day plan.
Skills required	Level 1–2 knowledge; careful, honest, observant; reads requirements; basic Git; clear written English.
Typical deliverables	Executed cases with evidence; defect reports; daily status line.
Next step in career	Manual QA Engineer after Levels 1–3 + two supervised projects.

Manual QA Engineer	
Purpose	Design and execute risk-based manual tests independently across web, mobile and desktop.
Core responsibilities	• Analyse requirements; raise questions; design cases with EP, BVA, decision tables, state transitions. • Run smoke, regression, exploratory and UAT-support sessions. • Review juniors' cases and defects; maintain the RTM. • Run cross-browser, responsive and first-line accessibility checks.
Skills required	Levels 1–3; strong technique knowledge; developer tools; SQL basics; coaching skills.
Typical deliverables	Test cases, RTM, execution reports, exploratory notes.
Next step in career	Automation QA Engineer, API Tester or QA Lead.

Automation QA Engineer	
Purpose	Build and maintain reliable UI and API automation integrated with CI.
Core responsibilities	• Select automation candidates by ROI; build frameworks (POM, fixtures, data factories). • Write and review Playwright/Selenium/Cypress and pytest code; keep flaky rate low. • Maintain CI pipelines and reports; publish automation coverage. • Mentor testers in automation practices.
Skills required	Level 5; one programming language well; Git, CI/CD; debugging; clean-code habits.
Typical deliverables	Automation code, CI workflows, automation coverage report, flaky register.
Next step in career	QA Lead or Test Architect.

API Tester	
Purpose	Validate service interfaces, integrations, data exchange, authentication and errors.
Core responsibilities	• Build Postman/pytest suites with schema validation and negative testing. • Verify integration scenarios, retries, timeouts, idempotency. • Validate data in the test database with read-only queries. • Hand security-relevant observations to the security coordinator.
Skills required	Level 4; HTTP, JSON, OAuth/JWT basics; SQL; scripting.
Typical deliverables	API collections/tests, integration report, schemas.
Next step in career	Automation QA Engineer or Performance Tester.

Performance Tester	
Purpose	Plan and run controlled performance, load and stress tests and analyse bottlenecks.
Core responsibilities	• Agree workload models and thresholds in writing. • Script and execute k6/JMeter/Locust tests in authorised environments. • Collect client and server metrics; analyse percentiles, errors and saturation. • Write the performance test report with limitations.
Skills required	Level 6; statistics basics; Linux and monitoring; networking; scripting.
Typical deliverables	Workload model, scripts, performance report.
Next step in career	QA Lead or Performance Architect.

QA Lead	
Purpose	Own the quality approach and delivery of QA on projects; coach the team; sign recommendations.
Core responsibilities	• Write strategy and plan; estimate effort; assign work. • Review test design, automation and reports; run triage. • Sign the QA release recommendation (second reviewer: Test Manager). • Coach and assess trainees at level gates; keep the team honest on metrics.
Skills required	All levels; technical depth plus communication, estimation, risk thinking and leadership.
Typical deliverables	Strategy, plan, reports, approvals, coaching notes.
Next step in career	Test Manager.

Test Manager	
Purpose	Own QA commercial and delivery governance across projects: scope, budget, staffing, quality of reporting.
Core responsibilities	• Qualify clients and approve proposals, quotes and change requests. • Allocate people and budget; manage margin, utilisation and break-even. • Approve strategies/plans; act as independent reviewer of release recommendations. • Handle escalations; own client relationships and department improvement.
Skills required	Management and commercial skills; strong QA background; contracts and risk.
Typical deliverables	Quotes, portfolio dashboard, approvals, improvement plan.
Next step in career	Head of Delivery / Quality.

Release Coordinator	
Purpose	Coordinate release readiness, sign-off paperwork and production smoke testing with the client.
Core responsibilities	• Maintain the release calendar and checklists; chase approvals. • Organise production smoke tests and rollback decision points with the client. • Collect evidence and file the QA sign-off package. • Communicate status to all stakeholders clearly and on time.
Skills required	Process discipline; communication; basic testing knowledge; configuration/release management concepts.
Typical deliverables	Release checklist, sign-off package, production smoke report.
Next step in career	Test Manager or Delivery Manager.

SOFTWARE SOLUTIONS

17. Practical Employee Projects

Eight projects take the trainee from manual testing to a complete client QA report. Each project is marked on the same rubric and must be reviewed by a person other than its author. Targets are the local practice systems or an explicitly authorised site.

#	Project	Level	Time	Primary deliverable
1	Test a business website manually	Level 3	5 days	`docs/test-plan.md` (lite)
2	Prepare a complete test case suite	Level 2	5 days	`test-cases/functional/shop.csv`
3	Test a REST API	Level 4	5 days	`automation/api/postman/collection.json`
4	Automate browser tests with Playwright	Level 5	5 days	`automation/ui/`
5	Automate API tests	Level 5	4 days	`automation/api/`
6	Create a regression suite	Level 2 + 5	3 days	`test-cases/regression/`
7	Perform a controlled performance test	Level 6	4 days	`performance/` scripts
8	Prepare a complete client QA report	All levels	5 days	Complete `qa-project/` repository

AUTHORISATION FIRST

Only test the Practice Shop and Practice API (running on your own machine) or a site whose owner has authorised testing in writing. Never run automation, load or scanning tools against public websites, third-party APIs or shared systems without that authorisation.

Project 1 — Test a business website manually

Level / time-box	Level 3 · 5 days
Target system	The company's own website, or a client site **with written authorisation**, or a public practice site. No authorisation, no testing.
Tasks	1. Write a one-page scope and test approach (what is in/out, browsers, devices, data). 2. Build a sitemap; list user journeys and risks. 3. Design and execute at least **40 test cases**: navigation, links, forms, content, responsive layout (360/768/1280 px), cross-browser (two browsers), first-line accessibility. 4. Log every defect with evidence; rate severity and priority. 5. Write the test execution report with an honest risk statement.
Deliverables (repository paths)	`docs/test-plan.md` (lite), `test-cases/functional/website.csv`, `evidence/`, `defects/`, `docs/test-execution-report.md`.

Acceptance criteria	Every case executed with evidence; ≥ 5 reproducible defects or documented zero-defect reasoning; counts reconcile; report states what was **not** tested.
Marking (100 points)	Planning and approach 15 · Technical quality 30 · Evidence and traceability 20 · Reporting honesty 20 · Professional conduct and timeliness 15. **Pass: 70.**

Project 2 — Prepare a complete test case suite

Level / time-box	Level 2 · 5 days
Target system	VEL Practice Shop, shop/cart/session/logout modules (REQ-010 to REQ-014) plus registration regression.
Tasks	1. Create the requirements traceability matrix. 2. Apply EP, BVA, decision table, state transition and error guessing; name the technique on each case. 3. Write at least **60 test cases** across functional, negative, boundary, session and accessibility types, each linked to a requirement. 4. Tag cases: smoke, regression, UAT, automation candidate. 5. Arrange a four-eyes review with a colleague; record and resolve every comment.
Deliverables (repository paths)	`test-cases/functional/shop.csv`, `docs/requirements-traceability.md`, review log.
Acceptance criteria	100 % requirement coverage; no duplicate cases; ≥ 25 % of cases are negative or boundary; review comments closed.
Marking (100 points)	Planning and approach 15 · Technical quality 30 · Evidence and traceability 20 · Reporting honesty 20 · Professional conduct and timeliness 15. **Pass: 70.**

Project 3 — Test a REST API

Level / time-box	Level 4 · 5 days
Target system	VEL Practice API (`qa-project-template/practice-api`) running locally.
Tasks	1. Read the contract and build a Postman collection with ≥ **40 requests**. 2. Cover positive, negative, authentication, boundary, schema and method tests for every endpoint. 3. Add assertions on status, headers, body, schema and response time. 4. Raise a defect for each deviation, with request/response evidence (tokens removed). 5. Hand over any access-control observation as a **security escalation** note, not a public defect.
Deliverables (repository paths)	`automation/api/postman/collection.json`, defects, `docs/api-test-report.md`.
Acceptance criteria	≥ 8 correct defects; no false defects against a documented behaviour; all endpoints covered; escalation note present.

Marking (100 points)	Planning and approach 15 · Technical quality 30 · Evidence and traceability 20 · Reporting honesty 20 · Professional conduct and timeliness 15. **Pass: 70.**

Project 4 — Automate browser tests with Playwright

Level / time-box	Level 5 · 5 days
Target system	VEL Practice Shop UI.
Tasks	1. Set up Playwright with pytest; configure base URL through an environment variable. 2. Create page objects for Register, Login and Shop. 3. Automate ≥ **15 tests** including parametrised boundary tests; mark known defects `xfail(strict=True)` with the defect ID. 4. Run on Chromium and Firefox; enable trace on first retry/failure. 5. Run each test 5 times; fix or remove any flaky test; document how to run.
Deliverables (repository paths)	`automation/ui/`, `automation/page-objects/`, `automation/fixtures/`, HTML/JUnit report.
Acceptance criteria	Suite green on 5 consecutive runs; no sleeps; no secrets; code review passed.
Marking (100 points)	Planning and approach 15 · Technical quality 30 · Evidence and traceability 20 · Reporting honesty 20 · Professional conduct and timeliness 15. **Pass: 70.**

Project 5 — Automate API tests

Level / time-box	Level 5 · 4 days
Target system	VEL Practice API.
Tasks	1. Create session fixtures for base URL and token (credentials from environment variables). 2. Write ≥ **15 pytest tests**, including JSON-schema validation of products. 3. Mark seeded defects as strict `xfail` with defect IDs. 4. Generate JUnit XML; run in CI. 5. Report automation coverage against the Level-4 manual cases.
Deliverables (repository paths)	`automation/api/`, `automation/api/schemas/`, CI run link, coverage note.
Acceptance criteria	Independent tests; zero shared state; schema checks present; CI green.
Marking (100 points)	Planning and approach 15 · Technical quality 30 · Evidence and traceability 20 · Reporting honesty 20 · Professional conduct and timeliness 15. **Pass: 70.**

Project 6 — Create a regression suite

Level / time-box	Level 2 + 5 · 3 days

Target system	Practice Shop build 1.0 vs build 1.1 (`?build=1.1`).
Tasks	1. Classify tests into tiers: **smoke** (≤ 15 min), **regression** (nightly), **full** (weekly). 2. Write selection rules and a change-impact table. 3. Run the suite on build 1.0 (baseline) and build 1.1. 4. Compare results; identify fixed defects, **new regressions**, and obsolete tests. 5. Report with a go/no-go style recommendation for build 1.1.
Deliverables (repository paths)	`test-cases/regression/` , `docs/regression-report.md` , updated RTM.
Acceptance criteria	At least one regression found and explained; every skipped test justified; recommendation supported by evidence.
Marking (100 points)	Planning and approach 15 · Technical quality 30 · Evidence and traceability 20 · Reporting honesty 20 · Professional conduct and timeliness 15. **Pass: 70.**

Project 7 — Perform a controlled performance test

Level / time-box	Level 6 · 4 days
Target system	VEL Practice API on **localhost only** (never a shared or third-party system).
Tasks	1. Write a one-page performance test plan: objectives, workload model (use the worked formula), thresholds (e.g. p95 < 500 ms, errors < 1 %), environment, data, stop conditions. 2. Script `GET /api/products` and `GET /api/slow?delay=200` in k6, JMeter or Locust; validate with one user. 3. Run baseline (1 VU), then load at 10, 25 and 50 VUs with ramp-up, steady state and ramp-down. 4. Record client metrics and the server's CPU/memory (e.g. `top`). 5. Report p50/p95/p99, throughput, errors, bottleneck notes and **limitations** (local machine, synthetic data).
Deliverables (repository paths)	`performance/` scripts, results summary, `docs/performance-report.md` .
Acceptance criteria	Plan approved before running; steady-state only is measured; limitations stated; no claim beyond measured data.
Marking (100 points)	Planning and approach 15 · Technical quality 30 · Evidence and traceability 20 · Reporting honesty 20 · Professional conduct and timeliness 15. **Pass: 70.**

Project 8 — Prepare a complete client QA report

Level / time-box	All levels · 5 days
Target system	Run the full lifecycle on the Practice Shop + Practice API as if it were a paying client.
Tasks	1. Produce strategy, plan, RTM, test cases, execution report, defect summary, automation coverage report, performance and accessibility summaries.

	2. Compute all metrics with the formulas in this handbook. 3. Write the release QA report with the recommendation (Go / Conditional Go / No-Go) and the unresolved-risk summary. 4. Complete the QA sign-off checklist; the QA Lead acts as second reviewer. 5. Present for 20 minutes to a mock client panel.
Deliverables (repository paths)	Complete `qa-project/` repository, `docs/release-report.md`, `docs/qa-signoff.md`, presentation.
Acceptance criteria	Numbers reconcile with source data; recommendation is consistent with evidence; risks are plain; panel questions answered honestly.
Marking (100 points)	Planning and approach 15 · Technical quality 30 · Evidence and traceability 20 · Reporting honesty 20 · Professional conduct and timeliness 15. **Pass: 70.**

Project sign-off sheet

Project	Date	Score /100	Reviewer	Pass (≥ 70)	Reviewer comments
1. Test a business website manually				☐	
2. Prepare a complete test case suite				☐	
3. Test a REST API				☐	
4. Automate browser tests with Playwright				☐	
5. Automate API tests				☐	
6. Create a regression suite				☐	
7. Perform a controlled performance test				☐	
8. Prepare a complete client QA report				☐	

18. 90-Day Employee Training Plan

Thirteen weeks (65 working days, about 90 calendar days) take a beginner from zero to a supervised, productive QA professional. Each week has daily focus areas, a required deliverable and a review gate. Use it with the Practical Workbook.

Milestone	When	Pass requirements	Decision
M1 — Foundations	End of week 4 (~Day 30)	Level 1 and Level 2 gates passed; A0 ≥ 70; one exploratory session reported.	Continue / extend by one week with coaching
M2 — Technical testing	End of week 8 (~Day 56)	Level 3 and 4 gates; Projects 1–3 ≥ 70; can write a defect that a developer reproduces first time.	Continue / re-test weak areas
M3 — Automation & advanced QA	Week 13 (Day 90)	Level 5 and 6 gates; Projects 4–8 ≥ 70; final assessment ≥ 75.	Place as Junior QA, Manual QA, or specialist track

Weekly schedule

Week and level	Daily focus	Deliverable	Review gate
Week 1 Onboarding · Level 1a	**Mon** Day 1 agenda: policies, accounts, buddy, Golden Rules **Tue** Workstation set-up; run Practice Shop and API **Wed** SDLC, STLC, QA vs QC, verification vs validation **Thu** Errors, defects, failures; seven testing principles **Fri** Read the A0 specification; write questions and the traceability skeleton	Set-up evidence; learning journal; questions list	Buddy check Friday
Week 2 Level 1b	**Mon** Test cases and scenarios **Tue** Test plans and test data **Wed** Bug lifecycle; severity and priority **Thu** Write first 10 test cases and the first defect report **Fri** Level 1 quiz and exit-gate review	10 reviewed cases; 1 defect report; quiz	**Level 1 gate** (QA Lead)
Week 3 Level 2a	**Mon** Requirement analysis and testability checklist **Tue** Equivalence partitioning **Wed** Boundary-value analysis **Thu** Decision tables, state transitions, error guessing **Fri** A0 test design: 30+ cases and RTM	A0 suite and RTM	Peer review

Week and level	Daily focus	Deliverable	Review gate
Week 4 Level 2b	**Mon** A0 execution on Chrome and Firefox **Tue** A0 execution at 360 px; evidence and defects **Wed** Retest and A0 report **Thu** Exploratory session; smoke, sanity, regression, UAT concepts **Fri** A0 review presentation to QA Lead	A0 report; defect log; exploratory sheet	**Milestone 1 (~Day 30): A0 ≥ 70; Level 2 quiz**
Week 5 Level 3a	**Mon** Browser matrix; developer tools **Tue** Responsive testing **Wed** Form testing **Thu** Navigation and session testing **Fri** Start Project 1 (manual website test)	Browser matrix; session checklist	Buddy check
Week 6 Level 3b	**Mon** Mobile testing approaches; emulation vs real devices **Tue** OS compatibility; accessibility first-line checks **Wed** Project 1 execution **Thu** Project 1 defects and report **Fri** Level 3 gate review	Project 1 package	**Level 3 gate**; Project 1 review
Week 7 Project 2 · Level 4a	**Mon** Project 2: traceability and design **Tue** Project 2: 60+ cases **Wed** Project 2: review and fixes **Thu** HTTP methods, status codes, curl **Fri** Authentication, JSON, Postman basics	Project 2 suite; Postman collection start	Project 2 review
Week 8 Level 4b	**Mon** Schema validation and negative API tests **Tue** Project 3: collection build **Wed** Project 3: defects and escalation note **Thu** Project 3 report **Fri** Level 4 gate; Milestone 2 review	Project 3 package	**Level 4 gate; Milestone 2 (~Day 56)**
Week 9 Level 5a	**Mon** Python fundamentals 1 **Tue** Python fundamentals 2; files and JSON **Wed** pytest basics; running and reading reports **Thu** Fixtures and parametrisation **Fri** Start Project 5 (API automation)	Python exercises; first API tests	Code review
Week 10 Level 5b	**Mon** Project 5 completion and schema checks **Tue** Playwright basics, locators, auto-wait **Wed** Page Object Model **Thu** Project 4: automate 15 UI tests **Fri** Cross-browser run; traces; flakiness check	Projects 4 and 5 code	Code review (independent)
Week 11 Level 5c	**Mon** GitHub Actions CI **Tue** Project 6: regression tiers and selection rules **Wed** Project 6: run on build 1.0 and 1.1 **Thu** Project 6 report **Fri** Level 5 gate	CI link; regression report	**Level 5 gate**

Week and level	Daily focus	Deliverable	Review gate
Week 12 Level 6	**Mon** Performance concepts; workload model; plan for Project 7 **Tue** k6 / JMeter / Locust scripts; validation **Wed** Project 7 runs and analysis **Thu** Accessibility automation; database validation **Fri** Flaky management; security coordination; coverage	Performance report; a11y scan; SQL checks	**Level 6 gate**
Week 13 Capstone · Assessment	**Mon** Project 8: assemble repository and documents **Tue** Project 8: metrics and recommendation **Wed** Project 8: sign-off checklist and rehearsal **Thu** Final assessment: written paper + practical **Fri** Presentation to management; role placement decision	Complete client QA report; assessment results	**Milestone 3 (Day 90): final review**

Weekly rhythm

When	Activity	Output
Every day 09:45	Stand-up with buddy (10 min): yesterday, today, blockers.	Blockers resolved or escalated.
Every day 17:30	Learning-journal entry: what I did, learned, what I do not understand.	`journal/day-nn.md` committed.
Every Friday	Review with buddy (30 min): deliverable demo, feedback, next week.	Feedback recorded in the trainee file.
End of each level	Gate review with the QA Lead.	Signed gate record.
Every 2 weeks	One-to-one with the Test Manager about progress and wellbeing.	Notes; adjustments to plan.

If the trainee falls behind

1. Identify the gap with the buddy within two working days; check understanding, not just output.
2. Add a focused coaching block (maximum 5 working days) with a specific exercise and a re-test.
3. If the gap remains after the re-test, the QA Lead and Test Manager agree a revised plan in writing.
4. Never lower an assessment pass mark to keep a schedule.

Final evidence checklist (Day 90)

- [ ] Learning repository with 90 daily journal entries.
- [ ] All level-gate records signed.
- [ ] Projects 1–8 with scores ≥ 70 and reviewer names.
- [ ] Capstone repository complete and reproducible.
- [ ] Final assessment papers and practical marksheets filed.
- [ ] Management decision and development plan recorded.

19. Assessment and Competency Framework

How the department decides, fairly and with evidence, that a trainee or employee is competent. Assessments test what people can **do** and **explain**, never only what they can memorise.

Assessment principles

- **Evidence-based:** judgement rests on reviewed work products, observed practice and oral explanation.
- **Second assessor:** the assessor is not the trainee's buddy for the same piece of work.
- **No lowering of standards:** pass marks are not reduced to meet schedules; coaching time is added instead.
- **Transparent:** rubrics are shared before the assessment; feedback is written within three working days.
- **Re-assessment:** one re-take after coaching; a second failure goes to the Test Manager for a development plan.

Level gates

Gate	Method	Pass mark	Assessor
Level 1	20-question quiz + 1 test case + 1 defect report	Quiz ≥ 80 %; work products without major comments	QA Lead
Level 2	Assignment A0 + technique quiz + exploratory report	A0 ≥ 70 / 100; quiz ≥ 80 %	QA Lead
Level 3	Cross-browser/responsive report + session and accessibility checklists + demo	Reviewed and approved	QA Lead
Level 4	Project 3 (API) + schema demo	≥ 8 valid defects; ≥ 70 / 100	QA Lead / API Tester
Level 5	Projects 4–6 + CI run + code review	Green CI; review passed; ≥ 70 / 100	Automation QA + QA Lead
Level 6	Project 7 + a11y automation + DB checks + flaky register	≥ 70 / 100; limitations stated	QA Lead / Performance Tester
Final	Written paper (40 marks) + practical (60 marks)	≥ 75 / 100 overall; ≥ 60 % in each part	Test Manager + QA Lead

Competency matrix

Proficiency scale: **0** not demonstrated · **1** foundation · **2** working (independent on routine work) · **3** proficient · **4** expert.

Competency	1 Foundation	2 Working	3 Proficient	4 Expert
C1 Testing fundamentals	Knows terms only	Explains lifecycle, principles, severity/priority correctly	Applies fundamentals to unfamiliar situations; coaches others	Shapes company standards and teaches them
C2 Test design	Writes cases from examples	Applies EP/BVA; atomic, traceable cases	Chooses techniques by risk; designs for coverage and maintainability	Designs strategy across products; reviews others' designs
C3 Execution and evidence	Executes scripts with help	Executes independently; complete evidence	Runs exploratory sessions; spots patterns and risks	Plans and steers execution across teams
C4 Defect management	Reports with help	Reproducible reports; sound severity	Triages; analyses root cause; reduces reopen rate	Defines defect process and quality gates
C5 Web, mobile, accessibility	Tests one browser/device	Cross-browser, responsive, first-line a11y checks	Builds matrices; manual a11y with screen readers	Defines compatibility and a11y strategy
C6 API and data	Calls APIs with a GUI tool	Designs positive/negative API tests with schemas	Integration scenarios; DB validation; automation	Designs contract-testing strategy
C7 Automation	Runs existing scripts	Writes POM tests with fixtures; CI basics	Designs frameworks; controls flakiness; reviews code	Architects automation and coaches
C8 Performance and non-functional	Aware of the concepts	Runs a scripted load test with guidance	Builds workload models; analyses bottlenecks	Leads capacity and reliability programmes
C9 Metrics and reporting	Reads reports	Computes metrics from raw data	Writes honest release reports and risk summaries	Designs measurement and governance
C10 Conduct and communication	Follows rules with reminders	Consistently honest, clear, punctual	Influences stakeholders; escalates well	Builds client trust and team culture

Target proficiency by role

Role	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Junior QA Tester	1	1	2	1	1	–	–	–	1	2
Manual QA Engineer	2	3	3	2	2	1	–	–	2	2
Automation QA Engineer	2	2	2	2	2	2	3	1	2	2
API Tester	2	2	2	2	1	3	2	1	2	2

Role	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Performance Tester	2	2	2	2	1	2	2	3	2	2
QA Lead	3	3	3	3	2	2	2	2	3	3
Test Manager	3	2	2	3	2	1	1	2	4	4

Marking rubrics

Test case rubric (20)	Points
ID, title and requirement link present and unique	3
Preconditions complete and verifiable	3
Steps numbered, unambiguous, repeatable by a stranger	4
Test data specified (values, not “valid data”)	3
Expected result observable and measurable	5
Atomic: one verification; correct priority and type tag	2

Defect report rubric (20)	Points
Title states what is wrong and where	2
Environment: build, browser/device, OS, date	2
Steps reproduce the defect first time for someone else	5
Expected (with requirement reference) versus actual	4
Evidence attached, secrets removed	3
Severity justified; priority proposed correctly	2
Neutral, factual tone; no blame; no duplicates	2

Report rubric (20)	Points
Numbers reconcile with source data; formulas stated	5
Coverage and gaps stated, including what was not tested	4
Risks and waivers listed with owners	4
Recommendation follows from evidence	4
Clear structure and language for the audience	3

Question banks with model answers

The banks give core questions with model answers. Trainers extend them (with ISTQB-style sample questions and company scenarios) to at least 20 questions for the Level 1 quiz and 10 for every other gate, drawn randomly. Never reveal the model answers to trainees before assessment.

Level 1 — Fundamentals

#	Question	Model answer
1	What is the difference between quality assurance and quality control?	QA is process-oriented and prevents defects (standards, reviews, training). QC is product-oriented and detects defects (testing, inspection).
2	Define error, defect and failure and give one example chain.	A human **error** creates a **defect** in a work product; running it can cause a **failure**. Example: developer writes `>` instead of `>=` (error) → age check wrong (defect) → age 18 rejected (failure).
3	Verification versus validation?	Verification: are we building the product right (against the specification)? Validation: are we building the right product (for user need)?
4	Why is exhaustive testing impossible, and what do we do instead?	Input combinations, paths and environments are effectively unlimited. We use risk, priorities and techniques (EP, BVA) to choose the most valuable tests.
5	A headline on the home page has a spelling mistake two days before a major launch. Severity and priority?	Low severity (cosmetic) but high priority (visible at launch; cheap to fix).
6	Who closes a defect, and when?	The tester, only after verifying the fix on the named fixed build using the original steps, with evidence.
7	Name five of the seven testing principles.	Testing shows presence of defects; exhaustive testing is impossible; early testing; defect clustering; pesticide paradox; context dependence; absence-of-defects fallacy (any five).
8	What makes an expected result good?	Observable, specific, measurable and unambiguous (for example “message ‘Registration successful’ is displayed and the Login view opens”), not “works correctly”.

Level 2 — Manual testing

#	Question	Model answer
1	A field accepts ages 18 to 60 inclusive. Give the two-value boundary tests.	17 (invalid), 18 (valid), 60 (valid), 61 (invalid). Three-value adds 19 and 59.
2	A password must be 8–20 characters. Which lengths do you test at the boundaries?	7, 8, 20, 21 (plus 9 and 19 for three-value BVA), each combined with the other password rules.
3	Discount: below ₹1,000 none; ₹1,000–4,999 5 %; ₹5,000 or more 10 %. List partitions and boundary values.	Partitions: <1,000; 1,000–4,999; ≥5,000. Boundaries: 999, 1,000, 4,999, 5,000 (plus a representative such as 500, 2,000, 7,000).

#	Question	Model answer
4	Explain smoke, sanity and regression testing.	Smoke: broad, shallow check that the build is testable. Sanity: narrow check that a fix/change works. Regression: confirm existing behaviour still works after change.
5	What is a charter in exploratory testing?	A short mission statement (what to explore, where, with what focus) for a time-boxed session (usually 60–90 minutes); results and notes are recorded.
6	When may a regression test be retired?	When the feature is removed or the requirement changed, when it duplicates another test, or when it cannot be stabilised and its risk is covered otherwise — always with a recorded reason.
7	Give one entry and one exit criterion for test execution.	Entry: build deployed, smoke test passed, test data loaded. Exit: all critical cases executed and no open Severity 1/2 defect without a written waiver.
8	Who decides whether UAT is accepted?	The client's business owner. QA supports and reports; it does not decide business acceptance.

Level 3 — Web and mobile

#	Question	Model answer
1	Why are emulators not enough for mobile testing?	They approximate hardware, performance, sensors, vendor OS modifications and real network behaviour; final verification needs real devices.
2	After logout you press the browser Back button. What must you verify?	That no protected content is shown and the session is ended (redirect to login), including data in storage.
3	Which cookie attributes do you check on the session cookie?	`Secure`, `HttpOnly` and `SameSite`, plus that the identifier changes at login and is cleared at logout.
4	What contrast ratio does WCAG AA require for normal-size text?	At least 4.5 : 1 (3 : 1 for large text).
5	How do you choose the supported browser matrix?	From the client's analytics, contract and risk — agreed in writing; not “all browsers”.
6	Name three keyboard-accessibility checks.	Every function reachable and operable by keyboard; visible focus indicator; logical tab order; no keyboard trap; Esc closes dialogs.
7	A user double-clicks the Submit button. What do you verify?	Only one request/record is created and the UI prevents duplicate submission (disabled button or idempotent handling).
8	What precaution do you take before sharing a HAR file?	Remove tokens, cookies, personal data and other secrets, or share only a redacted copy through approved channels.

Level 4 — API

#	Question	Model answer
1	Which status for a request without a token, and which for a valid user lacking permission?	401 Unauthorized for missing/invalid authentication; 403 Forbidden for authenticated but not permitted.
2	Which HTTP methods are idempotent?	GET, PUT, DELETE (and HEAD, OPTIONS). POST is not; PATCH is not guaranteed.
3	You send DELETE twice for the same resource. What do you expect?	First 204 (or 200); second 404 because the resource is already gone — unless the specification states otherwise.
4	Why add schema validation when you already check the status code?	Schema checks catch missing fields, wrong types or extra fields (for example price returned as text) that a 200 status hides.
5	A malformed request causes a 500 response. What is your conclusion?	A defect: client mistakes must produce a 4xx with a safe error body; 5xx indicates unhandled server behaviour and may leak internals.
6	Which header normally accompanies 201 Created?	`Location`, giving the URL of the new resource.
7	How do you treat tokens in defect reports and repositories?	Never include or commit them; redact in evidence; keep in environment variables or secret stores.
8	Explain safe versus idempotent.	Safe: does not change server state (GET). Idempotent: repeating the request leaves the same state (PUT, DELETE). All safe methods are idempotent, not vice versa.

Level 5 — Automation

#	Question	Model answer
1	When is automating a manual test worth it?	When (manual minutes per run × number of runs) exceeds (build time + maintenance), the test is stable, repeatable and has a precise expected result.
2	Why must you avoid fixed sleeps?	They make tests slow and flaky; use auto-waiting actions and web-first assertions or explicit waits on conditions.
3	In the Page Object Model, where do assertions go?	In the tests. Page objects hold locators and actions only.
4	What is the purpose of `xfail(strict=True)` with a defect ID?	Records a known defect as an expected failure; when fixed, the test passes and strict mode fails the run so the marker is removed and the defect closed.
5	Where do credentials for CI tests live?	In the CI secret store / environment variables — never in code, YAML or the repository.
6	What does test independence mean?	Each test creates its own data and can run alone, in any order and in parallel.

#	Question	Model answer
7	Which locators do you prefer?	Role, label, text or dedicated test ids; avoid positional XPath and long CSS chains.
8	How do you detect a flaky test?	Rerun it many times (e.g. 10) on the same commit, run in random/parallel order, inspect traces; any inconsistent result marks it flaky.

Level 6 — Advanced QA

#	Question	Model answer
1	Why report percentiles rather than the average response time?	Averages hide the slow tail; p95/p99 show what the worst-affected users experience.
2	Using N = X × R, how many concurrent sessions arise from 2 new sessions per second lasting 90 s?	N = 2 × 90 = 180 concurrent sessions.
3	Differentiate load, stress, spike and soak tests.	Load: expected peak. Stress: beyond peak to the breaking point. Spike: sudden surge. Soak: long duration to find leaks/degradation.
4	Where may you run a performance test?	Only in an environment authorised in writing, under controlled conditions with monitoring and a stop contact — not production or third-party systems without explicit approval.
5	What is QA's role in security testing?	Run agreed safe checks, escalate observations privately to the named security coordinator, retest fixes and report risk; never exploit or probe beyond the authorised scope.
6	Write the idea of a SQL check for orphan records.	LEFT JOIN child to parent and select child rows where the parent key IS NULL; expected result: zero rows.
7	Does 100 % code coverage prove quality?	No. It shows lines executed, not that behaviour is asserted correctly or that requirements are covered.
8	State the flaky-test policy.	Detect by rerun; log in the register; quarantine ≤ 10 working days; fix or delete; report the flaky rate; retries must be counted and reported.

Metrics, costing and conduct

#	Question	Model answer
1	450 cases planned; 360 passed, 40 failed, 20 blocked, 30 not run. Calculate execution % and pass rate.	Executed = 400 → execution % = 400 ÷ 450 = 88.9 %. Pass rate = 360 ÷ 400 = 90.0 %.
2	45 defects found in QA and 5 after release. Calculate defect leakage.	5 ÷ (45 + 5) = 10 %.
3	Delivery cost is ₹3,00,000 and the target gross margin is 40 %. What price do you quote?	Price = 3,00,000 ÷ (1 − 0.40) = ₹5,00,000 (before taxes).
4	The client asks you to mark blocked tests as “passed” to hit a target. What do you do?	Refuse, explain the metric formula, report blocked tests honestly, and escalate to the QA Lead/Test Manager if pressure continues.
5	You discover a client's password in a screenshot you committed. What do you do?	Tell the QA Lead immediately, have the secret revoked/rotated, remove it per the approved procedure, record an incident note.

Final practical assessment (60 marks)

Task	Marks	Standard
Design and execute 10 test cases on an unseen build with seeded defects	15	At least 60 % of seeded defects found; reports reproducible
Automate 3 UI and 3 API checks from a written specification	15	Clean code; independent tests; green run
Prepare a release QA summary from supplied raw data	15	Metrics recomputed correctly; honest risk statement
Oral examination (15 minutes)	15	Explains choices; handles ethics scenarios (metric pressure, secrets, authorisation)

ETHICS SCENARIOS ARE MANDATORY IN THE ORAL EXAM

At least two scenarios from: pressure to change a severity, request to test an unauthorised system, discovery of production data in a test environment, a metric that can be improved by a shortcut, a committed secret. A trainee who would compromise integrity does not pass, whatever their technical score.

20. Templates and QA Sign-off Checklist

Controlled templates for every document in the lifecycle. Copy the Markdown or CSV version from `qa-project-template/` into the project repository; the forms below show the required fields. Grey hint text describes what to enter — delete it when filling in.

HOW TO USE A TEMPLATE

- Copy it, fill every field or write "N/A — reason"; never leave a field silently blank.
- Keep the header block (ID, version, status, author, reviewer, date) and update it on every change.
- Store the file in the path shown; commit with a message such as `docs(test-plan): v0.2 add risks`.
- Evidence and attachments are referenced by file name or link, with secrets removed.

Test Strategy

Purpose: Defines how testing will be done and why (Stage 03). **Repository file:** `docs/test-strategy.md`

Document ID / title	*e.g. QA-TP-001 · Test Plan*
Version / status	*0.1 draft · in review · approved*
Author / reviewer / approver	*Names and dates*
Project / client / build	*Names and build identifiers*
1. Purpose and scope	*What quality objectives does this strategy serve? What is in/out of scope?*
2. Product overview	*Users, key workflows, architecture summary, integrations.*

3. Risks and test responses

ID	Risk	Impact	Likelihood	Test response	Owner

4. Test levels and types	*Which of: functional, regression, integration, E2E, API, database, cross-browser, mobile, accessibility, performance, security coordination, UAT support.*
5. Environments and test data	*Environments, owners, refresh rules, synthetic data approach, privacy rules.*
6. Browser / device / OS matrix	*Taken from client analytics; mark minimum versions.*
7. Automation approach	*What is automated, tools, framework, CI, reporting, maintenance.*

8. Tools and licences	*Tool, version, licence, cost, approver.*
9. Defect management	*Tracker, workflow, severity/priority scales, triage cadence.*
10. Metrics and reporting	*Metrics with formulas, cadence, audience.*
11. Non-functional targets	*Performance, accessibility (WCAG version/level), reliability targets.*
12. Security-testing boundary	*What QA checks; named security coordinator; escalation route.*
13. Roles and responsibilities	*Who does what; reviewer ≠ author.*
14. Assumptions, dependencies, exclusions	*Written and agreed.*
15. Approvals	*Test Manager, client representative, date, signature.*

Test Plan

Purpose: Turns the strategy into a schedule, resources, criteria and risks (Stage 04). **Repository file:** `docs/test-plan.md`

Document ID / title	*e.g. QA-TP-001 · Test Plan*
Version / status	*0.1 draft · in review · approved*
Author / reviewer / approver	*Names and dates*
Project / client / build	*Names and build identifiers*
1. Introduction and objectives	*What this plan covers and what success means.*
2. Scope	*In scope (features/REQ IDs, platforms) and out of scope.*
3. Test items and builds	*Application versions, build numbers, delivery dates.*
4. Approach	*Types of testing and techniques per area; automation plan.*

5. Schedule and milestones

Milestone	Start	End	Owner	Gate

6. Resources	*Roles, names, allocation (hours/percent).*
7. Test environment	*URLs, accounts, data, tools; link to test-environment.md.*

8. Entry criteria	*Measurable, e.g. build deployed, smoke passed, data loaded.*
9. Exit criteria	*Measurable, e.g. 100 % critical cases executed; no open S1/S2 without written acceptance.*
10. Suspension / resumption criteria	*When testing stops and what restarts it.*
11. Deliverables	*Which documents and reports will be produced and when.*
12. Communication and reporting	*Daily status, weekly report, triage meetings.*
13. Change control	*How scope/schedule changes are raised, estimated and approved.*
14. Approvals	*Test Manager, client PM, date, signature.*

Risks and mitigations

ID	Risk	Mitigation	Owner	Due

Test Scenario

Purpose: High-level conditions to test; cases are derived from them. **Repository file:** `test-cases/scenarios.csv`

Scenario ID	*TS-<area>-<nn>*
Title	*Verify that ...*
Requirement ID(s)	*REQ-nnn*
Priority / type	*High·Medium·Low / functional·negative·...*
Derived test case IDs	*TC-...*

Scenario list

Scenario ID	Title (Verify that ...)	REQ ID(s)	Priority	Type	Derived TC IDs

Test Case

Purpose: One precise, repeatable verification. **Repository file:** `test-cases/functional/<feature>.csv`

Test case ID	*TC-<area>-<nnn>*
Title	*One verification, stated positively*
Requirement ID(s)	*REQ-nnn*
Module / type / technique	*e.g. Registration / functional / BVA*
Priority	*High · Medium · Low*
Preconditions	*State of system and data before the test*
Test data	*Exact values*
Steps	*1. … 2. … 3. …*
Expected result	*Observable and measurable*
Postconditions	*Clean-up needed*
Automation candidate	*Yes / No / Done (link)*
Author / reviewer / date	

Execution record

Build	Date	Tester	Result (Pass/Fail/Blocked)	Evidence file	Defect ID

Defect Report

Purpose: Reproducible description of a failure (Stage 08). **Repository file:** `defects/DEF-nnn.md`

Defect ID / title	*DEF-nnn · what is wrong and where*
Reporter / date	
Environment	*Build, URL, browser/device, OS, data*
Module / requirement / test case	*REQ-nnn · TC-…*
Severity / priority (proposed)	*S1–S4 / High·Medium·Low*
Steps to reproduce	*Numbered; start from a known state*
Expected result	*With requirement reference*
Actual result	*What happened*

Frequency	*e.g. 3 of 3 attempts*
Evidence	*Screenshot / video / HAR / log file names (secrets removed)*
Workaround	*If any*
Status / assignee	*New · Assigned · Fixed · Retest · Closed · Reopened · Rejected · Duplicate · Deferred*
Fixed in build / retest result	

Test Execution Report

Purpose: Results of an execution cycle (Stages 07 and 12). **Repository file:** `docs/test-execution-report.md`

Document ID / title	*e.g. QA-TP-001 · Test Plan*
Version / status	*0.1 draft · in review · approved*
Author / reviewer / approver	*Names and dates*
Project / client / build	*Names and build identifiers*
1. Summary	*Period, build, environment, overall status (one paragraph).*
2. Scope tested / not tested	*Be explicit about gaps.*

3. Results

Planned	Executed	Passed	Failed	Blocked	Not run	Execution %	Pass rate

4. Defect summary	*Open/closed by severity; list S1/S2 with IDs.*
5. Coverage	*Requirement coverage %, automation coverage %, formulas stated.*
6. Environment and data issues	*Incidents affecting results.*
7. Risks and recommendations	*Facts, interpretation, recommendation separated.*
8. Evidence index	*Where evidence files are stored.*
9. Approvals	*QA Lead, Test Manager.*

Requirements Traceability Matrix

Purpose: Maps requirements to test cases, automation and defects. **Repository file:** `docs/requirements-traceability.md`

Rules	*Every requirement has ≥ 1 test case; every test case has a requirement; update on every change; coverage = requirements with ≥ 1 executed case ÷ total.*

Matrix

REQ ID	Requirement (short)	Risk	Test case IDs	Automated?	Last result	Defect IDs	Coverage status

Automation Coverage Report

Purpose: What is automated, stable and running (Stages 09 and 16). **Repository file:** `docs/automation-coverage.md`

Document ID / title	*e.g. QA-TP-001 · Test Plan*
Version / status	*0.1 draft · in review · approved*
Author / reviewer / approver	*Names and dates*
Project / client / build	*Names and build identifiers*
Summary metrics	*Total cases · automated · automation % (state denominator) · pass rate · suite runtime · flaky rate · last CI run link*
Candidates backlog	*Next tests to automate with ROI estimate*
Flaky register summary	*Open flaky tests, owners, ages*
Maintenance effort	*Hours this period*
Risks and limitations	*What automation does not cover*

Coverage by module

Module	Total cases	Automated	Automation %	Stable	Flaky	Notes

Performance Test Report

Purpose: Workload, thresholds, results and limitations (Stage 11). **Repository file:** `docs/performance-report.md`

Document ID / title	*e.g. QA-TP-001 · Test Plan*
Version / status	*0.1 draft · in review · approved*
Author / reviewer / approver	*Names and dates*
Project / client / build	*Names and build identifiers*
1. Objectives	*Questions this test answers.*
2. Environment	*Hardware, software, data volume, differences from production.*
3. Workload model	*Users, arrival rate, think time, scenarios, calculation.*
4. Tools and scripts	*Tool, version, script path, commit.*
5. Thresholds	*e.g. p95 < 500 ms; errors < 1 %.*

6. Results

Scenario	VUs	Throughput (req/s)	p50 (ms)	p95 (ms)	p99 (ms)	Error %

7. Server metrics	*CPU, memory, disk, network, connections.*
8. Analysis and bottlenecks	*Evidence-based; no speculation.*
9. Verdict against thresholds	*Pass / fail per threshold.*
10. Limitations	*What these results do not prove.*
11. Recommendations	*Actions with owners.*
12. Approvals	

UAT Report

Purpose: Client acceptance results (Stage 13). **Repository file:** `docs/uat-report.md`

Document ID / title	*e.g. QA-TP-001 · Test Plan*
Version / status	*0.1 draft · in review · approved*

Author / reviewer / approver	*Names and dates*
Project / client / build	*Names and build identifiers*
Scope and participants	*Business areas, UAT users, dates.*
Environment and data	*URL, build, accounts.*
Outstanding issues and decisions	*Each with owner and agreed action.*
Acceptance statement	*Accepted · accepted with conditions · not accepted.*
Signatures	*Client business owner, QA Lead, date.*

UAT results

Script ID	Business scenario	User	Result	Issue ID	Comment

Release QA Sign-off

Purpose: Evidence-based recommendation and unresolved-risk summary (Stage 14). **Repository file:** `docs/qa-signoff.md`

Document ID / title	*e.g. QA-TP-001 · Test Plan*
Version / status	*0.1 draft · in review · approved*
Author / reviewer / approver	*Names and dates*
Project / client / build	*Names and build identifiers*
Release / build	*Version, date, environment.*
Evidence summary	*Execution %, pass rate, coverage, open defects by severity, automation result, performance and accessibility findings (with report links).*
Exit criteria check	*Each criterion from the plan: met / not met.*
Unresolved risks and waivers	*Risk, impact, owner, decision, date.*
QA recommendation	*☐ Go ☐ Conditional Go (conditions: …) ☐ No-Go*
Reviewer (second person)	*Name, date, comments.*
Client release decision	*Name, decision, date.*

Open defects at sign-off

ID	Severity	Summary	Decision (fix/waive)	Approver

Test Environment Description

Purpose: Describes the environment, accounts, data and tools (Stage 06). **Repository file:** `docs/test-environment.md`

Document ID / title	*e.g. QA-TP-001 · Test Plan*
Version / status	*0.1 draft · in review · approved*
Author / reviewer / approver	*Names and dates*
Project / client / build	*Names and build identifiers*

1. Environments

Name	URL	Purpose	Owner	Build / version

2. Accounts and access	*Role, account name (never passwords), who authorised it in writing, expiry.*
3. Test data sets	*Synthetic only; purpose and refresh rule of each set.*
4. Browser / device / OS versions	*Exact versions installed or leased.*
5. Monitoring and logs	*Where testers and developers can see logs and metrics.*
6. Refresh, backup and reset rules	*Who resets, how often, how testers are told.*
7. Known limitations	*Differences from production; unavailable integrations (stubbed).*
8. Readiness checklist (Gate G6)	*☐ build deployed ☐ smoke passed ☐ data loaded ☐ accounts verified ☐ no production data present*
9. Approvals	*QA Lead, client technical lead, date.*

Client Qualification Record

Purpose: Go / no-go for a new engagement (Stage 01). **Repository file:** `docs/client-qualification.md`

Client / contact / decision maker	
Product and users	
Technology stack and environments	
Release cadence	
Testing needs and services wanted	
Existing test assets / tools	
Compliance / accessibility needs	
Timeline and budget range	
Red flags (guarantees, production testing, no environment ...)	
NDA signed (date)	
Capacity and skills check	
Decision and approver	*Proceed · decline · request more information*

QA sign-off checklist

Complete before every release recommendation (Stage 14). Every box ticked needs a reference to evidence; the second reviewer re-checks a sample.

Scope and plan

- [ ] Approved test strategy and plan are in the repository, versioned.
- [ ] Scope changes since approval are recorded and approved.
- [ ] Entry criteria were met before execution began (Gate G6).

Execution

- [ ] All planned critical cases executed; counts reconcile (passed + failed + blocked + not run = planned).
- [ ] Every result has evidence; evidence contains no secrets or personal data.
- [ ] Regression exit criteria met (Gate G12).

Defects

- [ ] All Severity 1 and 2 defects are fixed and verified, or waived in writing by the product owner.
- [ ] Every deferred defect is in the risk summary with an owner.
- [ ] No defect was closed without a verified retest.

Coverage and automation

- [ ] Requirement coverage computed and uncovered requirements listed.
- [ ] Automation results attached; flaky tests reported separately.
- [ ] Known defects marked as expected failures reference defect IDs.

Non-functional

- [ ] Performance report approved, with limitations stated (or explicitly out of scope).
- [ ] Accessibility findings listed against WCAG criteria (or out of scope).
- [ ] Security observations handed to the named coordinator.

UAT and release

- [ ] UAT acceptance signed (or not required by contract).
- [ ] Production smoke list, accounts, window and rollback decision-maker agreed.
- [ ] Second reviewer checked numbers against raw data.

Records

- [ ] Metrics have formulas, dates, builds and sources.
- [ ] Recommendation is Go / Conditional Go / No-Go and follows from the evidence.
- [ ] Evidence is archived per the retention rule; no real customer data exists in the repository.

Role	Name	Signature	Date	Decision
QA Lead				
Second reviewer (Test Manager)				
Client release owner				

21. Appendices

Official learning sources, the licence register, a glossary, daily checklists and credits.

A. Official learning sources

VERIFICATION STATUS

These addresses were supplied by management. During the build of this handbook the environment's network policy blocked access to these websites, so **page content was not cross-checked against live documentation**. Package licences and versions were verified from the npm and PyPI registries (see section B). Before teaching or quoting, open each source and confirm current instructions, licences and prices.

Source	Address	Use it for	Note
ISTQB	https://www.istqb.org/	Syllabi, glossary and certification path (Foundation Level first).	Terminology and syllabus in this handbook follow ISTQB concepts; read the current Foundation syllabus.
Playwright	https://playwright.dev/docs/intro	Installation, locators, assertions, tracing, CI.	Check the current API names before copying code.
Selenium	https://www.selenium.dev/documentation/	WebDriver, Grid, language bindings.	Selenium Manager handles drivers in current versions.
Cypress	https://docs.cypress.io/	Runner, commands, component testing, Cypress Cloud.	Check which features are paid.
Postman Learning Center	https://learning.postman.com/	Collections, tests, environments, Newman, monitors.	Plan limits change; check current terms.
pytest	https://docs.pytest.org/	Fixtures, parametrisation, markers, plugins.	Check plugin compatibility with your pytest version.
k6	https://k6.io/docs/	Scripting, thresholds, scenarios, results output.	The k6 documentation may redirect to Grafana's site; confirm the licence terms there.
Apache JMeter	https://jmeter.apache.org/usermanual/	Test plan elements, non-GUI mode, reports.	Requires a supported Java version.

Source	Address	Use it for	Note
W3C WAI	https://www.w3.org/WAI/	WCAG, ARIA, evaluation methods.	Confirm which WCAG version and level the contract demands.
GitHub Docs	https://docs.github.com/	Repositories, Actions, secrets, branch protection.	Free-tier limits change; check before quoting.

Supplementary references for awareness: OWASP Top 10 and ASVS (application security), the WCAG quick-reference and Understanding documents from W3C WAI, and the ISTQB glossary.

B. Licence register (registry-verified on 4 November 2026)

Versions are the latest at the time of checking and will move on. Record the exact versions you use in each project's `THIRD_PARTY.md`.

Package	Version	Licence
Playwright (npm / PyPI)	1.64.1	Apache-2.0
Selenium (selenium-webdriver / selenium)	4.52.0	Apache-2.0
Cypress	16.2.1	MIT
pytest	9.2.0	MIT
requests	2.34.2	Apache-2.0
jsonschema	4.26.0	MIT
pytest-playwright	0.10.0	Apache-2.0
pytest-html	4.2.0	MPL-2.0
pytest-xdist	3.8.0	MIT
Locust	2.47.0	MIT
Newman	6.2.3	Apache-2.0
axe-core / @axe-core/playwright	4.14.0	MPL-2.0
Lighthouse	13.6.0	Apache-2.0
Lighthouse CI (@lhci/cli)	0.16.0	Apache-2.0
Appium	3.9.0	Apache-2.0
WebdriverIO	10.1.0	MIT
Pa11y	10.0.0	LGPL-3.0-only
Allure Commandline	2.46.1	Apache-2.0

Package	Version	Licence
@sonar/scan (SonarQube scanner)	5.0.2	LGPL-3.0-only

Not verified from registries (check the vendor's official page): k6 (believed AGPL-3.0), Apache JMeter (believed Apache-2.0), SonarQube Community Build, Postman application and plans, Jira, TestRail, Git, GitHub and GitHub Actions allowances, NVDA, Kiwi TCMS, device-cloud services.

C. Daily tester checklist

- [ ] Read yesterday's journal entry and today's plan; confirm the build number.
- [ ] Smoke test passed before deeper execution.
- [ ] Every executed case has a result and evidence.
- [ ] Every failure reproduced twice and logged before end of day.
- [ ] No secrets or personal data in evidence or messages.
- [ ] Blockers raised in stand-up, not at the end of the week.
- [ ] Journal entry written and committed.

D. Pre-submission checklist for any deliverable

- [ ] Header block complete: ID, version, status, author, reviewer, date.
- [ ] All numbers re-computed from raw data by me and by a reviewer.
- [ ] Every claim has evidence or is marked as an assumption.
- [ ] Gaps and untested areas stated plainly.
- [ ] Spelling and terminology consistent with the glossary.
- [ ] Saved at the correct repository path and committed with a clear message.

E. Glossary

Term	Meaning
Acceptance criteria	Conditions a feature must satisfy to be accepted by the client.
Boundary-value analysis (BVA)	Technique testing values at and next to the edges of valid ranges.
Build	A specific compiled/packaged version of the software under test.
CI/CD	Continuous integration / delivery: automated build, test and deployment on every change.
Defect (bug)	A flaw in a work product that can cause a failure.
Defect leakage	Share of defects that escape QA and are found after release.
Entry / exit criteria	Conditions to start / finish a test activity.

Term	Meaning
Equivalence partitioning (EP)	Technique that groups inputs treated the same and tests one value per group.
Exploratory testing	Simultaneous learning, test design and execution guided by a charter.
Failure	Observable deviation of actual from expected behaviour.
Fixture	Reusable setup/teardown code supplied to tests.
Flaky test	A test that passes and fails on the same code without changes.
Idempotent	Repeating the operation leaves the system in the same state.
JSON Schema	A standard vocabulary for describing and validating JSON structure.
Page Object Model	Design pattern that wraps page locators and actions in classes.
Percentile (p95)	Value below which the stated percentage of measurements fall.
Regression testing	Re-testing to ensure changes did not break existing functionality.
Requirements Traceability Matrix	Table mapping requirements to tests, results and defects.
Risk-based testing	Choosing and prioritising tests by the likelihood and impact of failure.
Sanity testing	Narrow check that a specific change or fix works.
Severity / priority	Technical impact of a defect / business urgency to fix it.
Smoke testing	Broad, shallow check that a build is stable enough to test.
SLA	Service-level agreement: measurable service commitments.
Test case / scenario / suite	A precise verification / a high-level condition / a collection of cases.
Test environment	Hardware, software, data and configuration in which tests run.
Throughput	Requests or transactions processed per unit time.
UAT	User acceptance testing by the client's business users.
Virtual user (VU)	A simulated user in a load test.
WCAG	Web Content Accessibility Guidelines published by the W3C.
xfail	pytest marker for a test expected to fail because of a known defect.

G. Companion source files and what was verified

Path (in the `vel-qa-department` repository)	Contents
`dist/`	This handbook (PDF + standalone HTML), 90-day training plan, templates pack, practical workbook, assessment framework (PDFs).
`build/`	Document build system: content modules (Python), theme CSS, web layer, PDF renderer, font subsetting. `python3 build/build.py` rebuilds every deliverable.
`qa-project-template/`	The professional QA repository skeleton, controlled templates (Markdown/CSV), starter automation, CI workflow, k6 and Locust scripts.
`qa-project-template/practice-app/`, `practice-api/`	VEL Practice Shop and Practice API: runnable training targets with deliberate defects and written specifications.
`instructor-only/`	Answer key and reference solution. Not for trainees.

Item	Status in the build environment
Practice Shop, Practice API	**Executed.** Seeded defects confirmed by automated tests (strict `xfail`).
Starter and reference pytest + Playwright suites	**Executed** on Chromium against both practice targets, builds 1.0 and 1.1.
axe-core accessibility scan	**Executed** on the practice app.
Locust load script	**Executed** (10 virtual users, 15 s, local practice API): 0 failures.
k6 script	Syntax-checked only; k6 is not installed in the build environment.
JMeter, SonarQube, Postman, Jira, TestRail, Selenium and Cypress examples	Described and illustrated; **not executed** here.
GitHub Actions workflow	YAML parsed and validated; **not run** on GitHub.
Firefox / WebKit runs	Not executed (only Chromium was available).
Official documentation websites	Not reachable from the build environment; content not cross-checked online (see section A).

F. Credits and font licences

Typography: Inter (SIL Open Font License 1.1) and Liberation Serif/Mono (SIL Open Font License 1.1), embedded as subsets. The VEL Software Solutions logo is the property of VEL Software Solutions and is used as cover branding and a subtle watermark without distortion.